

**ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ**



ĐỒ ÁN TỐT NGHIỆP

**XÂY DỰNG WEBSITE GỢI Ý THỰC
ĐƠN DINH DƯỠNG THÔNG MINH**

Giảng viên hướng dẫn : **TS ĐOÀN PHƯỚC MIỀN**

Sinh viên thực hiện: **LÊ HÀ DUY**

Mã số sinh viên: **110122060**

Lớp : **DA22TTD**

Khoá : **2022**

Vĩnh Long, tháng 6 năm 2026

**ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ**



ĐỒ ÁN TỐT NGHIỆP

**XÂY DỰNG WEBSITE GỢI Ý THỰC
ĐƠN DINH DƯỠNG THÔNG MINH**

Giảng viên hướng dẫn : TS ĐOÀN PHƯỚC MIỀN

Sinh viên thực hiện: LÊ HÀ DUY

Mã số sinh viên: 110122060

Lớp : DA22TTD

Khoá : 2022

Vĩnh Long, tháng 6 năm 2026

LỜI MỞ ĐẦU

1. Lý do chọn đề tài

Trong bối cảnh công nghệ số đang phát triển mạnh mẽ, việc ứng dụng công nghệ thông tin vào đời sống hàng ngày ngày càng trở nên phổ biến và cần thiết. Đặc biệt, trong lĩnh vực chăm sóc sức khỏe và dinh dưỡng, nhu cầu xây dựng một chế độ ăn uống khoa học, hợp lý và phù hợp với từng cá nhân đang được nhiều người quan tâm. Tuy nhiên, trên thực tế, phần lớn người dùng vẫn gặp khó khăn trong việc lựa chọn thực đơn phù hợp với thể trạng, mục tiêu sức khỏe cũng như điều kiện tài chính của bản thân.

Hiện nay, mặc dù đã có nhiều website và ứng dụng hỗ trợ về dinh dưỡng, nhưng đa số vẫn mang tính chất cung cấp thông tin chung, thiếu khả năng cá nhân hóa và chưa tận dụng hiệu quả các công nghệ hiện đại như trí tuệ nhân tạo. Điều này dẫn đến việc người dùng phải tự tìm kiếm, chọn lọc thông tin, gây mất thời gian và đôi khi không đạt được hiệu quả như mong muốn.

Xuất phát từ những hạn chế trên, việc xây dựng một hệ thống gợi ý thực đơn dinh dưỡng thông minh là hết sức cần thiết. Hệ thống không chỉ giúp người dùng dễ dàng tiếp cận với các thực đơn phù hợp mà còn có khả năng cá nhân hóa dựa trên nhu cầu, sở thích và mục tiêu sức khỏe của từng người. Đồng thời, việc tích hợp trí tuệ nhân tạo vào hệ thống sẽ góp phần nâng cao độ chính xác, tối ưu hóa trải nghiệm người dùng và hướng đến việc chăm sóc sức khỏe một cách chủ động, hiệu quả hơn.

2. Mục đích nghiên cứu

Đề tài hướng đến việc xây dựng một website gợi ý thực đơn dinh dưỡng thông minh, đóng vai trò như một trợ lý ảo hỗ trợ người dùng trong việc thiết lập và quản lý chế độ ăn uống hàng ngày một cách khoa học và hợp lý.

Cụ thể, hệ thống được xây dựng nhằm đạt được các mục tiêu sau:

- Cho phép người dùng đăng ký, đăng nhập và quản lý thông tin cá nhân, bao gồm các chỉ số liên quan đến sức khỏe như BMI, mục tiêu dinh dưỡng và mức chi tiêu.
- Phân tích dữ liệu người dùng để đề xuất các thực đơn phù hợp với nhu cầu cá nhân, đảm bảo cân bằng dinh dưỡng và tối ưu chi phí.
- Ứng dụng trí tuệ nhân tạo để tự động xây dựng lộ trình thực đơn theo tuần, giúp người dùng tiết kiệm thời gian trong việc lên kế hoạch ăn uống.

- Tích hợp chatbot AI hỗ trợ tư vấn, giải đáp các thắc mắc liên quan đến dinh dưỡng thông qua ngôn ngữ tự nhiên.
- Lưu trữ và quản lý lịch sử thực đơn, danh sách món ăn yêu thích nhằm nâng cao khả năng cá nhân hóa trong quá trình sử dụng.
- Xây dựng hệ thống quản trị giúp quản lý dữ liệu món ăn, nguyên liệu và người dùng một cách hiệu quả, đảm bảo hệ thống vận hành ổn định.

Thông qua việc phát triển hệ thống này, đề tài không chỉ góp phần hỗ trợ người dùng cải thiện thói quen ăn uống mà còn hướng đến việc ứng dụng hiệu quả công nghệ trí tuệ nhân tạo trong lĩnh vực chăm sóc sức khỏe, từ đó nâng cao chất lượng cuộc sống và thúc đẩy xu hướng sống lành mạnh trong cộng đồng.

LỜI CẢM ƠN

Trước tiên, tôi xin gửi lời cảm ơn chân thành đến quý thầy, cô Đại học Trà Vinh nói chung và các thầy, cô Khoa Công nghệ Thông tin thuộc Trường Kỹ thuật và Công nghệ nói riêng. Trong suốt thời gian học tập tại trường, tôi đã nhận được rất nhiều sự hướng dẫn và hỗ trợ tận tình từ thầy cô. Những kiến thức và kinh nghiệm mà thầy cô truyền đạt đã giúp tôi có nền tảng vững chắc, đồng thời tạo điều kiện để tôi tiếp cận thực tế và định hướng rõ hơn cho công việc sau này.

Tôi xin bày tỏ lòng biết ơn đến Thầy Đoàn Phước Miên – người đã trực tiếp hướng dẫn tôi trong quá trình thực hiện khóa luận tốt nghiệp. Trong quá trình làm đề tài, có những lúc tôi gặp khó khăn và chưa xác định rõ hướng đi, nhưng nhờ sự chỉ dẫn, góp ý và động viên kịp thời của Thầy, tôi đã từng bước hoàn thiện bài làm của mình. Sự tận tâm và trách nhiệm của Thầy là điều tôi luôn trân trọng.

Bên cạnh đó, tôi cũng xin cảm ơn bạn bè, anh chị và những người đã hỗ trợ, chia sẻ kinh nghiệm cũng như động viên tôi trong suốt quá trình thực hiện khóa luận. Sự giúp đỡ này đã giúp tôi có thêm động lực để hoàn thành tốt nhiệm vụ của mình.

Do thời gian và kiến thức còn hạn chế, bài khóa luận của tôi chắc chắn vẫn còn những thiếu sót. Tôi rất mong nhận được sự góp ý từ quý thầy, cô để có thể rút kinh nghiệm và hoàn thiện hơn trong thời gian tới.

Tôi sẽ tiếp tục cố gắng học tập, rèn luyện và phát triển bản thân để đáp lại sự tin tưởng và kỳ vọng của thầy cô và mọi người.

Xin chân thành cảm ơn!

Vĩnh Long, ngày tháng năm 2025

Lê Hà Duy

MỤC LỤC

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT	9
1.1 ReactJS	9
1.1.1 Khái niệm.....	9
1.1.2 Ưu điểm của ReactJs	10
1.1.3 Nhược điểm của ReactJS	10
1.2 NodeJS	11
1.2.1 Khái niệm.....	11
1.2.2 Ứng dụng	11
1.2.3 Ưu điểm của NodeJS	12
1.2.4 Nhược điểm của NodeJS	13
1.3 ExpressJS	14
1.3.1 Khái niệm.....	14
1.3.2 Tính năng của ExpressJS	14
1.3.3 Ưu điểm của ExpressJS	15
1.3.4 Nhược điểm của ExpressJS	15
1.4 RESTful API	16
1.4.1 Tổng quan về RESTful API.....	16
1.4.2 Các thành phần chính của RESTful API	17
1.4.3 Các nguyên tắc của RESTful API	17
1.4.4 Ưu điểm của RESTful API.....	18
1.4.5 Nhược điểm của RESTful API	19
1.5 JavaScript	19
1.5.1 Khái niệm.....	19
1.5.2 Ứng dụng	20
1.5.3 Ưu điểm của JavaScript	21
1.5.4 Nhược điểm của JavaScript	21
1.6 MongoDB	22
1.6.1 Khái niệm.....	22
1.6.2 Tính năng nổi bật của MongoDB	22
1.6.3 Ưu điểm của MongoDB.....	23
1.6.4 Nhược điểm của MongoDB.....	23
1.7 Các nghiệp vụ liên quan đến đề tài	24
1.7.1 Nghiệp vụ quản lý người dùng	24
1.7.2 Nghiệp vụ quản lý thực đơn dinh dưỡng	25
1.7.3 Nghiệp vụ gợi ý thực đơn thông minh.....	25
1.7.4 Nghiệp vụ quản lý dữ liệu sức khỏe	26
1.7.5 Nghiệp vụ thông báo và nhắc nhở	26
1.7.6 Nghiệp vụ quản trị hệ thống	27
CHƯƠNG 2: HIỆN THỰC HÓA NGHIÊN CỨU.....	28
2.1 Mô tả bài toán	28
2.2 Phân tích chức năng	30
2.2.1 Đối với người quản trị	30
2.2.2 Đối với người dùng.....	30
2.2.3 Yêu cầu phi chức năng	31
2.3 Sơ đồ Use Case	33

2.3.1 Biểu đồ Use Case tổng quát.....	33
2.3.2 Biểu đồ Use Case thiết lập hồ sơ thể chất và mục tiêu dinh dưỡng	34
2.3.3 Biểu đồ Use Case kiến tạo lộ trình thực đơn AI.....	35
2.3.4 Biểu đồ Use Case quản lý nhật ký dinh dưỡng	35
2.3.5 Biểu đồ Use Case tương tác Chatbot AI tư vấn.....	36
2.3.6 Sơ đồ tuần tự chức năng kiến tạo lộ trình thực đơn AI (Sequence diagram)	37
2.4 Xây dựng website	38
2.4.1 Cài đặt môi trường.....	38
2.4.2 Cơ sở dữ liệu.....	38
2.5 Mô tả kiến trúc hệ thống	49
2.5.1 Giao diện người dùng (Client – Frontend)	49
2.5.2 Máy chủ xử lý nghiệp vụ (Server – Backend).....	50
2.5.3 Cơ sở dữ liệu (Database)	51
2.5.4 Luồng xử lý tổng quát.....	51
2.5.5 Ưu điểm kiến trúc	52
CHƯƠNG 3: KẾT QUẢ NGHIÊN CỨU	53
3.1 Giao diện người dùng	53
3.1.1 Giao diện trang chủ.....	53
3.1.2 Giao diện đăng ký.....	54
3.1.3 Giao diện đăng nhập	55
3.1.4 Giao diện khám phá thực đơn.....	56
3.1.5 Giao diện chi tiết món ăn.....	56
3.1.6 Giao diện gợi ý cá nhân hóa	57
3.1.7 Giao diện lộ trình tuần bằng AI	58
3.1.8 Giao diện trợ lý tư vấn dinh dưỡng bằng AI	58
3.1.9 Giao diện thông tin cá nhân	59
3.1.10 Giao diện món ăn yêu thích	60
3.1.11 Giao diện nhật ký thực đơn.....	60
3.2 Giao diện quản lý của Quản trị viên	61
3.2.1 Giao diện tổng quan hệ thống.....	61
3.2.2 Giao diện quản lý danh mục	62
3.2.3 Giao diện quản lý món ăn.....	62
3.2.4 Giao diện quản lý nguyên liệu.....	63
3.2.5 Giao diện quản lý nhật ký thực đơn.....	64
3.2.6 Giao diện quản lý món ăn yêu thích	64
3.2.7 Giao diện quản lý người dùng	65
CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	66
4.1 Kết quả đạt được	66
4.2 Ưu nhược điểm	67
4.2.1 Ưu điểm	67
4.2.2 Nhược điểm	67
4.3 Hướng phát triển	67

NHẬN XÉT

(Của giảng viên hướng dẫn trong đề án, khoá luận của sinh viên)

This image shows a full page of primary-ruled paper. It features multiple sets of horizontal dotted lines spaced evenly down the page, providing a guide for handwriting practice. The background is white, and there are no margins or other markings present.

Giảng viên hướng dẫn
(ký và ghi rõ họ tên)

5. Giá trị thực trên đề tài:

6. Đề nghị sửa chữa bổ sung:

7. Đánh giá:

Vĩnh Long, ngày tháng năm 2026
Giảng viên hướng dẫn
(Ký & ghi rõ họ tên)

NHẬN XÉT
(Của giảng viên chấm trong đề án, khoá luận của sinh viên)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Giảng viên hướng dẫn
(ký và ghi rõ họ tên)

DANH MỤC CÁC BẢNG, HÌNH

Bảng 1. User.....	46
Bảng 2. Meal	46
Bảng 3. MealLog.....	47
Bảng 4. Ingredient.....	47
Bảng 5. Category.....	47
Bảng 6. FavoriteMeal.....	48
Bảng 7. Session	48
Bảng 8. MealPlan	48
Bảng 9. ChatHistory.....	48
Bảng 10. Notification	49
Hình 2.1. Flow Chart hệ thống.....	29
Hình 2.2. Use Case tổng quát.....	33
Hình 2.3. Use Case thiết lập hồ sơ thể chất và mục tiêu dinh dưỡng	34
Hình 2.4. Use Case kiến tạo lộ trình thực đơn AI.....	35
Hình 2.5. Use Case quản lý nhật ký dinh dưỡng	35
Hình 2.6. Use Case tương tác Chatbot AI tư vấn.....	36
Hình 2.7. Sơ đồ tuần tự chức năng kiến tạo lộ trình thực đơn AI.....	37
Hình 3.1. Giao diện trang chủ	53
Hình 3.2. Chức năng cốt lõi ở trang chủ	54
Hình 3.3. Giao diện đăng ký	54
Hình 3.4. Giao diện đăng nhập	55
Hình 3.5. Giao diện khám phá thực đơn	56
Hình 3.6. Giao diện chi tiết món ăn	57
Hình 3.7. Giao diện gợi ý cá nhân hóa.....	57
Hình 3.8. Giao diện lộ trình tuần bằng AI	58
Hình 3.9. Giao diện trợ lý tư vấn dinh dưỡng bằng AI.....	59
Hình 3.10. Giao diện thông tin cá nhân	59
Hình 3.11. Giao diện món ăn yêu thích	60
Hình 3.12. Giao diện nhật ký thực đơn.....	61
Hình 3.13. Giao diện tổng quan hệ thống	61

Hình 3.14. Giao diện quản lý danh mục	62
Hình 3.15. Giao diện quản lý món ăn	63
Hình 3.16. Giao diện quản lý nguyên liệu	63
Hình 3.17. Giao diện quản lý nhật ký thực đơn	64
Hình 3.18. Giao diện quản lý món ăn yêu thích	65
Hình 3.19. Giao diện quản lý người dùng	65

MỞ ĐẦU

1. Lý do chọn đề tài

Trong những năm gần đây, nhu cầu chăm sóc sức khỏe và xây dựng chế độ ăn uống khoa học ngày càng được nhiều người quan tâm. Cùng với sự phát triển của xã hội hiện đại, con người không chỉ chú trọng đến chất lượng cuộc sống mà còn quan tâm nhiều hơn đến vấn đề dinh dưỡng, vóc dáng và phòng ngừa bệnh tật thông qua thói quen ăn uống hợp lý. Tuy nhiên, trên thực tế, không phải ai cũng có đủ kiến thức để xây dựng cho mình một thực đơn phù hợp với thể trạng, nhu cầu dinh dưỡng và điều kiện kinh tế cá nhân.

Hiện nay, phần lớn người dùng thường tìm kiếm thực đơn hoặc thông tin dinh dưỡng thông qua mạng xã hội, các website chia sẻ kinh nghiệm hoặc ứng dụng phổ thông. Tuy nhiên, các nguồn thông tin này còn mang tính chung chung, thiếu sự cá nhân hóa và chưa đáp ứng tốt nhu cầu riêng của từng người dùng. Ngoài ra, việc tự lên kế hoạch ăn uống hằng ngày thường mất nhiều thời gian và khó đảm bảo sự cân bằng dinh dưỡng một cách hợp lý.

Sự phát triển mạnh mẽ của công nghệ thông tin, đặc biệt là Trí tuệ nhân tạo (AI), đã mở ra nhiều hướng tiếp cận mới trong việc xây dựng các hệ thống hỗ trợ thông minh. Việc ứng dụng AI vào lĩnh vực dinh dưỡng không chỉ giúp tự động hóa quá trình đề xuất thực đơn mà còn nâng cao khả năng phân tích và cá nhân hóa nội dung theo nhu cầu của từng người dùng.

Xuất phát từ những lý do trên, tôi quyết định lựa chọn đề tài “Xây dựng website gợi ý thực đơn dinh dưỡng thông minh”. Đề tài hướng đến việc xây dựng một hệ thống hỗ trợ người dùng thiết lập thực đơn phù hợp với mục tiêu sức khỏe, sở thích cá nhân và mức chi tiêu. Đồng thời, hệ thống còn tích hợp chatbot AI nhằm hỗ trợ tư vấn và giải đáp các kiến thức dinh dưỡng một cách trực quan và thuận tiện hơn.

Đề tài không chỉ mang ý nghĩa thực tiễn trong việc hỗ trợ người dùng xây dựng lối sống lành mạnh mà còn góp phần ứng dụng hiệu quả công nghệ AI vào lĩnh vực chăm sóc sức khỏe và dinh dưỡng trong thời đại chuyển đổi số hiện nay.

2. Mục tiêu đề tài

Mục tiêu của đề tài là xây dựng một website gợi ý thực đơn dinh dưỡng thông minh nhằm hỗ trợ người dùng thiết lập chế độ ăn uống phù hợp với nhu cầu cá nhân, góp phần nâng cao sức khỏe và cải thiện chất lượng cuộc sống. Hệ thống được phát triển theo hướng hiện đại, tích hợp công nghệ Trí tuệ nhân tạo (AI) để tăng khả năng cá

nhân hóa và tối ưu trải nghiệm người dùng. Cụ thể, đề tài hướng đến việc xây dựng các chức năng chính như quản lý tài khoản người dùng, cho phép đăng ký, đăng nhập và cập nhật thông tin cá nhân. Người dùng có thể thiết lập hồ sơ dinh dưỡng thông qua các thông tin như chiều cao, cân nặng, chỉ số BMI, mục tiêu sức khỏe và mức chi tiêu, từ đó hệ thống có cơ sở để đưa ra các đề xuất thực đơn phù hợp.

Hệ thống còn hỗ trợ chức năng gợi ý thực đơn thông minh dựa trên sở thích cá nhân, nhu cầu dinh dưỡng và dữ liệu lịch sử sử dụng của người dùng. Website có khả năng tự động xây dựng lộ trình thực đơn theo tuần bằng công nghệ AI, giúp người dùng tiết kiệm thời gian trong việc lựa chọn món ăn hằng ngày nhưng vẫn đảm bảo cân bằng dinh dưỡng và tối ưu chi phí. Bên cạnh đó, đề tài còn hướng đến việc tích hợp chatbot AI hỗ trợ tư vấn và giải đáp các câu hỏi liên quan đến dinh dưỡng thông qua ngôn ngữ tự nhiên. Người dùng có thể dễ dàng trao đổi với hệ thống để nhận được các gợi ý phù hợp về chế độ ăn uống, nguyên liệu hoặc các kiến thức cơ bản liên quan đến sức khỏe và dinh dưỡng.

Hệ thống cũng hỗ trợ lưu trữ lịch sử thực đơn, danh sách món ăn yêu thích và các thông tin tương tác nhằm nâng cao khả năng cá nhân hóa trong quá trình sử dụng. Đồng thời, website cung cấp trang quản trị dành cho quản trị viên nhằm quản lý dữ liệu món ăn, nguyên liệu, tài khoản người dùng và theo dõi hoạt động của hệ thống một cách hiệu quả. Thông qua việc xây dựng hệ thống này, đề tài hướng đến mục tiêu ứng dụng hiệu quả công nghệ AI trong lĩnh vực chăm sóc sức khỏe và dinh dưỡng, đồng thời mang lại cho người dùng một công cụ hỗ trợ xây dựng lối sống khoa học, hiện đại và lành mạnh hơn.

3. Nội dung đề tài

Nội dung của đề tài tập trung vào việc thiết kế và phát triển một website gợi ý thực đơn dinh dưỡng thông minh, hướng đến việc hỗ trợ người dùng xây dựng chế độ ăn uống khoa học và phù hợp với nhu cầu cá nhân. Hệ thống được xây dựng với hai nhóm chức năng chính gồm người dùng và quản trị viên nhằm đảm bảo khả năng vận hành, quản lý và cá nhân hóa dữ liệu hiệu quả.

Đối với người dùng, hệ thống cung cấp các chức năng như đăng ký, đăng nhập và quản lý thông tin cá nhân. Người dùng có thể thiết lập hồ sơ dinh dưỡng thông qua các thông tin như chiều cao, cân nặng, chỉ số BMI, mục tiêu sức khỏe và mức chi tiêu. Dựa trên các dữ liệu này, hệ thống sẽ phân tích và đề xuất các thực đơn phù hợp với

nhu cầu và sở thích của từng người dùng. Ngoài ra, người dùng còn có thể lưu danh sách món ăn yêu thích, theo dõi lịch sử thực đơn và sử dụng chatbot AI để trao đổi, hỏi đáp các vấn đề liên quan đến dinh dưỡng và chế độ ăn uống.

Đối với quản trị viên, hệ thống cung cấp các công cụ quản lý chuyên sâu nhằm kiểm soát và vận hành toàn bộ website. Quản trị viên có thể quản lý dữ liệu món ăn, nguyên liệu, danh mục dinh dưỡng, tài khoản người dùng cũng như thực hiện các thao tác thêm, sửa, xóa dữ liệu thông qua chức năng CRUD. Hệ thống hỗ trợ theo dõi lịch sử hoạt động và thống kê dữ liệu nhằm nâng cao hiệu quả quản lý và tối ưu chất lượng nội dung gợi ý.

Hệ thống được xây dựng dựa trên các công nghệ hiện đại thuộc nền tảng MERN Stack. Phần giao diện người dùng (Frontend) được phát triển bằng React kết hợp HTML, CSS và JavaScript nhằm tạo ra giao diện trực quan, thân thiện và dễ sử dụng. Phần Backend được xây dựng bằng Node.js và Express để xử lý logic nghiệp vụ, xây dựng RESTful API và kết nối với hệ thống AI thông qua giao thức HTTP API. Cơ sở dữ liệu MongoDB được sử dụng để lưu trữ dữ liệu người dùng, món ăn, nguyên liệu, lịch sử thực đơn và các thông tin liên quan khác một cách linh hoạt.

Hệ thống tích hợp các mô hình ngôn ngữ lớn (LLMs) nhằm xây dựng chatbot AI và hỗ trợ chức năng tạo thực đơn tự động theo tuần. Việc áp dụng các công nghệ hiện đại không chỉ giúp nâng cao trải nghiệm người dùng mà còn góp phần tăng khả năng cá nhân hóa và tối ưu hiệu quả hoạt động của hệ thống.

4. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài “Xây dựng website gợi ý thực đơn dinh dưỡng thông minh” là hệ thống website ứng dụng công nghệ Trí tuệ nhân tạo (AI) nhằm hỗ trợ người dùng xây dựng và quản lý chế độ ăn uống phù hợp với nhu cầu cá nhân. Hệ thống được thiết kế để hỗ trợ quá trình gợi ý thực đơn, tư vấn dinh dưỡng và quản lý dữ liệu người dùng một cách hiệu quả, hiện đại và dễ sử dụng.

Cụ thể, đề tài tập trung nghiên cứu các nội dung sau:

Các chức năng nghiệp vụ chính của hệ thống

Bao gồm:

- Quản lý tài khoản người dùng (đăng ký, đăng nhập, xác thực và cập nhật thông tin cá nhân).

- Thiết lập hồ sơ dinh dưỡng (chiều cao, cân nặng, chỉ số BMI, mục tiêu sức khỏe và mức chi tiêu).
- Gợi ý thực đơn phù hợp dựa trên nhu cầu dinh dưỡng, sở thích cá nhân và dữ liệu lịch sử sử dụng.
- Xây dựng lộ trình thực đơn tự động theo tuần bằng công nghệ AI.
- Lưu trữ danh sách món ăn yêu thích và lịch sử lựa chọn thực đơn của người dùng.
- Tích hợp chatbot AI hỗ trợ tư vấn và giải đáp các vấn đề liên quan đến dinh dưỡng.
- Quản lý dữ liệu món ăn, nguyên liệu, danh mục và tài khoản người dùng dành cho quản trị viên.
- Thống kê và theo dõi dữ liệu tương tác nhằm nâng cao hiệu quả hoạt động của hệ thống.

Các quy trình và luồng hoạt động của hệ thống

- Luồng đăng ký, đăng nhập và xác thực người dùng.
- Luồng thiết lập hồ sơ dinh dưỡng và cập nhật dữ liệu cá nhân.
- Luồng phân tích dữ liệu và đề xuất thực đơn phù hợp.
- Luồng tương tác giữa người dùng với chatbot AI.
- Luồng quản lý dữ liệu món ăn, nguyên liệu và danh mục của quản trị viên.
- Luồng lưu trữ và xử lý lịch sử thực đơn, danh sách yêu thích và dữ liệu tương tác.
- Việc nghiên cứu các quy trình trên giúp hệ thống đảm bảo tính logic trong xử lý nghiệp vụ, nâng cao khả năng cá nhân hóa và đáp ứng tốt nhu cầu thực tế của người dùng.

Công nghệ và kỹ thuật áp dụng

Hệ thống được xây dựng dựa trên các công nghệ hiện đại thuộc nền tảng MERN Stack, bao gồm:

- React: Xây dựng giao diện người dùng trực quan và linh hoạt.
- Node.js kết hợp Express: Phát triển backend, xử lý logic nghiệp vụ và xây dựng RESTful API.
- MongoDB: Lưu trữ dữ liệu phi cấu trúc với khả năng mở rộng linh hoạt.

- JWT (JSON Web Token): Hỗ trợ xác thực và phân quyền người dùng an toàn.
- API kết nối với các Mô hình ngôn ngữ lớn (LLMs): Hỗ trợ xây dựng chatbot AI và chức năng tạo thực đơn tự động.
- Thuật toán Hybrid Recommendation: Kết hợp giữa Content-Based Filtering và Collaborative Filtering nhằm nâng cao độ chính xác trong việc gợi ý thực đơn.
- Việc nghiên cứu và áp dụng các công nghệ trên giúp hệ thống đảm bảo hiệu năng, khả năng mở rộng, bảo mật và nâng cao trải nghiệm người dùng.

Kiến trúc hệ thống và mô hình dữ liệu: Đề tài cũng tập trung nghiên cứu mô hình kiến trúc client-server và cách tổ chức luồng dữ liệu giữa frontend, backend và cơ sở dữ liệu. Ngoài ra, hệ thống sử dụng cơ sở dữ liệu MongoDB với các collection như User, Meal, Ingredient, Category, FavoriteMeal và MealLog nhằm tổ chức và quản lý dữ liệu một cách hợp lý.

Thông qua việc nghiên cứu toàn diện các nội dung trên, đề tài hướng đến xây dựng một hệ thống website gợi ý thực đơn dinh dưỡng thông minh có tính ứng dụng thực tiễn cao, hỗ trợ người dùng xây dựng chế độ ăn uống khoa học và góp phần nâng cao chất lượng cuộc sống.

5. Phạm vi và đối tượng nghiên cứu

Đề tài tập trung nghiên cứu và xây dựng một website gợi ý thực đơn dinh dưỡng thông minh nhằm hỗ trợ người dùng thiết lập chế độ ăn uống khoa học và phù hợp với nhu cầu cá nhân. Hệ thống được phát triển trên nền tảng web với mục tiêu hỗ trợ người dùng trong việc quản lý dinh dưỡng, xây dựng lộ trình ăn uống và nâng cao chất lượng cuộc sống thông qua việc ứng dụng công nghệ Trí tuệ nhân tạo (AI).

Đối tượng nghiên cứu của đề tài bao gồm người dùng có nhu cầu cải thiện chế độ ăn uống như sinh viên, nhân viên văn phòng, người tập luyện thể thao hoặc những người quan tâm đến sức khỏe và dinh dưỡng hằng ngày. Ngoài ra, đề tài còn hướng đến đối tượng quản trị viên tham gia quản lý dữ liệu món ăn, nguyên liệu, danh mục và vận hành toàn bộ hệ thống.

Phạm vi nghiên cứu của đề tài chủ yếu tập trung vào việc phát triển các chức năng liên quan đến quản lý tài khoản người dùng, thiết lập hồ sơ dinh dưỡng, gợi ý thực đơn cá nhân hóa, xây dựng lộ trình thực đơn tự động theo tuần và tích hợp chatbot

AI hỗ trợ tư vấn dinh dưỡng. Bên cạnh đó, hệ thống còn hỗ trợ lưu trữ lịch sử thực đơn, danh sách món ăn yêu thích và quản lý dữ liệu thông qua trang quản trị dành cho quản trị viên.

Về mặt công nghệ, hệ thống được xây dựng dựa trên nền tảng MERN Stack với các công nghệ như React cho phần giao diện người dùng, Node.js kết hợp Express để phát triển backend và xử lý nghiệp vụ hệ thống, cùng với MongoDB để lưu trữ dữ liệu. Ngoài ra, hệ thống còn tích hợp API với các mô hình ngôn ngữ lớn (LLMs) nhằm hỗ trợ chức năng chatbot AI và tạo thực đơn tự động.

Quá trình nghiên cứu và phát triển đề tài được thực hiện trong khoảng thời gian từ tháng 4 năm 2026 đến tháng 6 năm 2026, bao gồm các giai đoạn phân tích yêu cầu, thiết kế hệ thống, xây dựng chức năng, kiểm thử và hoàn thiện sản phẩm. Đề tài tập trung triển khai và thử nghiệm hệ thống trong môi trường phát triển cục bộ (local environment), phục vụ cho mục đích học tập, nghiên cứu và định hướng mở rộng trong tương lai.

6. Phương pháp nghiên cứu

5.1 Phương pháp nghiên cứu lý thuyết

Nghiên cứu tài liệu:

Tiến hành thu thập và nghiên cứu các tài liệu, sách, bài báo khoa học và nguồn thông tin liên quan đến lĩnh vực dinh dưỡng, hệ gợi ý thông minh, trí tuệ nhân tạo (AI) và phát triển ứng dụng web. Bên cạnh đó, đề tài cũng tìm hiểu các công nghệ thuộc nền tảng MERN Stack nhằm phục vụ cho quá trình thiết kế và xây dựng hệ thống.

Ngoài ra, còn khảo sát và phân tích một số website, ứng dụng hỗ trợ dinh dưỡng và gợi ý thực đơn hiện có trên thị trường nhằm đánh giá ưu điểm, hạn chế cũng như cách thức hoạt động của các hệ thống này. Qua đó, làm cơ sở để xây dựng một hệ thống phù hợp với nhu cầu thực tế của người dùng.

Phân tích các mô hình và công nghệ liên quan:

Nghiên cứu các phương pháp gợi ý thực đơn phổ biến như Content-Based Filtering, Collaborative Filtering và Hybrid Recommendation nhằm lựa chọn giải pháp phù hợp cho hệ thống. Đồng thời, nghiên cứu việc ứng dụng các mô hình ngôn ngữ lớn (LLMs) vào chatbot AI hỗ trợ tư vấn dinh dưỡng và xây dựng thực đơn tự động.

Thông qua việc phân tích các mô hình hiện có, đề tài xác định được nhu cầu thực tế của người dùng trong việc xây dựng chế độ ăn uống khoa học, từ đó đề xuất các chức năng phù hợp cho hệ thống.

6.2 Phương pháp thực nghiệm

Xây dựng hệ thống thử nghiệm:

Tiến hành xây dựng phiên bản thử nghiệm của website với các chức năng cơ bản như đăng ký, đăng nhập, quản lý hồ sơ cá nhân, gợi ý thực đơn, lưu món ăn yêu thích và chatbot AI hỗ trợ tư vấn dinh dưỡng.

Hệ thống được phát triển bằng các công nghệ hiện đại như React cho frontend, Node.js kết hợp Express cho backend và MongoDB cho cơ sở dữ liệu nhằm đảm bảo tính linh hoạt, khả năng mở rộng và dễ bảo trì.

Kiểm thử và đánh giá hệ thống:

Sau khi hoàn thiện phiên bản thử nghiệm, tiến hành kiểm thử các chức năng của hệ thống nhằm đảm bảo tính chính xác và ổn định trong quá trình hoạt động. Việc kiểm thử tập trung vào giao diện người dùng, khả năng xử lý dữ liệu, tốc độ phản hồi và tính chính xác của các gợi ý thực đơn.

Ngoài ra, còn thu thập phản hồi từ người dùng thử nghiệm để đánh giá trải nghiệm sử dụng, từ đó điều chỉnh và cải thiện hệ thống trước khi hoàn thiện sản phẩm.

Phương pháp phân tích và tổng hợp

Phân tích dữ liệu:

Tiến hành phân tích dữ liệu người dùng, dữ liệu món ăn và kết quả gợi ý thực đơn nhằm đánh giá hiệu quả hoạt động của hệ thống. Các tiêu chí được xem xét bao gồm độ phù hợp của thực đơn được đề xuất, mức độ cá nhân hóa, tốc độ xử lý và trải nghiệm người dùng.

Việc phân tích dữ liệu còn giúp xác định những hạn chế của hệ thống để đưa ra các giải pháp cải thiện trong tương lai.

Tổng hợp kết quả nghiên cứu:

Sau quá trình nghiên cứu, thiết kế và xây dựng hệ thống, tiến hành tổng hợp các kết quả đạt được để đánh giá mức độ hoàn thành mục tiêu đề ra. Từ đó, đưa ra các kết luận về hiệu quả của website gợi ý thực đơn dinh dưỡng thông minh trong việc hỗ trợ người dùng xây dựng chế độ ăn uống khoa học và hợp lý.

Bên cạnh đó, cũng đề xuất các hướng phát triển trong tương lai như mở rộng dữ liệu món ăn, nâng cao độ chính xác của hệ thống gợi ý, cải thiện chatbot AI và triển khai hệ thống trên nhiều nền tảng khác nhau nhằm nâng cao trải nghiệm người dùng.

7. Đóng góp của đề án

Đề tài hướng đến việc xây dựng một website gợi ý thực đơn dinh dưỡng thông minh nhằm hỗ trợ người dùng xây dựng chế độ ăn uống khoa học, phù hợp với nhu cầu và mục tiêu sức khỏe cá nhân. Hệ thống không chỉ giúp người dùng tiết kiệm thời gian trong việc lựa chọn món ăn và lập kế hoạch dinh dưỡng mà còn góp phần nâng cao nhận thức về tầm quan trọng của chế độ ăn uống hợp lý đối với sức khỏe. Bên cạnh đó, việc ứng dụng công nghệ Trí tuệ nhân tạo (AI) vào hệ thống sẽ giúp nâng cao khả năng cá nhân hóa thực đơn, hỗ trợ người dùng nhận được các gợi ý phù hợp với thể trạng, sở thích ăn uống và mức chi tiêu của bản thân. Đồng thời, chatbot AI được tích hợp trong hệ thống cũng giúp người dùng dễ dàng tiếp cận các kiến thức dinh dưỡng và nhận tư vấn một cách nhanh chóng, thuận tiện hơn. Ngoài ra, đề tài còn góp phần ứng dụng hiệu quả công nghệ thông tin trong lĩnh vực chăm sóc sức khỏe và dinh dưỡng, giúp tối ưu hóa quá trình quản lý dữ liệu, nâng cao trải nghiệm người dùng và tạo nền tảng cho việc phát triển các hệ thống hỗ trợ sức khỏe thông minh trong tương lai.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1 ReactJS

1.1.1 Khái niệm

React là một thư viện JavaScript mã nguồn mở được sử dụng rộng rãi để xây dựng giao diện người dùng (frontend) cho các ứng dụng web. React tập trung chủ yếu vào việc phát triển phần hiển thị (view), không can thiệp trực tiếp vào cấu trúc tổng thể hay logic nghiệp vụ của ứng dụng.

React có khả năng hoạt động linh hoạt trên cả phía client và server, đồng thời hỗ trợ cơ chế cập nhật giao diện một cách tối ưu. Thay vì cập nhật toàn bộ DOM khi có thay đổi, React sẽ so sánh trạng thái giữa các lần render và chỉ cập nhật những phần thực sự thay đổi, từ đó giúp nâng cao hiệu năng ứng dụng.

Một số khái niệm:

Virtual DOM (DOM ảo) là một trong những cơ chế cốt lõi giúp React đạt hiệu suất cao. Đây là một bản sao nhẹ của DOM thật (Document Object Model), nơi chứa toàn bộ cấu trúc HTML của trang web.

Khi có sự thay đổi từ phía người dùng hoặc dữ liệu, React sẽ cập nhật trên Virtual DOM trước, sau đó so sánh (diff) với phiên bản trước đó để xác định những phần cần thay đổi. Cuối cùng, chỉ các phần tử thay đổi mới được cập nhật lên DOM thật. Cách tiếp cận này giúp giảm thiểu thao tác trực tiếp lên DOM, từ đó cải thiện tốc độ xử lý và tiết kiệm tài nguyên hệ thống, đặc biệt trong các ứng dụng có giao diện phức tạp.

JSX (JavaScript XML) là một cú pháp mở rộng của JavaScript, cho phép viết mã giao diện với cú pháp tương tự HTML ngay trong JavaScript. JSX đóng vai trò là cầu nối giúp việc xây dựng giao diện trở nên trực quan và dễ hiểu hơn.

Tuy nhiên, JSX không phải là HTML thuần. Khi chương trình được biên dịch, JSX sẽ được chuyển đổi thành các lệnh JavaScript thông qua các công cụ như Babel, sau đó tạo ra các phần tử DOM tương ứng.

Việc sử dụng JSX giúp kết hợp logic và giao diện trong cùng một cấu trúc, từ đó nâng cao hiệu quả lập trình và khả năng kiểm soát code.

Component là đơn vị cơ bản trong React, đóng vai trò như các “khối xây dựng” của giao diện người dùng. Mỗi component đại diện cho một phần nhỏ của giao diện và có thể được tái sử dụng nhiều lần trong ứng dụng.

Các component thường được thiết kế độc lập, có thể quản lý riêng biệt và dễ dàng bảo trì. Việc chia nhỏ giao diện thành nhiều component giúp code trở nên rõ ràng, dễ đọc và dễ mở rộng. Ngoài ra, các component còn có thể tái sử dụng ở nhiều vị trí khác nhau, góp phần tăng tính linh hoạt và giảm thời gian phát triển.

1.1.2 Ưu điểm của ReactJs

React được đánh giá là một trong những thư viện phổ biến nhất hiện nay trong phát triển giao diện người dùng nhờ khả năng tối ưu hiệu suất và tính linh hoạt cao. Một trong những ưu điểm nổi bật của React là cơ chế Virtual DOM, cho phép hệ thống tạo ra một bản sao của DOM thật để theo dõi sự thay đổi dữ liệu. Khi giao diện thay đổi, React chỉ cập nhật những thành phần cần thiết thay vì tải lại toàn bộ trang, giúp ứng dụng hoạt động nhanh hơn và cải thiện trải nghiệm người dùng.

ReactJS sử dụng kiến trúc Component-Based, cho phép chia giao diện thành nhiều thành phần nhỏ độc lập. Mỗi component đảm nhiệm một chức năng riêng và có thể tái sử dụng ở nhiều vị trí khác nhau trong ứng dụng. Điều này giúp mã nguồn trở nên rõ ràng, dễ quản lý và thuận tiện hơn trong quá trình bảo trì hoặc mở rộng hệ thống.

Ngoài khả năng tối ưu hiệu suất, ReactJS còn là thư viện mã nguồn mở với cộng đồng phát triển lớn trên toàn thế giới. Người học có thể dễ dàng tìm kiếm tài liệu, ví dụ minh họa và các thư viện hỗ trợ phục vụ cho quá trình phát triển ứng dụng. Đồng thời, React được phát triển và duy trì bởi Meta nên luôn được cập nhật thường xuyên, đảm bảo tính ổn định và độ tin cậy cao trong quá trình sử dụng.

Một ưu điểm khác của ReactJS là khả năng phát triển ứng dụng trên nhiều nền tảng. Ngoài xây dựng website, React còn có thể kết hợp với React Native để phát triển ứng dụng di động trên Android và iOS. Điều này giúp lập trình viên tận dụng lại kiến thức và giảm thời gian phát triển sản phẩm.

Ngoài ra, ReactJS còn hỗ trợ tốt trong quá trình kiểm tra và sửa lỗi thông qua các công cụ như React Developer Tools. Các công cụ này giúp lập trình viên theo dõi trạng thái dữ liệu, cấu trúc component và luồng hoạt động của ứng dụng một cách trực quan, từ đó nâng cao hiệu quả phát triển và tối ưu chất lượng sản phẩm.

1.1.3 Nhược điểm của ReactJS

Mặc dù React mang lại nhiều lợi ích trong phát triển giao diện người dùng, nhưng thư viện này vẫn tồn tại một số hạn chế nhất định. Đối với người mới bắt đầu,

React có thể gây khó khăn trong quá trình học tập do yêu cầu nắm vững JavaScript hiện đại và cú pháp JSX. Việc kết hợp giữa giao diện và logic xử lý trong cùng một file đôi khi khiến người học dễ nhầm lẫn, đặc biệt với những người đã quen với cách xây dựng web truyền thống sử dụng HTML, CSS và JavaScript tách biệt.

Bên cạnh đó, React chỉ tập trung vào phần giao diện nên trong quá trình phát triển ứng dụng, lập trình viên thường phải sử dụng thêm nhiều thư viện hỗ trợ khác. Điều này làm cho cấu trúc dự án trở nên phức tạp hơn và có thể làm tăng dung lượng ứng dụng trong giai đoạn phát triển. Nếu không được tối ưu hợp lý, tốc độ tải trang và hiệu suất hệ thống có thể bị ảnh hưởng, đặc biệt đối với các dự án có quy mô lớn. Tuy nhiên, phần lớn các hạn chế này có thể được cải thiện thông qua các công cụ tối ưu hóa và kinh nghiệm triển khai thực tế.[9]

1.2 NodeJS

1.2.1 Khái niệm

Node.js là môi trường chạy JavaScript phía máy chủ được xây dựng dựa trên V8 Engine của Google. Công nghệ này cho phép lập trình viên sử dụng JavaScript để phát triển backend thay vì chỉ hoạt động trên trình duyệt như trước đây. Nhờ đó, quá trình phát triển ứng dụng trở nên linh hoạt hơn khi có thể sử dụng cùng một ngôn ngữ cho cả frontend và backend.

Một trong những ưu điểm nổi bật của Node.js là cơ chế xử lý bất đồng bộ (Asynchronous) và mô hình Event-Driven. Cơ chế này giúp hệ thống xử lý nhiều yêu cầu cùng lúc mà không làm gián đoạn hoạt động của ứng dụng, từ đó cải thiện hiệu suất và tối ưu tài nguyên hệ thống. Vì vậy, Node.js thường được sử dụng trong các ứng dụng yêu cầu tốc độ xử lý cao và khả năng phản hồi theo thời gian thực như chat trực tuyến, livestream hoặc hệ thống thông báo realtime.

Node.js còn sở hữu hệ sinh thái thư viện phong phú thông qua npm (Node Package Manager), hỗ trợ lập trình viên dễ dàng cài đặt và quản lý các package cần thiết cho dự án. Kết hợp với các framework như Express hay công nghệ như Socket.IO, Node.js giúp việc xây dựng các ứng dụng web hiện đại trở nên nhanh chóng, thuận tiện và dễ mở rộng hơn.

1.2.2 Ứng dụng

Node.js được ứng dụng rộng rãi trong nhiều lĩnh vực phát triển phần mềm nhờ khả năng xử lý nhanh, hoạt động bất đồng bộ và hỗ trợ nhiều kết nối cùng lúc. Một

trong những ứng dụng phổ biến nhất của Node.js là xây dựng các hệ thống yêu cầu phản hồi theo thời gian thực như ứng dụng nhắn tin, họp trực tuyến, livestream hoặc game online nhiều người chơi. Nhờ cơ chế Event-Driven và khả năng xử lý dữ liệu liên tục, Node.js giúp các ứng dụng hoạt động ổn định và giảm độ trễ khi trao đổi dữ liệu giữa máy chủ và người dùng.

Ngoài ra, Node.js còn được sử dụng để phát triển các hệ thống API phục vụ website và ứng dụng di động. Với khả năng xử lý lượng lớn request đồng thời, Node.js phù hợp cho việc xây dựng RESTful API hoặc GraphQL API nhằm cung cấp dữ liệu nhanh chóng và linh hoạt. Trong các ứng dụng Single Page Application (SPA), Node.js đóng vai trò xử lý dữ liệu phía server, giúp giao diện hoạt động mượt mà mà không cần tải lại toàn bộ trang web sau mỗi thao tác của người dùng.

Bên cạnh lĩnh vực web, Node.js còn được áp dụng trong các hệ thống IoT và xử lý dữ liệu theo luồng (streaming). Công nghệ này có khả năng tiếp nhận và xử lý dữ liệu từ nhiều thiết bị thông minh trong thời gian ngắn, phù hợp với các hệ thống giám sát, cảm biến hoặc nhà thông minh. Đồng thời, Node.js cũng hỗ trợ tốt cho các ứng dụng phát video, âm thanh hoặc xử lý tệp dung lượng lớn nhờ cơ chế streaming giúp truyền tải dữ liệu liên tục và tối ưu hiệu suất hệ thống.

Không chỉ dừng lại ở ứng dụng web, Node.js còn hỗ trợ phát triển phần mềm đa nền tảng khi kết hợp với các công nghệ như Electron. Điều này cho phép lập trình viên xây dựng ứng dụng desktop hoạt động trên nhiều hệ điều hành như Windows, macOS và Linux bằng JavaScript. Nhờ tính linh hoạt và hiệu năng cao, Node.js ngày càng trở thành lựa chọn phổ biến trong quá trình phát triển các hệ thống hiện đại.

1.2.3 Ưu điểm của NodeJS

Node.js được đánh giá cao nhờ khả năng xử lý nhiều yêu cầu cùng lúc mà vẫn đảm bảo hiệu suất ổn định. Với cơ chế hoạt động bất đồng bộ và mô hình Event-Driven, Node.js có thể quản lý số lượng lớn kết nối đồng thời mà không tiêu tốn quá nhiều tài nguyên hệ thống. Điều này giúp Node.js trở thành lựa chọn phù hợp cho các ứng dụng cần trao đổi dữ liệu liên tục như chat trực tuyến, livestream, game online hoặc các hệ thống cập nhật dữ liệu theo thời gian thực.

Một ưu điểm nổi bật khác của Node.js là tốc độ xử lý nhanh nhờ sử dụng V8 Engine của Google. Công nghệ này cho phép JavaScript được biên dịch trực tiếp thành mã máy trong quá trình thực thi, từ đó giảm thời gian xử lý và tăng hiệu năng ứng

dụng. Ngoài ra, Node.js còn cho phép sử dụng JavaScript ở cả frontend và backend, giúp lập trình viên dễ dàng tái sử dụng mã nguồn và giảm sự phức tạp trong quá trình phát triển hệ thống.

Bên cạnh hiệu năng cao, Node.js còn sở hữu hệ sinh thái thư viện phong phú thông qua npm (Node Package Manager). Kho thư viện này cung cấp hàng triệu package hỗ trợ nhiều chức năng khác nhau, giúp việc phát triển ứng dụng trở nên nhanh chóng và thuận tiện hơn. Đồng thời, Node.js có tính linh hoạt cao, có thể được sử dụng để xây dựng nhiều loại hệ thống khác nhau như API, ứng dụng web, IoT hoặc phần mềm đa nền tảng.

Cộng đồng phát triển Node.js rất lớn và hoạt động mạnh mẽ, giúp người dùng dễ dàng tìm kiếm tài liệu, thư viện và giải pháp hỗ trợ trong quá trình học tập cũng như phát triển dự án. Các công nghệ liên quan như Express hay Socket.IO cũng góp phần mở rộng khả năng của Node.js, giúp việc xây dựng các ứng dụng hiện đại trở nên hiệu quả và dễ triển khai hơn.

1.2.4 Nhược điểm của NodeJS

Mặc dù Node.js sở hữu nhiều ưu điểm trong phát triển ứng dụng web hiện đại, nhưng nền tảng này vẫn tồn tại một số hạn chế nhất định. Do sử dụng cơ chế single-thread kết hợp với event loop, Node.js không thật sự phù hợp với các tác vụ yêu cầu xử lý tính toán phức tạp hoặc tiêu tốn nhiều tài nguyên CPU. Khi hệ thống phải thực hiện các phép tính nặng hoặc xử lý lượng dữ liệu lớn trong thời gian dài, hiệu suất tổng thể của ứng dụng có thể bị ảnh hưởng do luồng xử lý chính bị chặn.

Node.js thường không phải lựa chọn tối ưu đối với những hệ thống cần xử lý đa luồng hoặc chạy song song nhiều tiến trình phức tạp. Trong các trường hợp này, những nền tảng hỗ trợ multi-threading mạnh như Java hoặc .NET thường mang lại hiệu quả tốt hơn. Bên cạnh đó, việc xử lý bất đồng bộ trong Node.js đôi khi cũng làm mã nguồn trở nên khó quản lý nếu dự án có quy mô lớn và chứa nhiều luồng xử lý lồng nhau. Mặc dù async/await đã giúp cải thiện vấn đề này, nhưng việc tổ chức và bảo trì mã nguồn vẫn đòi hỏi lập trình viên có kinh nghiệm.

Một hạn chế khác của Node.js đến từ hệ sinh thái npm với số lượng thư viện rất lớn và đa dạng. Điều này giúp tăng tốc quá trình phát triển nhưng cũng tiềm ẩn nhiều rủi ro nếu lựa chọn các package kém chất lượng hoặc không còn được duy trì. Ngoài ra, một số thư viện bên thứ ba có thể chứa lỗ hổng bảo mật hoặc mã độc, gây ảnh

hưởng đến độ an toàn của hệ thống. Vì vậy, trong quá trình phát triển ứng dụng với Node.js, việc kiểm tra, đánh giá và quản lý các thư viện sử dụng là yếu tố rất quan trọng để đảm bảo tính ổn định và bảo mật cho hệ thống. [10]

1.3 ExpressJS

1.3.1 Khái niệm

Express là một framework được phát triển dựa trên Node.js, chuyên dùng để xây dựng ứng dụng web và hệ thống API phía máy chủ. Express.js cung cấp nhiều công cụ hỗ trợ giúp quá trình phát triển backend trở nên đơn giản, nhanh chóng và dễ quản lý hơn. Nhờ cấu trúc gọn nhẹ và khả năng tùy biến cao, framework này được sử dụng rộng rãi trong các ứng dụng web hiện đại.

Một trong những điểm nổi bật của Express.js là khả năng xử lý routing hiệu quả, cho phép lập trình viên dễ dàng quản lý các đường dẫn và yêu cầu từ phía người dùng. Ngoài ra, Express còn hỗ trợ middleware giúp mở rộng chức năng của hệ thống như xác thực người dùng, xử lý dữ liệu, quản lý lỗi hoặc bảo mật ứng dụng. Điều này giúp việc xây dựng và tổ chức mã nguồn trở nên rõ ràng, thuận tiện hơn trong quá trình phát triển và bảo trì.

Express.js sở hữu cộng đồng phát triển lớn cùng hệ sinh thái thư viện phong phú, hỗ trợ lập trình viên dễ dàng tích hợp các tính năng cần thiết vào ứng dụng. Với khả năng hoạt động linh hoạt và hiệu suất ổn định, Express.js hiện đang là một trong những framework phổ biến nhất trong việc phát triển RESTful API và các hệ thống backend sử dụng JavaScript.

1.3.2 Tính năng của ExpressJS

Express cung cấp cơ chế middleware linh hoạt, cho phép hệ thống xử lý dữ liệu theo từng bước trước khi trả kết quả về cho người dùng. Thông qua middleware, lập trình viên có thể dễ dàng thực hiện các chức năng như kiểm tra đăng nhập, xử lý lỗi, xác thực dữ liệu hoặc quản lý bảo mật cho ứng dụng. Cơ chế này giúp mã nguồn được tổ chức rõ ràng hơn và thuận tiện trong việc mở rộng chức năng khi cần thiết.

Express.js còn hỗ trợ hệ thống routing mạnh mẽ giúp quản lý các URL và phương thức HTTP như GET, POST, PUT hoặc DELETE một cách dễ dàng. Nhờ đó, việc xây dựng RESTful API trở nên nhanh chóng và có cấu trúc rõ ràng hơn. Express cũng hỗ trợ kết nối với nhiều loại cơ sở dữ liệu khác nhau như MongoDB, PostgreSQL

hoặc Redis, giúp việc xử lý và lưu trữ dữ liệu hiệu quả hơn trong các ứng dụng web hiện đại.

Bên cạnh việc phát triển API, Express.js còn hỗ trợ nhiều template engine như EJS, Pug hay Handlebars để xây dựng giao diện động phía server. Framework này hoạt động ổn định cùng với hệ sinh thái npm của Node.js, cho phép tích hợp thêm nhiều thư viện và công nghệ khác nhau. Nhờ tính linh hoạt, dễ sử dụng và khả năng mở rộng cao, Express.js hiện là một trong những framework phổ biến nhất trong phát triển backend bằng JavaScript.

1.3.3 Ưu điểm của ExpressJS

Express là một framework có cấu trúc tối giản, cho phép lập trình viên chủ động xây dựng và tổ chức hệ thống theo nhiều cách khác nhau thay vì bị ràng buộc bởi khuôn mẫu cố định. Nhờ tính linh hoạt này, Express.js có thể được áp dụng cho nhiều loại dự án từ ứng dụng nhỏ đến các hệ thống web có quy mô lớn và phức tạp.

Một ưu điểm khác của Express.js là khả năng tiếp cận dễ dàng đối với những người đã có kiến thức về JavaScript hoặc Node.js. Framework này sử dụng cú pháp đơn giản, dễ đọc và dễ triển khai, giúp rút ngắn thời gian học tập cũng như phát triển ứng dụng.

Tài liệu hướng dẫn phong phú cùng cộng đồng hỗ trợ lớn cũng giúp lập trình viên dễ dàng tìm kiếm giải pháp trong quá trình làm việc.

Bên cạnh đó, Express.js giúp tăng tốc độ phát triển nhờ hỗ trợ sẵn nhiều chức năng cần thiết cho backend như routing, middleware và xử lý request/response. Kết hợp với hệ sinh thái npm, lập trình viên có thể nhanh chóng tích hợp thêm các package hỗ trợ mà không cần xây dựng lại từ đầu. Đồng thời, Express.js kế thừa hiệu suất cao từ Node.js, cho phép xử lý nhiều yêu cầu đồng thời và duy trì tốc độ phản hồi ổn định cho ứng dụng.

Ngoài việc xây dựng website động, Express.js còn được sử dụng rộng rãi trong phát triển RESTful API, hệ thống quản lý dữ liệu, ứng dụng thương mại điện tử và nhiều nền tảng web hiện đại khác. Nhờ khả năng mở rộng tốt và hoạt động ổn định, Express.js hiện là một trong những framework backend phổ biến nhất trong hệ sinh thái JavaScript.

1.3.4 Nhược điểm của ExpressJS

Mặc dù Express mang lại nhiều lợi ích trong phát triển backend, nhưng framework này vẫn tồn tại một số hạn chế nhất định. Do Express.js có cấu trúc khá tự

do và không áp đặt mô hình phát triển cụ thể nên trong các dự án lớn, việc tổ chức mã nguồn có thể trở nên khó kiểm soát nếu không được xây dựng hợp lý ngay từ đầu. Điều này đặc biệt ảnh hưởng khi làm việc theo nhóm hoặc khi hệ thống cần mở rộng trong thời gian dài.

Express.js phụ thuộc khá nhiều vào middleware để xử lý các chức năng như xác thực, bảo mật hoặc quản lý request/response. Nếu sử dụng quá nhiều middleware trong cùng một ứng dụng, mã nguồn dễ trở nên phức tạp và khó theo dõi. Đồng thời, việc xử lý qua nhiều tầng middleware cũng có thể ảnh hưởng đến hiệu suất của hệ thống nếu không được tối ưu phù hợp.

Express.js chỉ cung cấp những chức năng cơ bản nên lập trình viên thường phải tự xây dựng thêm nhiều thành phần cho ứng dụng hoặc tích hợp các thư viện bên ngoài. So với một số framework khác có cấu trúc đầy đủ hơn, Express.js đòi hỏi người phát triển cần có kiến thức và kinh nghiệm tốt để triển khai hệ thống hoàn chỉnh.

Ngoài vấn đề cấu trúc, Express.js cũng chưa tích hợp sẵn nhiều cơ chế bảo mật và quản lý lỗi nâng cao. Các chức năng như chống XSS, CSRF hoặc xử lý lỗi tập trung thường phải được cài đặt thông qua middleware hoặc package bổ sung. Vì vậy, trong quá trình phát triển ứng dụng bằng Express.js, lập trình viên cần chú trọng đến việc xây dựng cơ chế bảo mật và kiểm soát lỗi nhằm đảm bảo tính ổn định và an toàn cho hệ thống. [11]

1.4 RESTful API

1.4.1 Tổng quan về RESTful API

RESTful API là một chuẩn kiến trúc được sử dụng phổ biến trong quá trình xây dựng hệ thống web và các ứng dụng hiện đại. Kiến trúc này cho phép các thành phần trong hệ thống giao tiếp với nhau thông qua giao thức HTTP, giúp việc truyền tải và xử lý dữ liệu giữa máy khách (client) và máy chủ (server) diễn ra hiệu quả hơn. Nhờ tính đơn giản và linh hoạt, RESTful API hiện được áp dụng rộng rãi trong phát triển website, ứng dụng di động và các nền tảng trực tuyến.

RESTful API tổ chức dữ liệu theo dạng tài nguyên (resource), trong đó mỗi tài nguyên sẽ được định danh bằng một đường dẫn URL cụ thể. Các thao tác với dữ liệu như lấy thông tin, thêm mới, cập nhật hoặc xóa dữ liệu sẽ được thực hiện thông qua các phương thức HTTP như GET, POST, PUT và DELETE. Điều này giúp hệ thống có cấu trúc rõ ràng, dễ quản lý và thuận tiện trong quá trình phát triển cũng như bảo trì.

Để trao đổi dữ liệu, RESTful API thường sử dụng các định dạng phổ biến như JSON hoặc XML. Trong đó, JSON được sử dụng nhiều hơn nhờ cấu trúc gọn nhẹ, dễ đọc và dễ xử lý trên nhiều ngôn ngữ lập trình khác nhau. Ngoài ra, RESTful API còn hoạt động theo cơ chế stateless, nghĩa là mỗi request từ client gửi lên server đều độc lập và chứa đầy đủ thông tin cần thiết để xử lý. Nhờ đặc điểm này, hệ thống có khả năng mở rộng tốt, hoạt động ổn định và dễ dàng tích hợp với nhiều nền tảng khác nhau.

1.4.2 Các thành phần chính của RESTful API

API (Application Programming Interface) là cơ chế cho phép các phần mềm hoặc hệ thống khác nhau có thể kết nối và trao đổi dữ liệu với nhau. Thông qua API, các ứng dụng có thể gửi yêu cầu và nhận dữ liệu từ hệ thống khác mà không cần biết chi tiết cách hệ thống đó được xây dựng hay xử lý bên trong. Trong quá trình phát triển ứng dụng web hiện đại, API đóng vai trò trung gian giúp frontend và backend giao tiếp hiệu quả. Dữ liệu được truyền tải thường ở dạng JSON hoặc XML để dễ dàng xử lý và hiển thị trên nhiều nền tảng khác nhau.

REST là một kiểu kiến trúc được xây dựng dựa trên giao thức HTTP nhằm hỗ trợ việc thiết kế API theo hướng đơn giản và dễ mở rộng. Kiến trúc này sử dụng các phương thức HTTP phổ biến như GET để lấy dữ liệu, POST để thêm dữ liệu mới, PUT hoặc PATCH để cập nhật dữ liệu và DELETE để xóa dữ liệu khỏi hệ thống. Với REST, mỗi tài nguyên sẽ được đại diện thông qua một URL cụ thể, giúp việc quản lý và truy cập dữ liệu trở nên trực quan, rõ ràng và thuận tiện hơn trong quá trình phát triển ứng dụng.

1.4.3 Các nguyên tắc của RESTful API

RESTful API được xây dựng dựa trên nhiều nguyên tắc nhằm đảm bảo hệ thống hoạt động ổn định, dễ mở rộng và thuận tiện trong quá trình phát triển. Một trong những nguyên tắc quan trọng là mô hình Client – Server (Client-Server Architecture), trong đó phần giao diện người dùng và phần xử lý dữ liệu được tách biệt với nhau. Cách tổ chức này giúp frontend và backend có thể phát triển độc lập, dễ nâng cấp và thuận tiện trong việc bảo trì hệ thống. Ví dụ, ứng dụng web hoặc mobile có thể thay đổi giao diện mà không ảnh hưởng đến phần xử lý dữ liệu trên server.

RESTful API còn hoạt động theo cơ chế Không lưu trạng thái (Statelessness), nghĩa là mỗi request gửi từ client lên server đều phải chứa đầy đủ thông tin cần thiết để xử lý. Server sẽ không lưu trạng thái của người dùng sau mỗi lần yêu cầu, giúp giảm

tải bộ nhớ và nâng cao khả năng mở rộng hệ thống. Nhờ đó, hệ thống có thể xử lý nhiều yêu cầu cùng lúc và hỗ trợ tốt cho các mô hình phân tán hoặc cân bằng tải.

Ngoài ra, RESTful API hỗ trợ cơ chế Cache (Cacheable) nhằm tối ưu hiệu suất xử lý dữ liệu. Những dữ liệu ít thay đổi có thể được lưu tạm trên phía client để tái sử dụng trong các lần truy cập tiếp theo mà không cần gửi lại yêu cầu đến server. Điều này giúp giảm số lượng request, tiết kiệm tài nguyên và cải thiện tốc độ phản hồi của ứng dụng.

Một đặc điểm khác của RESTful API là khả năng xây dựng hệ thống phân lớp (Layered System). Trong mô hình này, client không cần biết dữ liệu được xử lý qua bao nhiêu lớp trung gian trước khi đến server chính. Các lớp như proxy, gateway hoặc load balancer có thể được thêm vào để hỗ trợ bảo mật, kiểm tra dữ liệu hoặc cân bằng tải mà không làm ảnh hưởng đến cách client hoạt động.

RESTful API cũng yêu cầu sử dụng giao diện thống nhất (Uniform Interface) để việc giao tiếp giữa các hệ thống trở nên rõ ràng và dễ hiểu hơn. Mỗi tài nguyên trong hệ thống sẽ được định danh thông qua một URL riêng biệt, đồng thời dữ liệu trao đổi thường ở dạng JSON hoặc XML. Các request và response phải chứa đầy đủ thông tin cần thiết như định dạng dữ liệu hoặc quyền truy cập để đảm bảo quá trình giao tiếp diễn ra chính xác và hiệu quả.

Trong một số trường hợp, RESTful API còn hỗ trợ cơ chế Code on Demand, cho phép server gửi thêm đoạn mã xử lý về phía client để mở rộng chức năng của ứng dụng. Tuy nhiên, đây không phải là thành phần bắt buộc và chỉ được sử dụng khi cần tăng tính linh hoạt cho hệ thống.

1.4.4 Ưu điểm của RESTful API

RESTful API giúp quá trình xây dựng ứng dụng trở nên thuận tiện hơn nhờ sử dụng các chuẩn giao tiếp phổ biến của giao thức HTTP. Các thao tác như lấy dữ liệu, thêm mới, cập nhật hay xóa dữ liệu đều được thực hiện thông qua những phương thức quen thuộc, giúp lập trình viên dễ dàng triển khai và quản lý hệ thống hiệu quả hơn.

Một ưu điểm nổi bật khác của RESTful API là khả năng mở rộng linh hoạt. Khi hệ thống cần bổ sung chức năng mới hoặc tích hợp thêm dịch vụ khác, RESTful API có thể đáp ứng dễ dàng mà không ảnh hưởng nhiều đến cấu trúc hiện có. Điều này giúp các ứng dụng hoạt động ổn định và thích nghi tốt với nhu cầu phát triển trong tương lai.

RESTful API còn tạo sự tách biệt rõ ràng giữa các thành phần trong hệ thống. Frontend và backend có thể hoạt động độc lập, giúp việc chỉnh sửa hoặc nâng cấp một phần của ứng dụng không làm gián đoạn toàn bộ hệ thống. Nhờ đó, quá trình bảo trì và phát triển phần mềm trở nên thuận tiện và tiết kiệm thời gian hơn.

RESTful API có khả năng tương thích với nhiều nền tảng và ngôn ngữ lập trình khác nhau. Các hệ thống web, ứng dụng di động hoặc phần mềm bên thứ ba đều có thể dễ dàng kết nối và trao đổi dữ liệu thông qua cùng một API. Điều này giúp tăng tính linh hoạt và khả năng tích hợp trong quá trình phát triển ứng dụng hiện đại.

1.4.5 Nhược điểm của RESTful API

Mặc dù RESTful API mang lại nhiều lợi ích trong phát triển ứng dụng, nhưng việc xây dựng và quản lý hệ thống API cũng đòi hỏi khá nhiều thời gian và nguồn lực. Lập trình viên cần thiết kế cấu trúc API hợp lý, xử lý dữ liệu hiệu quả và đảm bảo hệ thống có thể hoạt động ổn định khi số lượng người dùng tăng lên. Đối với các dự án lớn, quá trình bảo trì và nâng cấp API cũng có thể trở nên phức tạp hơn nếu không được tổ chức tốt ngay từ đầu.

Một hạn chế khác của RESTful API là các vấn đề liên quan đến an toàn bảo mật. Vì API đóng vai trò trung gian trao đổi dữ liệu giữa client và server nên rất dễ trở thành mục tiêu của các cuộc tấn công mạng. Nếu hệ thống không được bảo vệ cẩn thận, dữ liệu có thể bị đánh cắp hoặc truy cập trái phép thông qua các hình thức tấn công phổ biến. Do đó, trong quá trình phát triển, lập trình viên thường phải áp dụng thêm các cơ chế như xác thực người dùng, mã hóa dữ liệu và kiểm tra dữ liệu đầu vào để tăng cường độ an toàn cho hệ thống.

RESTful API hoạt động chủ yếu dựa trên giao thức HTTP nên trong một số trường hợp đặc biệt, hiệu suất có thể chưa thật sự tối ưu. Những ứng dụng yêu cầu truyền dữ liệu liên tục theo thời gian thực hoặc xử lý giao tiếp phức tạp đôi khi sẽ phù hợp hơn với các công nghệ khác như WebSocket hoặc gRPC. Vì vậy, việc lựa chọn RESTful API cần phụ thuộc vào đặc điểm và yêu cầu cụ thể của từng hệ thống. [12]

1.5 JavaScript

1.5.1 Khái niệm

JavaScript là một ngôn ngữ lập trình phổ biến được sử dụng chủ yếu trong quá trình phát triển website và ứng dụng web. Ngôn ngữ này cho phép tạo ra các chức năng tương tác trực tiếp trên trình duyệt như xử lý sự kiện, thay đổi nội dung trang web hoặc

cập nhật dữ liệu mà không cần tải lại toàn bộ trang. Nhờ khả năng hoạt động linh hoạt, JavaScript đã trở thành một trong những công nghệ cốt lõi của lập trình web hiện đại.

Ban đầu, JavaScript chỉ được sử dụng để xử lý các tác vụ đơn giản trên phía trình duyệt. Tuy nhiên, cùng với sự phát triển của công nghệ, JavaScript hiện nay có thể được sử dụng trong nhiều lĩnh vực khác nhau như xây dựng giao diện người dùng, phát triển backend, ứng dụng di động và cả ứng dụng desktop. Việc kết hợp với các công nghệ như AJAX cũng giúp JavaScript hỗ trợ giao tiếp dữ liệu với máy chủ theo thời gian thực, nâng cao trải nghiệm người dùng và tối ưu hiệu suất hệ thống.

1.5.2 Ứng dụng

JavaScript được ứng dụng rộng rãi trong quá trình phát triển giao diện web hiện đại. Ngôn ngữ này cho phép tạo ra các hiệu ứng động, xử lý thao tác người dùng và cập nhật nội dung trực tiếp trên trang web mà không cần tải lại toàn bộ dữ liệu. Các chức năng như menu động, slideshow, popup, animation hoặc kiểm tra dữ liệu biểu mẫu đều có thể được xây dựng bằng JavaScript nhằm tăng tính tương tác và cải thiện trải nghiệm người dùng.

JavaScript cũng đóng vai trò quan trọng trong việc giao tiếp dữ liệu giữa client và server. Thông qua AJAX hoặc Fetch API, ứng dụng có thể gửi và nhận dữ liệu từ máy chủ một cách bất đồng bộ. Điều này giúp các website hoạt động mượt mà hơn, hỗ trợ cập nhật dữ liệu theo thời gian thực như hệ thống chat, thông báo trực tuyến hoặc hiển thị dữ liệu từ API mà không cần tải lại trang.

Trong lĩnh vực phát triển ứng dụng web, JavaScript thường được kết hợp với các framework và thư viện như React, Angular hoặc Vue để xây dựng các hệ thống SPA (Single Page Application). Những công nghệ này hỗ trợ tạo ra giao diện hiện đại, tốc độ xử lý nhanh và dễ dàng mở rộng chức năng cho ứng dụng.

Không chỉ giới hạn ở frontend, JavaScript còn được sử dụng để phát triển backend thông qua môi trường Node.js. Nhờ đó, lập trình viên có thể sử dụng cùng một ngôn ngữ cho cả phía client và server, giúp đồng bộ mã nguồn và tối ưu thời gian phát triển hệ thống.

JavaScript cũng được ứng dụng trong phát triển phần mềm đa nền tảng. Với các công nghệ như React Native hoặc Electron, lập trình viên có thể xây dựng ứng dụng di động và desktop hoạt động trên nhiều hệ điều hành khác nhau. Đồng thời, JavaScript

còn hỗ trợ phát triển game trên trình duyệt bằng HTML5 Canvas hoặc WebGL, giúp tạo ra các trò chơi tương tác trực tiếp trên web.

1.5.3 Ưu điểm của JavaScript

JavaScript có cú pháp tương đối đơn giản và dễ tiếp cận đối với người mới học lập trình. Hầu hết các trình duyệt hiện đại đều hỗ trợ sẵn JavaScript nên người dùng không cần cài đặt thêm môi trường phức tạp để bắt đầu phát triển ứng dụng web. Điều này giúp JavaScript trở thành một trong những ngôn ngữ phổ biến nhất hiện nay.

Một ưu điểm nổi bật của JavaScript là tính linh hoạt cao. Ngôn ngữ này có thể hoạt động trên nhiều nền tảng khác nhau như trình duyệt, máy chủ hoặc ứng dụng di động. JavaScript không chỉ được sử dụng để xây dựng giao diện frontend mà còn hỗ trợ phát triển backend, desktop app và nhiều hệ thống khác nhau thông qua các framework và công nghệ hiện đại.

JavaScript cũng mang lại hiệu suất tốt trong quá trình xử lý giao diện người dùng. Việc thực thi trực tiếp trên trình duyệt giúp giảm tải cho máy chủ và tăng tốc độ phản hồi của ứng dụng. Đồng thời, khả năng xử lý bất đồng bộ giúp hệ thống trao đổi dữ liệu nhanh chóng mà không làm gián đoạn trải nghiệm người dùng.

Hệ sinh thái của JavaScript rất phong phú với nhiều thư viện và framework phổ biến như React, Vue, Angular hay Node.js. Cộng đồng lập trình viên lớn giúp người học dễ dàng tìm kiếm tài liệu, ví dụ thực tế và giải pháp hỗ trợ trong quá trình phát triển dự án. JavaScript cũng dễ dàng tích hợp với nhiều công nghệ và dịch vụ khác nhau, giúp việc xây dựng ứng dụng trở nên thuận tiện và linh hoạt hơn.

1.5.4 Nhược điểm của JavaScript

Mặc dù có nhiều ưu điểm, JavaScript vẫn tồn tại một số hạn chế trong quá trình sử dụng. Vì mã JavaScript được thực thi trực tiếp trên trình duyệt nên các vấn đề liên quan đến bảo mật luôn cần được quan tâm. Nếu không xử lý dữ liệu đúng cách, ứng dụng có thể gặp các lỗ hổng như XSS hoặc bị khai thác bởi các hình thức tấn công khác.

Hiệu suất của JavaScript cũng phụ thuộc nhiều vào trình duyệt và cấu hình thiết bị của người dùng. Trên các thiết bị có cấu hình thấp hoặc trình duyệt cũ, tốc độ xử lý có thể bị giảm đáng kể. Trong một số trường hợp, người dùng tắt JavaScript trên trình duyệt sẽ khiến nhiều chức năng động của website không thể hoạt động bình thường.

JavaScript là ngôn ngữ thông dịch nên nhiều lỗi chỉ được phát hiện khi chương trình đang chạy. Điều này khiến quá trình kiểm tra và xử lý lỗi đôi khi trở nên khó khăn, đặc biệt với các dự án có quy mô lớn và nhiều chức năng phức tạp.

Một hạn chế khác là khả năng hỗ trợ JavaScript giữa các trình duyệt đôi khi không hoàn toàn giống nhau. Một số tính năng mới có thể chưa được hỗ trợ trên các trình duyệt cũ, buộc lập trình viên phải tối ưu hoặc bổ sung giải pháp thay thế để đảm bảo ứng dụng hoạt động ổn định trên nhiều nền tảng khác nhau. [13]

1.6 MongoDB

1.6.1 Khái niệm

MongoDB là hệ quản trị cơ sở dữ liệu thuộc nhóm NoSQL, hoạt động theo mô hình lưu trữ dữ liệu dạng tài liệu (document). Dữ liệu trong MongoDB được biểu diễn dưới định dạng BSON – một dạng mở rộng của JSON – giúp việc lưu trữ và truy xuất thông tin trở nên linh hoạt hơn so với mô hình bảng của cơ sở dữ liệu quan hệ truyền thống.

Thay vì phải thiết kế cấu trúc cố định gồm bảng, cột và quan hệ ràng buộc, MongoDB cho phép mỗi document có cấu trúc dữ liệu khác nhau. Điều này tạo thuận lợi cho các ứng dụng cần xử lý dữ liệu đa dạng, thường xuyên thay đổi hoặc có khối lượng lớn. Nhờ tính linh hoạt cao, MongoDB được sử dụng phổ biến trong các hệ thống web hiện đại, ứng dụng thời gian thực và các nền tảng dữ liệu lớn.

MongoDB cũng cung cấp dịch vụ điện toán đám mây mang tên MongoDB Atlas, hỗ trợ triển khai cơ sở dữ liệu trực tuyến với khả năng mở rộng, sao lưu và bảo mật tự động. Giải pháp này giúp doanh nghiệp giảm bớt chi phí quản lý hạ tầng và dễ dàng vận hành hệ thống trên môi trường cloud.

1.6.2 Tính năng nổi bật của MongoDB

Một trong những đặc điểm quan trọng của MongoDB là khả năng lưu trữ dữ liệu dưới dạng document thay vì bảng dữ liệu truyền thống. Mỗi document có thể chứa nhiều kiểu dữ liệu khác nhau và được tổ chức linh hoạt, giúp lập trình viên dễ dàng biểu diễn các dữ liệu phức tạp hoặc phi cấu trúc.

MongoDB không yêu cầu schema cố định cho toàn bộ hệ thống. Nhờ đó, việc thêm mới hoặc chỉnh sửa trường dữ liệu có thể thực hiện mà không cần thay đổi cấu trúc cơ sở dữ liệu tổng thể. Điều này giúp quá trình phát triển và nâng cấp ứng dụng trở nên nhanh chóng hơn.

Hệ quản trị này còn hỗ trợ sharding để phân chia dữ liệu trên nhiều máy chủ khác nhau, giúp hệ thống hoạt động ổn định khi lượng truy cập hoặc dữ liệu tăng cao. Đồng thời, cơ chế Replica Set hỗ trợ sao lưu và đồng bộ dữ liệu tự động, góp phần tăng khả năng dự phòng và đảm bảo tính sẵn sàng cho hệ thống.

MongoDB cung cấp nhiều công cụ truy vấn mạnh mẽ như Aggregation Framework, hỗ trợ lọc, nhóm, tính toán và phân tích dữ liệu trực tiếp trong cơ sở dữ liệu. Hệ thống cũng tương thích với nhiều ngôn ngữ lập trình phổ biến như JavaScript, Java, Python hay Node.js, giúp việc tích hợp vào ứng dụng trở nên thuận tiện hơn.

1.6.3 Ưu điểm của MongoDB

MongoDB có khả năng xử lý dữ liệu với hiệu suất cao và hỗ trợ mở rộng tốt nhờ kiến trúc phân tán. Hệ thống phù hợp với các ứng dụng có lượng dữ liệu lớn hoặc số lượng truy cập cao như mạng xã hội, thương mại điện tử và hệ thống quản lý dữ liệu thời gian thực.

Việc sử dụng cấu trúc JSON/BSON giúp MongoDB trở nên dễ tiếp cận với lập trình viên, đặc biệt trong các dự án sử dụng JavaScript hoặc Node.js. Dữ liệu được biểu diễn trực quan và gần giống với object trong lập trình nên quá trình thao tác và xử lý trở nên đơn giản hơn.

MongoDB còn mang lại tính linh hoạt cao khi làm việc với dữ liệu không đồng nhất. Nhà phát triển có thể thay đổi cấu trúc dữ liệu theo nhu cầu mà không phải chỉnh sửa toàn bộ hệ thống như trong các cơ sở dữ liệu quan hệ. Điều này giúp rút ngắn thời gian phát triển và dễ thích nghi với yêu cầu thay đổi của dự án.

Cộng đồng sử dụng MongoDB rất lớn cùng với tài liệu hướng dẫn phong phú giúp việc học tập và triển khai hệ thống trở nên thuận lợi hơn. Các công cụ hỗ trợ phân tích và tổng hợp dữ liệu cũng góp phần nâng cao khả năng xử lý dữ liệu phức tạp trong ứng dụng.

1.6.4 Nhược điểm của MongoDB

Mặc dù có tính linh hoạt cao, MongoDB thường tiêu tốn nhiều bộ nhớ hơn so với các hệ quản trị cơ sở dữ liệu quan hệ do sử dụng định dạng BSON để lưu trữ dữ liệu. Khi hệ thống chứa lượng dữ liệu lớn, yêu cầu tài nguyên phần cứng cũng tăng đáng kể.

MongoDB chưa thực sự phù hợp với những hệ thống yêu cầu tính toàn vẹn dữ liệu nghiêm ngặt hoặc các giao dịch phức tạp như lĩnh vực ngân hàng và tài chính.

Việc thiếu các ràng buộc chặt chẽ về schema có thể dẫn đến dữ liệu không đồng nhất nếu không được kiểm soát tốt trong quá trình phát triển.

Khả năng xử lý quan hệ dữ liệu của MongoDB cũng không mạnh như các cơ sở dữ liệu SQL truyền thống. Hệ thống không hỗ trợ JOIN theo cách thông thường, vì vậy khi làm việc với dữ liệu có nhiều mối liên kết phức tạp, lập trình viên cần thiết kế cấu trúc dữ liệu hợp lý để tránh phát sinh nhiều truy vấn không cần thiết.

Việc triển khai các tính năng nâng cao như sharding hoặc cấu hình bảo mật cũng đòi hỏi kiến thức chuyên môn tương đối cao. Nếu cấu hình không hợp lý, hệ thống có thể gặp vấn đề về hiệu suất hoặc tiềm ẩn rủi ro liên quan đến an toàn dữ liệu. [14]

1.7 Các nghiệp vụ liên quan đến đề tài

Trong quá trình xây dựng website gợi ý thực đơn dinh dưỡng thông minh, hệ thống được thiết kế nhằm hỗ trợ người dùng quản lý chế độ ăn uống, theo dõi thông tin dinh dưỡng và cá nhân hóa trải nghiệm sử dụng. Các nghiệp vụ chính của hệ thống tập trung vào việc quản lý tài khoản, quản lý thực đơn, hỗ trợ gợi ý thông minh, theo dõi dữ liệu sức khỏe và quản trị hệ thống. Những nghiệp vụ này giúp hệ thống hoạt động ổn định, đáp ứng tốt nhu cầu của người dùng trong việc xây dựng lối sống lành mạnh.

1.7.1 Nghiệp vụ quản lý người dùng

1.7.1.1 Đăng ký tài khoản

Người dùng có thể tạo tài khoản mới bằng cách nhập tên đăng nhập, mật khẩu và một số thông tin cá nhân cơ bản như họ tên hoặc số điện thoại. Hệ thống sẽ kiểm tra tính hợp lệ của tên đăng nhập nhằm tránh trùng lặp với các tài khoản đã tồn tại trước đó.

Sau khi đăng ký thành công, thông tin tài khoản sẽ được lưu trữ trong cơ sở dữ liệu để phục vụ cho quá trình đăng nhập và quản lý người dùng. Đồng thời, hệ thống cũng ghi nhận trạng thái tài khoản và lịch sử đăng ký nhằm hỗ trợ kiểm tra và đảm bảo tính an toàn cho hệ thống.

1.7.1.2 Đăng ký tài khoản

Người dùng sử dụng tên đăng nhập và mật khẩu để truy cập vào hệ thống. Sau khi xác thực thành công, hệ thống sẽ cấp token đăng nhập nhằm duy trì phiên làm việc và tăng cường bảo mật dữ liệu.

Hệ thống hỗ trợ phân quyền giữa người dùng thông thường và quản trị viên để kiểm soát quyền truy cập các chức năng khác nhau. Toàn bộ lịch sử đăng nhập đều

được lưu lại nhằm phục vụ cho việc kiểm tra và quản lý hoạt động người dùng khi cần thiết.

1.7.1.3 Quản lý thông tin cá nhân

Người dùng có thể cập nhật hồ sơ cá nhân như chiều cao, cân nặng, tuổi, giới tính, mục tiêu sức khỏe hoặc chế độ ăn mong muốn. Những dữ liệu này được sử dụng để hỗ trợ quá trình gợi ý thực đơn phù hợp hơn với từng cá nhân.

1.7.2 Nghiệp vụ quản lý thực đơn dinh dưỡng

1.7.2.1 Thêm mới thực đơn

Quản trị viên có thể bổ sung thực đơn mới vào hệ thống bằng cách nhập các thông tin như tên món ăn, nguyên liệu, giá trị dinh dưỡng, hình ảnh minh họa và hướng dẫn chế biến.

Trước khi lưu dữ liệu, hệ thống sẽ tiến hành kiểm tra nhằm đảm bảo thông tin đầy đủ và chính xác. Sau khi được thêm thành công, thực đơn sẽ hiển thị trên website để người dùng tra cứu và sử dụng.

1.7.2.2 Cập nhật thực đơn

Quản trị viên có thể chỉnh sửa thông tin thực đơn khi cần thay đổi nguyên liệu, điều chỉnh thành phần dinh dưỡng hoặc cập nhật hình ảnh mới.

Mọi thao tác cập nhật đều được lưu lại nhằm phục vụ cho việc quản lý và kiểm tra dữ liệu khi cần thiết.

1.7.2.3 Xóa hoặc ẩn thực đơn

Hệ thống cho phép quản trị viên xóa hoặc ẩn các thực đơn không còn phù hợp hoặc không còn sử dụng.

Đối với những thực đơn đã được người dùng lưu hoặc sử dụng trong hệ thống gợi ý, dữ liệu sẽ được kiểm tra trước khi thực hiện xóa để tránh ảnh hưởng đến các chức năng liên quan.

1.7.2.4 Tìm kiếm và hiển thị thực đơn

Người dùng có thể tìm kiếm thực đơn dựa trên nhiều tiêu chí như tên món ăn, nhóm thực phẩm, lượng calo hoặc mục tiêu dinh dưỡng.

Hệ thống hỗ trợ hiển thị đầy đủ thông tin về món ăn bao gồm thành phần nguyên liệu, hàm lượng calo, protein, carbohydrate và chất béo để giúp người dùng lựa chọn thực đơn phù hợp.

1.7.3 Nghiệp vụ gợi ý thực đơn thông minh

1.7.3.1 Gợi ý thực đơn cá nhân hóa

Hệ thống phân tích thông tin cá nhân và lịch sử sử dụng để đề xuất các thực đơn phù hợp với mục tiêu của từng người dùng như tăng cân, giảm cân hoặc duy trì vóc dáng.

Cơ chế gợi ý được xây dựng dựa trên dữ liệu dinh dưỡng và hành vi sử dụng nhằm nâng cao độ chính xác của kết quả đề xuất.

1.7.3.2 Xây dựng lộ trình thực đơn

Người dùng có thể lựa chọn khoảng thời gian mong muốn để hệ thống tự động tạo kế hoạch ăn uống theo ngày hoặc theo tuần.

Mỗi lộ trình sẽ bao gồm các bữa ăn được cân đối dựa trên nhu cầu năng lượng và mục tiêu dinh dưỡng của người dùng.

1.7.3.3 Tương tác với AI Chatbot

Hệ thống tích hợp chatbot AI nhằm hỗ trợ giải đáp các câu hỏi liên quan đến dinh dưỡng, thực phẩm và chế độ ăn uống.

Người dùng có thể tương tác trực tiếp với chatbot để nhận tư vấn hoặc đề xuất món ăn phù hợp với nhu cầu cá nhân.

1.7.4 Nghiệp vụ quản lý dữ liệu sức khỏe

1.7.4.1 Theo dõi chỉ số cơ thể

Người dùng có thể nhập và theo dõi các thông tin như cân nặng, chiều cao và chỉ số BMI.

Dựa trên các dữ liệu này, hệ thống sẽ đưa ra đánh giá cơ bản về tình trạng sức khỏe và hỗ trợ lựa chọn thực đơn phù hợp.

1.7.4.2 Lưu lịch sử sử dụng

Toàn bộ hoạt động như tìm kiếm món ăn, lựa chọn thực đơn hoặc cập nhật thông tin cá nhân đều được lưu trữ trong hệ thống.

Dữ liệu lịch sử giúp cải thiện khả năng gợi ý và hỗ trợ người dùng theo dõi quá trình thay đổi chế độ dinh dưỡng theo thời gian.

1.7.5 Nghiệp vụ thông báo và nhắc nhở

1.7.5.1 Gửi thông báo tự động

Hệ thống có khả năng gửi thông báo khi có thực đơn mới, cập nhật dữ liệu dinh dưỡng hoặc nhắc nhở người dùng theo dõi kế hoạch ăn uống hằng ngày.

Thông báo được hiển thị trực tiếp trên website để người dùng dễ dàng theo dõi.

1.7.5.2 Quản lý thông báo cá nhân

Người dùng có thể xem danh sách thông báo, đánh dấu đã đọc hoặc tùy chỉnh chế độ nhận thông báo theo nhu cầu sử dụng.

Lịch sử thông báo được lưu lại nhằm hỗ trợ quản lý và kiểm tra thông tin dễ dàng hơn.

1.7.6 Nghiệp vụ quản trị hệ thống

1.7.6.1 Quản lý tài khoản người dùng

Quản trị viên có quyền xem danh sách tài khoản, phân quyền sử dụng và xử lý các tài khoản vi phạm quy định của hệ thống.

Hệ thống hỗ trợ khóa hoặc mở khóa tài khoản khi phát hiện hoạt động bất thường.

1.7.6.2 Quản lý dữ liệu hệ thống

Quản trị viên có thể quản lý thực đơn, nguyên liệu, dữ liệu dinh dưỡng và các nội dung liên quan trong hệ thống.

Việc quản lý tập trung giúp đảm bảo dữ liệu luôn chính xác và đồng bộ.

1.7.6.3 Thống kê và báo cáo

Hệ thống hỗ trợ thống kê số lượng người dùng, lượt truy cập, thực đơn được quan tâm nhiều nhất và dữ liệu tương tác của người dùng.

Các báo cáo này giúp quản trị viên đánh giá hiệu quả hoạt động và cải thiện chất lượng hệ thống.

1.7.6.4 Quản lý cấu hình hệ thống

Quản trị viên có thể thay đổi các thông số cấu hình như kết nối cơ sở dữ liệu, cấu hình API hoặc cơ chế bảo mật của hệ thống.

Mọi thay đổi cấu hình đều được lưu lại nhằm đảm bảo an toàn và hỗ trợ khôi phục khi xảy ra sự cố.

CHƯƠNG 2: HIỆN THỰC HÓA NGHIÊN CỨU

2.1 Mô tả bài toán

Hiện nay, cùng với sự phát triển mạnh mẽ của công nghệ thông tin, nhu cầu chăm sóc sức khỏe và xây dựng chế độ ăn uống hợp lý đang được nhiều người quan tâm hơn trước. Tuy nhiên, phần lớn người dùng vẫn gặp không ít khó khăn trong việc lựa chọn thực đơn phù hợp với thể trạng, mục tiêu cá nhân cũng như điều kiện tài chính. Nhiều người chưa có đủ kiến thức về dinh dưỡng nên việc xây dựng khẩu phần ăn thường thiếu cân đối giữa các nhóm chất, dễ ảnh hưởng đến sức khỏe trong thời gian dài.

Bên cạnh đó, việc tìm kiếm thông tin về dinh dưỡng trên Internet hiện nay còn khá rời rạc và thiếu tính đồng bộ. Người dùng phải tham khảo từ nhiều nguồn khác nhau nhưng khó kiểm tra được độ chính xác của nội dung. Một số website hoặc ứng dụng hỗ trợ dinh dưỡng đã được phát triển, tuy nhiên nhiều hệ thống vẫn còn hạn chế trong việc cá nhân hóa thực đơn, chưa đáp ứng tốt nhu cầu riêng của từng người dùng. Ngoài ra, khả năng tương tác giữa người dùng và hệ thống còn chưa cao, đặc biệt là các chức năng hỗ trợ tư vấn trực tiếp hoặc đề xuất thực đơn tự động.

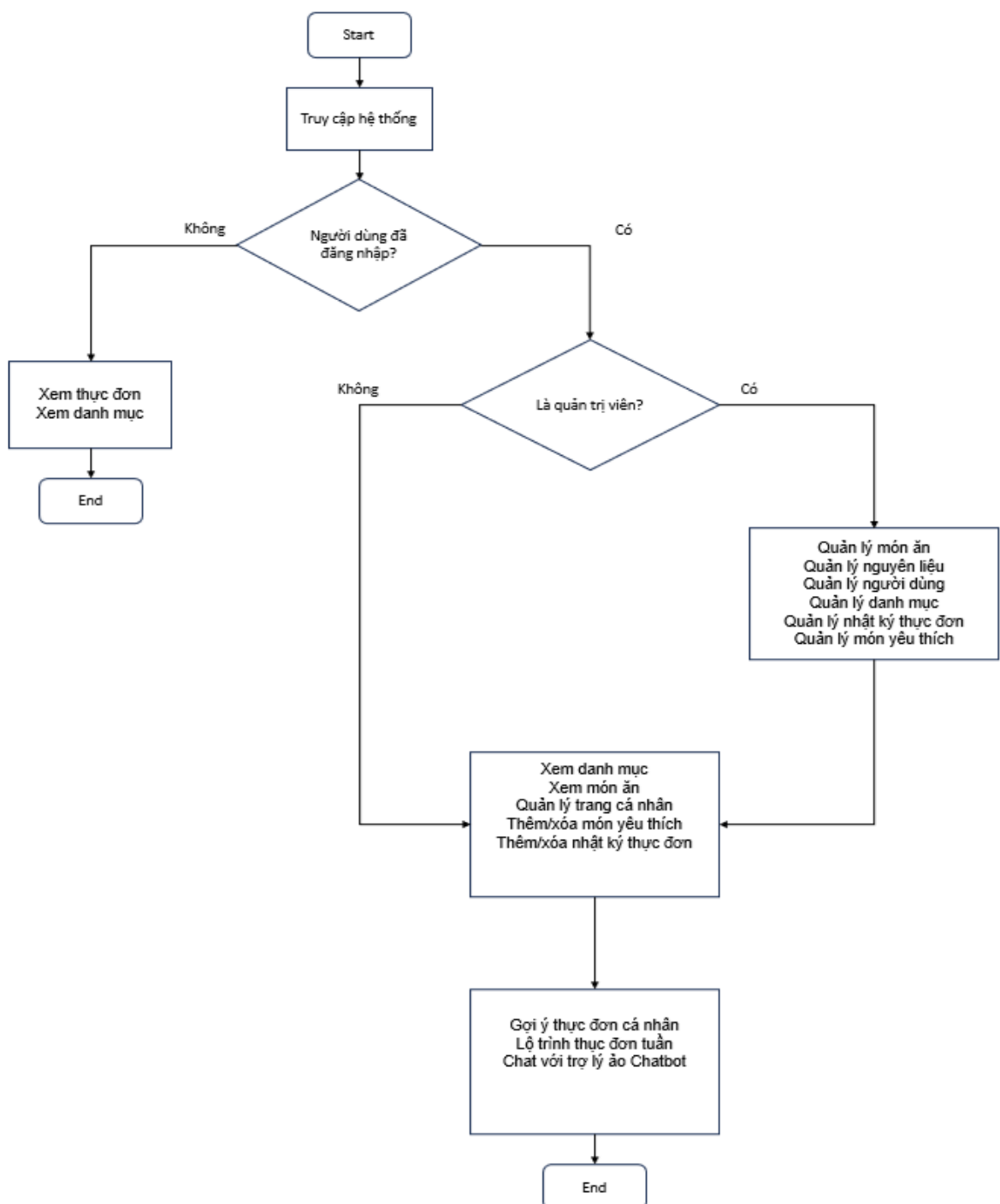
Trong thời đại chuyển đổi số, việc ứng dụng trí tuệ nhân tạo vào lĩnh vực chăm sóc sức khỏe đang dần trở thành xu hướng phổ biến. Các công nghệ AI có khả năng phân tích dữ liệu người dùng, từ đó đưa ra các gợi ý phù hợp với nhu cầu thực tế. Việc xây dựng một website gợi ý thực đơn dinh dưỡng không chỉ giúp người dùng tiết kiệm thời gian lựa chọn món ăn mà còn hỗ trợ thiết lập chế độ ăn hợp lý dựa trên nhiều yếu tố như chỉ số cơ thể, sở thích ăn uống, mục tiêu sức khỏe và mức chi tiêu cá nhân.

Xuất phát từ thực tế đó, đề tài “Xây dựng website gợi ý thực đơn dinh dưỡng thông minh” được thực hiện nhằm phát triển một nền tảng hỗ trợ người dùng trong việc lựa chọn và quản lý thực đơn hằng ngày. Hệ thống cho phép người dùng đăng ký tài khoản, thiết lập thông tin cá nhân và nhận các gợi ý thực đơn phù hợp. Ngoài ra, website còn tích hợp AI Chatbot hỗ trợ giải đáp các câu hỏi liên quan đến dinh dưỡng bằng ngôn ngữ tự nhiên, giúp tăng khả năng tương tác và hỗ trợ người dùng thuận tiện hơn trong quá trình sử dụng.

Bên cạnh chức năng gợi ý thực đơn, hệ thống còn hỗ trợ xây dựng lộ trình ăn uống theo tuần dựa trên nhu cầu của từng người dùng. Đồng thời, thuật toán gợi ý lại được áp dụng nhằm nâng cao độ chính xác của các đề xuất món ăn. Ngoài ra, quản trị

viên cũng có thể quản lý dữ liệu món ăn, nguyên liệu, danh mục và theo dõi hoạt động của hệ thống thông qua trang quản trị riêng.

Việc thực hiện đề tài không chỉ góp phần ứng dụng công nghệ thông tin và trí tuệ nhân tạo vào lĩnh vực dinh dưỡng mà còn mang ý nghĩa thực tiễn trong việc hỗ trợ người dùng xây dựng thói quen ăn uống khoa học hơn. Trong tương lai, hệ thống có thể tiếp tục được nâng cấp và bổ sung thêm nhiều chức năng nhằm cải thiện trải nghiệm người dùng và đáp ứng tốt hơn nhu cầu thực tế.



Hình 2.1. Flow Chart hệ thống

2.2 Phân tích chức năng

2.2.1 Đối với người quản trị

Người quản trị (admin) là đối tượng nắm giữ quyền kiểm soát cao nhất, chịu trách nhiệm vận hành, duy trì tính chính xác của dữ liệu và đảm bảo sự ổn định cho toàn bộ nền tảng NutriFood. Các chức năng dành cho quản trị viên bao gồm:

Quản lý người dùng: Quản trị viên có thẩm quyền xem danh sách toàn bộ thành viên, tìm kiếm và phân loại theo vai trò (admin, user) hoặc trạng thái tài khoản. Cung cấp các công cụ để cập nhật quyền hạn, khóa hoặc xóa các tài khoản có hành vi bất thường. Đồng thời, quản trị viên có thể theo dõi lịch sử truy cập để hỗ trợ giải quyết các vấn đề kỹ thuật cho người dùng khi cần thiết.

Quản lý dữ liệu ẩm thực: Đây là chức năng cốt lõi giúp duy trì "bộ não" của hệ thống. Quản trị viên chịu trách nhiệm thêm mới, kiểm duyệt, chỉnh sửa hoặc xóa các danh mục món ăn, nguyên liệu và công thức chế biến. Đảm bảo tính chuẩn xác tuyệt đối của bảng thành phần dinh dưỡng (Calories, Protein, Carbs, Fat) và mức giá tham khảo, làm cơ sở dữ liệu nền tảng cho thuật toán AI hoạt động hiệu quả.

Quản lý nhật ký và tương tác: Quản trị viên có khả năng giám sát luồng dữ liệu nhật ký bữa ăn của hệ thống (dưới dạng ẩn danh) nhằm phân tích xu hướng tiêu dùng. Quản lý danh sách các món ăn được lưu yêu thích nhiều nhất, đồng thời tiếp nhận và xử lý các phản hồi, báo cáo lỗi từ người dùng liên quan đến chất lượng thực đơn đề xuất.

Thống kê và báo cáo: Quản trị viên được cung cấp bảng điều khiển trực quan (Dashboard) để theo dõi các chỉ số phát triển của nền tảng như: sự gia tăng lượng người dùng mới, tổng số lộ trình thực đơn đã được AI tạo ra, hay tỷ lệ hoàn thành mục tiêu sức khỏe của cộng đồng. Hệ thống hỗ trợ xuất dữ liệu báo cáo và biểu đồ nhằm phục vụ công tác đánh giá hiệu năng và định hướng nâng cấp website.

Quản lý hệ thống: Quản trị viên nắm quyền thiết lập các cấu hình lõi, đặc biệt là quản lý kết nối và cập nhật API Key của hệ thống Trí tuệ nhân tạo (Gemini AI). Hệ thống cung cấp công cụ ghi nhận lỗi (error logs), giám sát tình trạng server, tự động sao lưu cơ sở dữ liệu (MongoDB) và cảnh báo khi có sự cố tắc nghẽn API.

2.2.2 Đối với người dùng

Người dùng (user) là đối tượng phục vụ trung tâm của đồ án, được cung cấp chuỗi tính năng cá nhân hóa chuyên sâu nhằm hỗ trợ duy trì lối sống khoa học:

Quản lý tài khoản cá nhân: Người dùng có thể đăng ký, đăng nhập an toàn và khởi tạo hồ sơ thể chất. Hệ thống cho phép cập nhật linh hoạt các chỉ số sức khỏe (chiều cao, cân nặng để tự động nội suy BMI), thiết lập mục tiêu vóc dáng (giảm cân, tăng cơ, duy trì), cũng như khai báo sở thích ẩm thực và giới hạn ngân sách ăn uống.

Khám phá và lưu trữ thực đơn: Người dùng có thể tra cứu thư viện ẩm thực thông qua bộ lọc thông minh (theo nhóm vi chất, lượng calo, danh mục món). Chi tiết mỗi món ăn đều hiển thị rõ công thức, thành phần dinh dưỡng và chi phí. Hệ thống tích hợp tính năng lưu trữ "Món ăn yêu thích" giúp người dùng cá nhân hóa bộ sưu tập thực đơn của riêng mình.

Sử dụng Trợ lý AI và Lộ trình: Người dùng được trải nghiệm tính năng cốt lõi (Meal Plan Generator) – tự động nhận lộ trình thực đơn 3 bữa/ngày do AI thiết kế riêng biệt dựa trên hồ sơ thể chất và ngân sách. Ngoài ra, tích hợp AI Chatbot cho phép người dùng giao tiếp bằng ngôn ngữ tự nhiên để nhận tư vấn dinh dưỡng hoặc mẹo thay thế nguyên liệu linh hoạt.

Theo dõi nhật ký dinh dưỡng: Cung cấp không gian làm việc (Bảng điều khiển) để ghi nhận thực tế các món đã tiêu thụ. Người dùng có thể tự thêm món thủ công hoặc sử dụng tính năng "Ghi nhận 1 chạm" trực tiếp từ lộ trình AI đề xuất. Hệ thống tự động tính toán, cộng dồn và hiển thị biểu đồ Macro (Calo, P, C, F) theo thời gian thực.

Theo dõi tiến độ sức khỏe: Người dùng có thể xem lại toàn bộ lịch sử ăn uống trong quá khứ, đối chiếu tổng lượng năng lượng đã nạp so với mục tiêu hệ thống đề ra ban đầu, từ đó dễ dàng tự điều chỉnh thói quen dinh dưỡng để đạt được kết quả thể trạng tốt nhất.

2.2.3 Yêu cầu phi chức năng

Tính bảo mật: Đảm bảo an toàn tuyệt đối cho cơ sở dữ liệu sức khỏe cá nhân. Mật khẩu phải được băm (hashing) bảo mật, sử dụng JSON Web Token (JWT) cho phiên đăng nhập. Hệ thống phải có cơ chế phòng chống các đợt tấn công phổ biến (XSS, CSRF) và giới hạn kích thước dữ liệu tải lên (payload limit) để tránh quá tải máy chủ.

Tính ổn định: Server Node.js phải duy trì khả năng hoạt động liên tục, xử lý tốt các ngoại lệ (Global Error Handling) để không bị sập (crash) khi có lỗi logic. Đặc biệt, hệ thống phải trang bị cơ chế dự phòng (Fallback mechanism) tự động trích xuất món

ăn từ cơ sở dữ liệu nội bộ thay thế ngay lập tức khi máy chủ AI quốc tế gặp sự cố (lỗi 503).

Tính mở rộng: Kiến trúc phần mềm phải được xây dựng chuẩn mô hình RESTful API, phân tách rõ ràng giữa Frontend và Backend. Cấu trúc linh hoạt này cho phép dễ dàng tích hợp thêm các dịch vụ bên thứ ba (như cổng thanh toán VNPAY, gửi email tự động) hoặc nâng cấp phiên bản mô hình ngôn ngữ lớn (LLMs) trong tương lai mà không làm vỡ cấu trúc lõi.

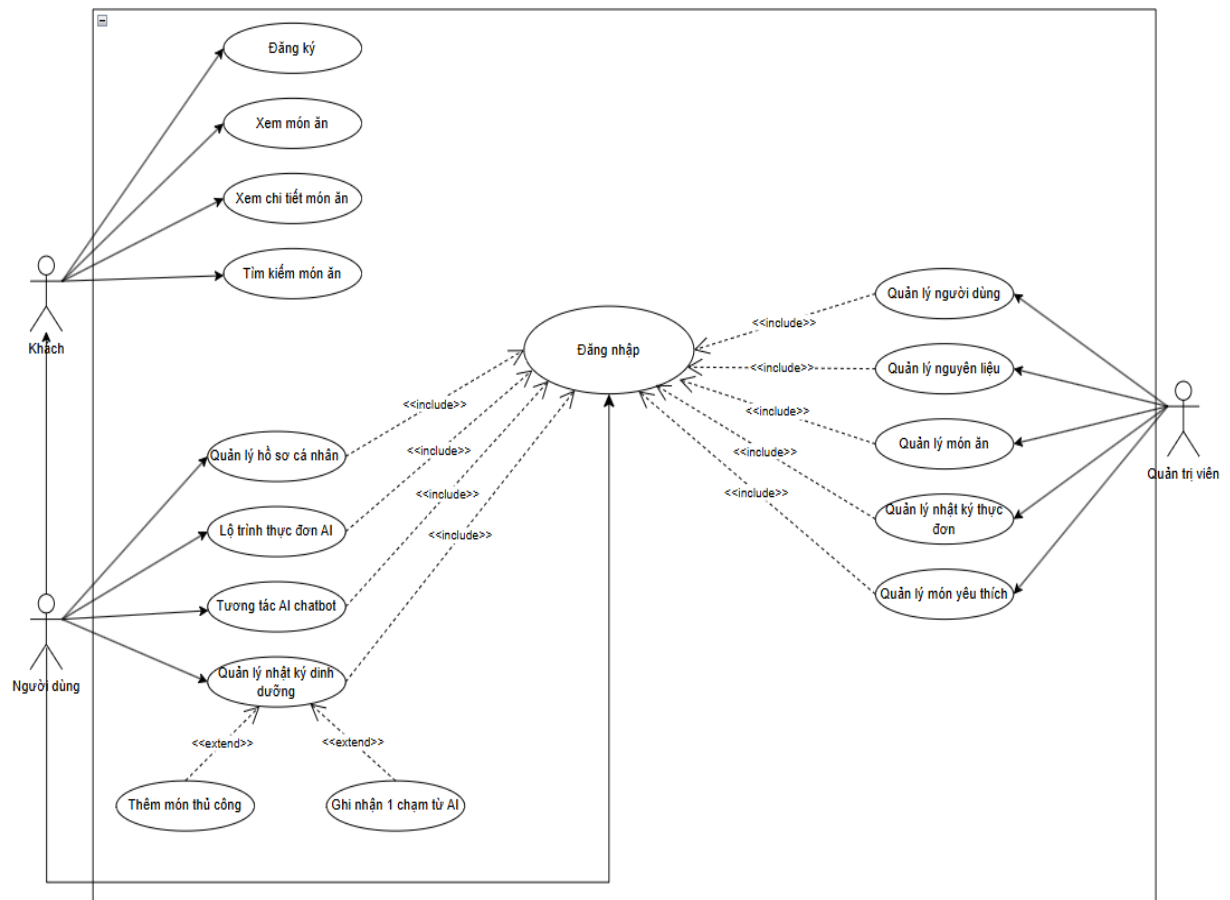
Tính thân thiện: Giao diện người dùng (UI/UX) phải được thiết kế hiện đại, trực quan với các biểu đồ theo dõi dinh dưỡng dễ đọc. Website cần tuân thủ nguyên tắc thiết kế phản hồi (Responsive Web Design), đảm bảo các thao tác thêm nhật ký, xem thực đơn hoạt động mượt mà trên mọi kích thước màn hình (PC, máy tính bảng, điện thoại di động).

Hiệu năng: Ứng dụng phải tối ưu hóa thời gian tải trang và giảm độ trễ khi gọi API AI. Áp dụng kỹ thuật RAG (Retrieval-Augmented Generation) khéo léo để đóng gói dữ liệu món ăn, đảm bảo thuật toán AI phản hồi lộ trình thực đơn nhanh chóng nhất có thể, mang lại trải nghiệm tương tác thời gian thực cho hội viên.

Tính chính xác và khoa học: Yêu cầu tiên quyết đối với một ứng dụng sức khỏe là tính chuẩn xác. Mọi công thức nội suy (tính BMI, BMR, TDEE) và bộ thông số Macro (calo, protein, carbs, fat) của món ăn phải tuân thủ nghiêm ngặt các tiêu chuẩn y khoa và chuyên gia dinh dưỡng hiện hành, đảm bảo lộ trình đề xuất là an toàn và có giá trị thực tiễn cao.

2.3 Sơ đồ Use Case

2.3.1 Sơ đồ Use Case tổng quát



Hình 2.2. Use Case tổng quát

Hệ thống Gợi ý Thực đơn Dinh dưỡng Thông minh (NutriFood) được xây dựng nhằm cung cấp một nền tảng trực tuyến cá nhân hóa, hỗ trợ cộng đồng thiết lập và duy trì thói quen ăn uống khoa học. Dựa trên biểu đồ Use Case tổng quát, luồng vận hành của hệ thống được phân quyền chặt chẽ cho ba nhóm đối tượng tác nhân (Actor) chính: Khách, Người dùng và Quản trị viên.

Đối với Khách (người truy cập vãng lai), nền tảng mở cửa tự do cho các trải nghiệm khám phá cơ bản. Đối tượng này có thể thực hiện việc tìm kiếm món ăn, xem danh sách tổng hợp, xem chi tiết từng món ăn và tiến hành đăng ký tài khoản để chính thức tham gia vào hệ sinh thái của website.

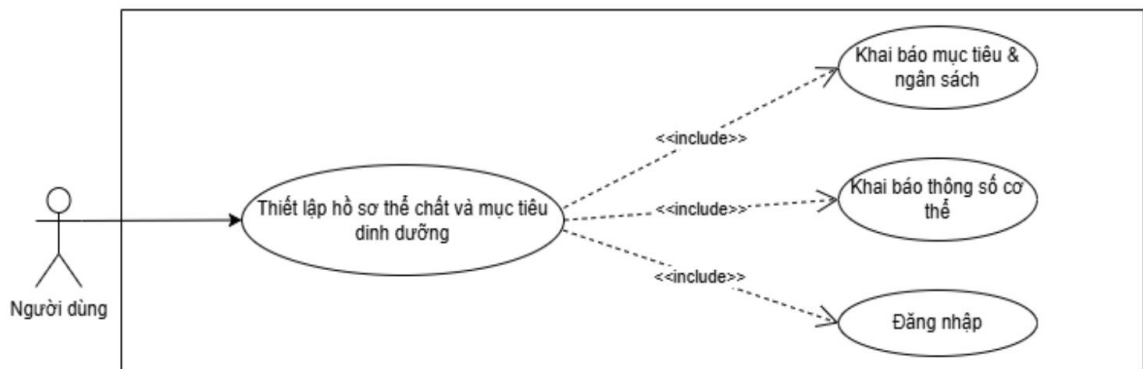
Khi trở thành Người dùng chính thức và thông qua bước bắt buộc là đăng nhập, cá nhân sẽ được kế thừa toàn bộ quyền hạn của Khách, đồng thời mở khóa các tính năng chuyên sâu. Cụ thể, hội viên có thể quản lý hồ sơ cá nhân (cập nhật chỉ số cơ thể), nhận lộ trình thực đơn AI, tương tác AI Chatbot để nhận tư vấn, và quản lý nhật ký dinh dưỡng. Đặc biệt, module nhật ký dinh dưỡng được thiết kế linh hoạt với hai tùy

chọn mở rộng: thêm món thủ công đối với thức ăn ngoài hệ thống, hoặc ghi nhận 1 chạm từ AI để tự động hóa việc tính toán calo hằng ngày.

Về phía Quản trị viên (Admin), sau khi xác thực đăng nhập, bộ phận điều hành được cấp quyền truy cập vào không gian kiểm soát cốt lõi nhằm đảm bảo tính toàn vẹn của dữ liệu. Các đặc quyền này bao gồm: quản lý người dùng, quản lý nguyên liệu, quản lý món ăn, cũng như giám sát quản lý nhật ký thực đơn và quản lý món yêu thích của cộng đồng.

Thông qua sự phân chia vai trò rõ ràng, kết hợp cùng sức mạnh của Trí tuệ Nhân tạo, đề án không chỉ số hóa quy trình đề xuất thực đơn mà còn mang lại một không gian tương tác liền mạch, bảo mật và tối ưu trải nghiệm cho mọi đối tượng tiếp cận.

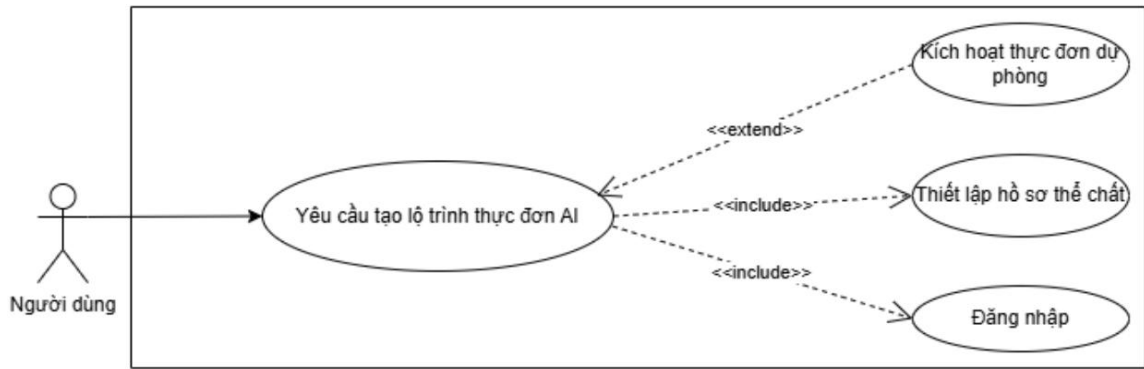
2.3.2 Sơ đồ Use Case thiết lập hồ sơ thể chất và mục tiêu dinh dưỡng



Hình 2.3. Use Case thiết lập hồ sơ thể chất và mục tiêu dinh dưỡng

Chức năng thiết lập hồ sơ thể chất và mục tiêu dinh dưỡng là bước khởi tạo nền tảng của hệ thống NutriFood, yêu cầu người dùng phải đăng nhập trước tiên để hệ thống có thể định danh và thấu hiểu trọn vẹn thể trạng của từng cá nhân. Tại đây, người dùng sẽ tiến hành khai báo các chỉ số cơ thể cơ bản như chiều cao, cân nặng. Ngay khi tiếp nhận dữ liệu, hệ thống sẽ tự động tính toán và nội suy ra các chỉ số y khoa quan trọng như BMI, BMR và tổng năng lượng tiêu hao hàng ngày (TDEE). Tiếp đó, người dùng có thể linh hoạt thiết lập mục tiêu vóc dáng mong muốn chẳng hạn như giảm mỡ, tăng cơ hay duy trì cân nặng, đồng thời định mức ngân sách chi tiêu phù hợp cho việc ăn uống. Tất cả những thông tin này sẽ được tổng hợp lại thành một "hồ sơ sức khỏe số" hoàn chỉnh và bảo mật. Nhờ đó, Trợ lý ảo AI có được một bộ dữ liệu đầu vào mang tính cá nhân hóa tuyệt đối, làm cơ sở vững chắc để kiến tạo ra những lộ trình thực đơn vừa chuẩn xác về mặt khoa học, vừa bám sát nhu cầu và túi tiền, giúp người dùng dễ dàng theo đuổi lối sống lành mạnh.

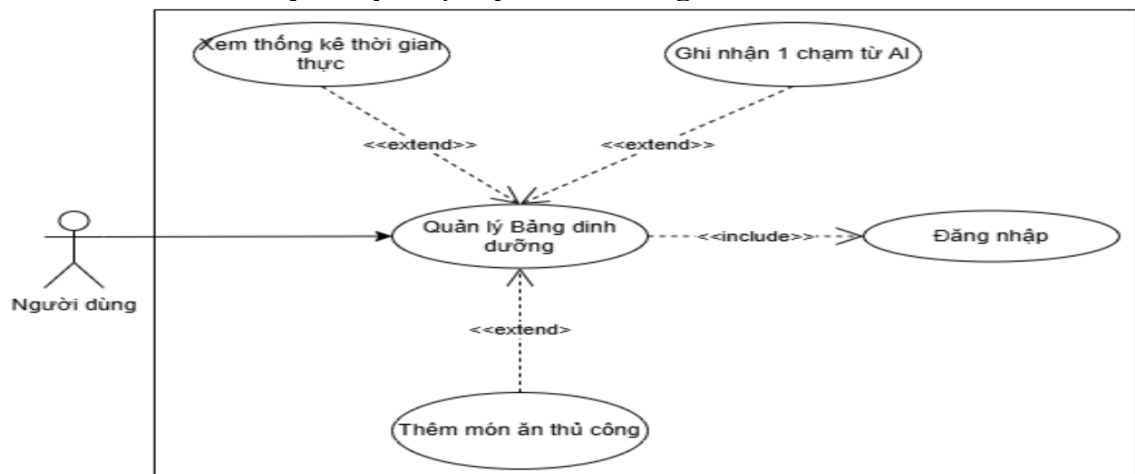
2.3.3 Sơ đồ Use Case kiến tạo lộ trình thực đơn AI



Hình 2.4. Use Case kiến tạo lộ trình thực đơn AI

Chức năng yêu cầu tạo lộ trình thực đơn AI được xem là "trái tim" công nghệ của hệ thống NutriFood, mang đến cho người dùng một chế độ ăn uống hoàn toàn tự động và mang đậm tính cá nhân hóa. Tuy nhiên, để Trợ lý ảo AI có thể "hiểu" và thiết kế ra một thực đơn chuẩn xác nhất, quá trình này đòi hỏi hai điều kiện tiên quyết: người dùng bắt buộc phải đăng nhập để định danh tài khoản, đồng thời phải hoàn tất bước thiết lập hồ sơ thể chất từ trước. Chỉ khi nắm trong tay bộ dữ liệu nền tảng này, AI mới có đủ cơ sở để phân tích và đưa ra các gợi ý phù hợp với thể trạng. Một điểm nhấn cực kỳ tinh tế của luồng chức năng này là sự xuất hiện của cơ chế xử lý dự phòng linh hoạt. Cụ thể, trong quá trình gọi API, nếu máy chủ AI gặp sự cố gián đoạn hoặc phản hồi quá lâu, hệ thống sẽ lập tức rẽ nhánh để tự động kích hoạt thực đơn dự phòng. Cơ chế "cứu cánh" này nhằm đảm bảo rằng dù trong bất kỳ tình huống rủi ro kỹ thuật nào, người dùng vẫn luôn nhận được một lộ trình dinh dưỡng thay thế tức thời, giúp trải nghiệm sử dụng luôn được liền mạch và không làm gián đoạn kế hoạch ăn uống của họ.

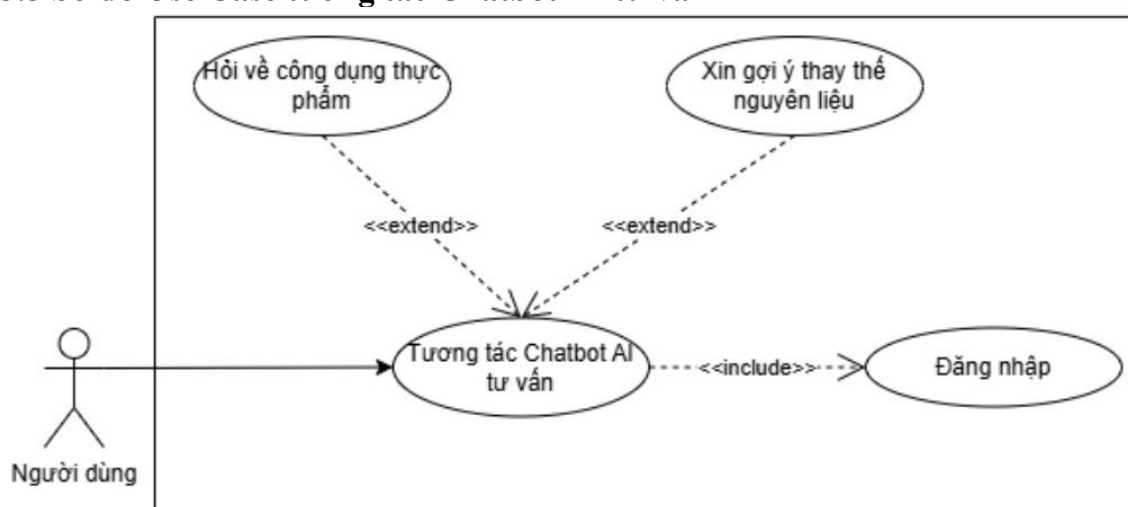
2.3.4 Sơ đồ Use Case quản lý nhật ký dinh dưỡng



Hình 2.5. Use Case quản lý nhật ký dinh dưỡng

Chức năng quản lý nhật ký dinh dưỡng đóng vai trò như một trung tâm kiểm soát năng lượng hằng ngày của nền tảng NutriFood, yêu cầu người dùng phải đăng nhập trước tiên để hệ thống có thể xác thực và bảo mật tuyệt đối dữ liệu nhật ký của riêng họ. Ngay khi bước vào không gian bảng điều khiển này, hội viên sẽ được cung cấp một loạt các công cụ linh hoạt để ghi nhận và theo dõi chế độ ăn uống. Để tối ưu hóa sự tiện lợi, hệ thống mở rộng thêm tính năng "ghi nhận 1 chạm từ AI", cho phép người dùng đẩy trực tiếp toàn bộ các món ăn do Trợ lý ảo gợi ý vào nhật ký chỉ với một thao tác click duy nhất. Song song đó, nhằm đáp ứng các tình huống ăn uống thực tế, người dùng hoàn toàn có thể chủ động "thêm món ăn thủ công" để ghi lại những bữa ăn vặt hoặc thực phẩm ngoài luồng, đảm bảo không bỏ sót bất kỳ lượng calo nào. Đặc biệt, mọi thay đổi và biến động trong khẩu phần ăn sẽ lập tức được hệ thống phân tích và hiển thị thông qua chức năng "xem thống kê thời gian thực". Sự kết hợp nhịp nhàng của các tính năng này giúp người dùng có một cái nhìn trực quan nhất về tiến độ nạp năng lượng (calo, protein, carbs, fat) trong ngày, từ đó dễ dàng chủ động điều chỉnh hành vi ăn uống sao cho bám sát với mục tiêu sức khỏe đã đề ra.

2.3.5 Sơ đồ Use Case tương tác Chatbot AI tư vấn

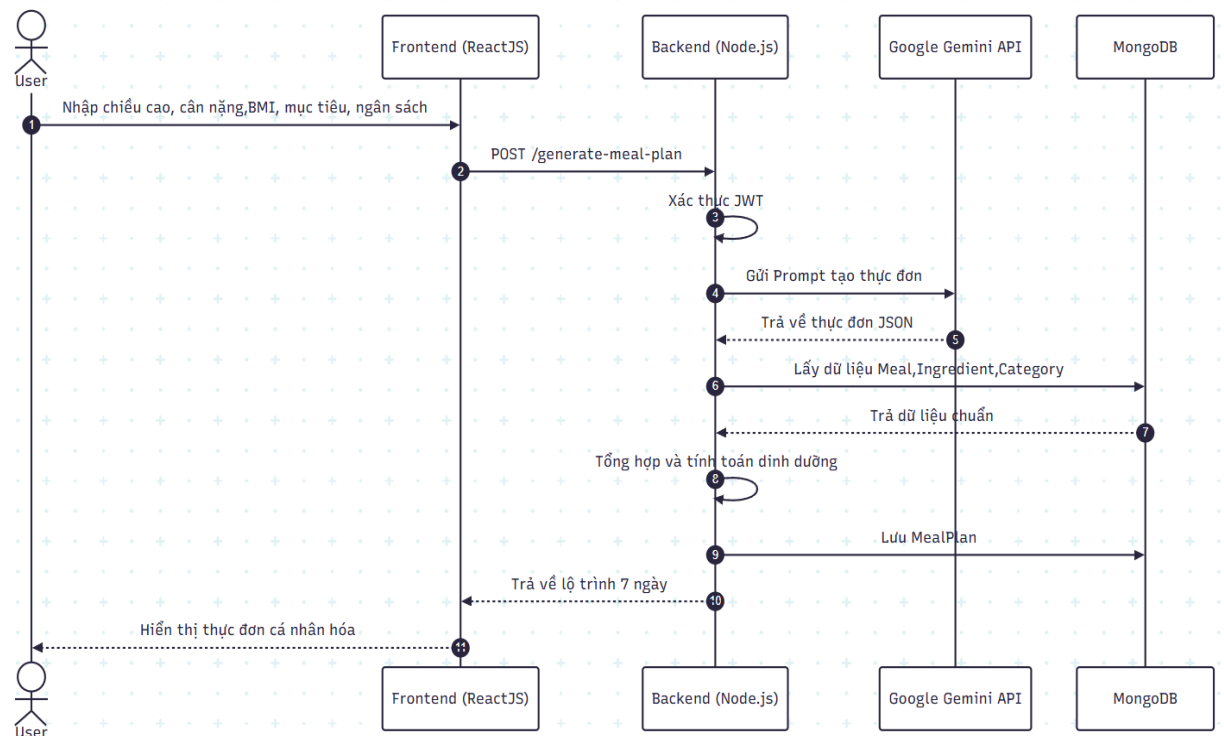


Hình 2.6. Use Case tương tác Chatbot AI tư vấn

Chức năng tương tác Chatbot AI tư vấn đóng vai trò như một chuyên gia dinh dưỡng ảo túc trực 24/7 của nền tảng NutriFood, yêu cầu người dùng phải thực hiện đăng nhập trước tiên để hệ thống có thể nhận diện và đồng bộ hóa với hồ sơ sức khỏe cá nhân của họ. Khi bước vào không gian trò chuyện, người dùng có thể thoải mái giao tiếp bằng ngôn ngữ tự nhiên để giải quyết các vấn đề ăn uống hằng ngày. Trong quá trình trò chuyện, hệ thống cung cấp các tiện ích mở rộng vô cùng linh hoạt. Điển hình là người dùng có thể "xin gợi ý thay thế nguyên liệu", một tính năng cực kỳ "cứu cánh"

khi họ bị dị ứng với một thành phần nào đó trong thực đơn hoặc đơn giản là tủ lạnh ở nhà đang thiếu đồ. Song song đó, người dùng cũng có thể chủ động "hỏi về công dụng thực phẩm" để hiểu căn kẽ hơn về giá trị dinh dưỡng, lượng calo hay lợi ích cụ thể của một món ăn trước khi quyết định sử dụng. Thông qua những cuộc hội thoại thông minh và liên mạch này, Chatbot AI không chỉ giúp giải đáp các thắc mắc tức thời mà còn trở thành một người bạn đồng hành đáng tin cậy, giúp người dùng trau dồi kiến thức dinh dưỡng và tự tin hơn trên hành trình chăm sóc sức khỏe.

2.3.6 Sơ đồ tuần tự chức năng kiến tạo lộ trình thực đơn AI (Sequence diagram)



Hình 2.7. Sơ đồ tuần tự chức năng kiến tạo lộ trình thực đơn AI

Hình 2.7 mô tả trình tự tương tác giữa các thành phần trong quá trình kiến tạo lộ trình thực đơn AI. Sau khi người dùng nhập các thông tin về thể trạng, mục tiêu dinh dưỡng và ngân sách, giao diện ReactJS sẽ gửi yêu cầu đến máy chủ Node.js. Backend tiến hành xác thực thông tin người dùng, tổng hợp dữ liệu hồ sơ dinh dưỡng và xây dựng Prompt gửi đến Google Gemini API để phân tích. Kết quả thực đơn được AI trả về dưới dạng dữ liệu có cấu trúc, sau đó hệ thống tiếp tục đối chiếu với cơ sở dữ liệu MongoDB nhằm bổ sung thông tin dinh dưỡng, giá thành và dữ liệu món ăn chuẩn hóa. Cuối cùng, backend tổng hợp kết quả, lưu lộ trình thực đơn nếu cần thiết và trả về giao diện người dùng. Trong trường hợp AI không phản hồi hoặc xảy ra lỗi kết nối, hệ thống sẽ tự động kích hoạt cơ chế thực đơn dự phòng nhằm đảm bảo tính liên tục của dịch vụ.

2.4 Xây dựng website

2.4.1 Cài đặt môi trường

Frontend: ReactJs + Vite + Tailwind CSS: Thư viện JavaScript phổ biến giúp xây dựng giao diện người dùng tương tác cao, linh hoạt, dễ mở rộng. Kết hợp với Vite để tối ưu tốc độ khởi động và build dự án. Sử dụng Tailwind CSS để thiết kế giao diện nhanh chóng, đồng nhất và Responsive trên mọi thiết bị.

Backend: NodeJs + ExpressJs: Cung cấp môi trường backend xử lý yêu cầu người dùng nhanh chóng, dễ tích hợp với MongoDB, hỗ trợ tốt cho các API RESTful và đóng vai trò trung gian xử lý, kết nối với API của Mô hình ngôn ngữ lớn (Gemini AI).

Hệ quản trị cơ sở dữ liệu MongoDB: Cơ sở dữ liệu NoSQL dạng document, phù hợp với dữ liệu linh hoạt, dễ mở rộng, hỗ trợ lưu trữ các đối tượng JSON phức tạp như thông số sức khỏe, thành phần dinh dưỡng, nhật ký bữa ăn...

2.4.2 Cơ sở dữ liệu

Hệ thống được thiết kế tối ưu với 11 Collection chính phục vụ cho các nghiệp vụ quản lý dinh dưỡng và tích hợp Trí tuệ nhân tạo (AI):

2.4.2.1 User

Các trường chính:

- **_id:** Định danh duy nhất (ObjectId).
- **username:** Tên đăng nhập hệ thống, yêu cầu duy nhất.
- **email:** Địa chỉ email duy nhất, dùng để đăng nhập và xác thực.
- **displayName:** Họ tên đầy đủ, dùng hiển thị và nhận diện.
- **hashedPassword:** Mật khẩu đã mã hóa, không bao giờ trả về phía client.
- **role:** Phân quyền, giá trị thường là 'user' hoặc 'admin'.
- **height, weight:** Chỉ số chiều cao (cm) và cân nặng (kg).
- **bmi:** Chỉ số khối cơ thể (được tính toán tự động qua Middleware).
- **healthGoal:** Mục tiêu sức khỏe (Duy trì cân nặng, Giảm cân an toàn, Tăng cân khoa học, Tăng cơ giảm mỡ).
- **budgetPreference:** Mức chi tiêu ăn uống hợp lý.
- **interests:** Danh sách sở thích hoặc chế độ ăn đặc biệt.
- **isActive:** Trạng thái tài khoản (true/false).
- **createdAt, updatedAt:** Thời gian tạo và cập nhật, tự động sinh.

Liên kết: User là trung tâm, liên kết với hầu hết các collection khác như MealLog, FavoriteMeal, Session, MealPlan, ChatHistory...

Chức năng & vai trò:

- Là trung tâm quản lý thông tin người dùng, bao gồm cả người dùng thường và quản trị viên.
- Đảm nhận xác thực (login, đăng ký), phân quyền và lưu trữ thông tin cá nhân.
- Quản lý "Hồ sơ sức khỏe số" (BMI, Mục tiêu, Ngân sách), đóng vai trò cung cấp bộ tham số đầu vào (Prompt) để thuật toán AI phân tích và kiến tạo lộ trình thực đơn.
- Hỗ trợ cá nhân hóa giao diện và theo dõi tiến độ sức khỏe.

Chức năng chính:

- Đăng ký, đăng nhập, xác thực.
- Quản lý hồ sơ cá nhân, cập nhật chỉ số cơ thể và mục tiêu.
- Phân quyền truy cập các chức năng hệ thống.
- Tự động tính toán BMI dựa trên chiều cao và cân nặng trước khi lưu dữ liệu.

2.4.2.2 Meal

Các trường chính:

- **_id:** Định danh món ăn.
- **name:** Tên món ăn.
- **slug:** Đường dẫn tĩnh (tự động tạo từ tên món ăn).
- **description:** Mô tả chi tiết món ăn.
- **categoryIds:** Danh sách tham chiếu Category phân loại.
- **ingredients:** Danh sách nguyên liệu cấu thành kèm định lượng (quantity) và đơn vị (unit).
- **instructions:** Mảng các bước hướng dẫn chế biến chi tiết.
- **imageUrl:** Ảnh đại diện món ăn.
- **prepTime, cookTime:** Thời gian chuẩn bị và thời gian nấu.
- **servings:** Số khẩu phần ăn tiêu chuẩn.
- **totalNutrition:** Tổng dinh dưỡng (calories, protein, carbs, fat) tự động tính toán.

- **totalEstimatedCost:** Tổng chi phí ước tính tự động.
- **isActive:** Trạng thái hiển thị (true/false).
- **createdAt, updatedAt:** Thời gian tạo/cập nhật.

Liên kết: Gắn với Category, Ingredient, MealLog, FavoriteMeal, MealPlan.

Chức năng & vai trò:

- Quản lý toàn bộ kho dữ liệu nền tảng chuyên y khoa: tạo mới, cập nhật, ẩn món ăn.
- Lưu trữ thông tin công thức, hình ảnh, chỉ số Macro và chi phí.
- Cung cấp "nguyên liệu thô" chuẩn xác để kỹ thuật RAG của AI truy xuất, tính toán và đề xuất thực đơn.
- Hỗ trợ phân loại theo danh mục, lọc tìm kiếm.

Chức năng chính:

- Tạo, chỉnh sửa, ẩn món ăn (quản trị viên).
- Tự động tính toán tổng dinh dưỡng và chi phí dựa trên nguyên liệu đầu vào.
- Hiển thị danh sách, chi tiết món ăn cho người dùng.

2.4.2.3 MealLog

Các trường chính:

- **_id:** Định danh bản ghi nhật ký.
- **userId:** Tham chiếu User ghi nhận.
- **mealId:** Tham chiếu Meal đã tiêu thụ (có thể rỗng nếu nhập món ngoài hệ thống).
- **foodName:** Tên món ăn.
- **mealType:** Phân loại bữa ăn (Bữa sáng, trưa, tối, phụ).
- **consumedAt:** Thời gian ăn.
- **servingsConsumed:** Số lượng khẩu phần thực tế đã ăn.
- **nutritionSnapshot:** Lưu cứng dữ liệu dinh dưỡng (calories, protein, carbs, fat) tại thời điểm ăn.
- **notes:** Ghi chú thêm.
- **createdAt, updatedAt:** Thời gian tạo/cập nhật.

Liên kết: Gắn với User và Meal.

Chức năng & vai trò:

- Ghi nhận chi tiết lịch sử tiêu thụ năng lượng hàng ngày của hội viên.
- Cung cấp cơ chế "Snapshot" (chụp nhanh dữ liệu) để đảm bảo lịch sử dinh dưỡng không bị sai lệch ngay cả khi quản trị viên cập nhật lại công thức món ăn gốc trong tương lai.
- Cung cấp dữ liệu thời gian thực để vẽ biểu đồ và đối chiếu với mục tiêu TDEE.

Chức năng chính:

- Tạo mới bản ghi khi người dùng thực hiện ăn uống (thủ công hoặc 1 chạm từ AI).
- Tự động bóc tách và nhân tỷ lệ dinh dưỡng dựa trên khẩu phần ăn thực tế.
- Thống kê tổng năng lượng và Macro trong ngày theo từng người dùng.

2.4.2.4 Ingredient**Các trường chính:**

- **_id:** Định danh nguyên liệu.
- **name:** Tên nguyên liệu.
- **slug:** Đường dẫn tĩnh tự động tạo.
- **baseUnit:** Đơn vị đo lường chuẩn (VD: 100g, 1 quả).
- **referencePrice:** Giá tham khảo tính trên 1 đơn vị chuẩn.
- **nutritionalValue:** Nhóm giá trị dinh dưỡng (calo, protein, carbs, fat) trên 1 đơn vị.
- **description:** Mô tả chi tiết.
- **isActive:** Trạng thái hiển thị.
- **createdAt, updatedAt:** Thời gian tạo/cập nhật.

Liên kết: Được tham chiếu trong mảng ingredients của bảng Meal.

Chức năng & vai trò:

- Cung cấp dữ liệu hạt nhân cơ sở để hệ thống tính toán dinh dưỡng cho các món ăn phức tạp.
- Phục vụ tính năng tính toán chi phí thực đơn.
- Hỗ trợ đặc lực cho Chatbot AI trong việc đưa ra các gợi ý thay thế nguyên liệu linh hoạt khi người dùng bị dị ứng.

Chức năng chính:

- Thêm, sửa, ẩn nguyên liệu (quản trị viên).
- Cung cấp tham số để tự động nội suy dinh dưỡng và chi phí khi ghép vào Meal.

2.4.2.5 Category

Các trường chính:

- **_id:** Định danh danh mục.
- **name:** Tên danh mục.
- **slug:** Đường dẫn tĩnh.
- **type:** Phân loại danh mục (Bữa ăn theo buổi, Chế độ sức khỏe, Loại món ăn, Ngân sách).
- **description:** Mô tả danh mục.
- **isActive:** Trạng thái hiển thị.
- **createdAt, updatedAt:** Thời gian tạo/cập nhật.
- **Liên kết:** Gắn với Meal để phân loại.

Chức năng & vai trò:

- Quản lý các danh mục phân loại thực phẩm.
- Hỗ trợ lọc, tìm kiếm, tổ chức nội dung hệ thống trên giao diện người dùng.
- Giúp thuật toán tìm kiếm và AI phân tích đúng ngữ cảnh để trích xuất món ăn chính xác.

Chức năng chính:

- Tạo, chỉnh sửa, xóa danh mục (quản trị viên).
- Gán danh mục cho món ăn.
- Lọc, tìm kiếm nội dung thực đơn theo danh mục.

2.4.2.6 FavoriteMeal

Các trường chính:

- **_id:** Định danh bản ghi yêu thích.
- **userId:** Tham chiếu User.
- **mealId:** Tham chiếu Meal được yêu thích.
- **createdAt, updatedAt:** Thời gian lưu.
- **Liên kết:** Gắn với User và Meal.

Chức năng & vai trò:

- Lưu trữ danh sách món ăn yêu thích cá nhân của người dùng.
- Hỗ trợ hệ thống thuật toán gợi ý (Recommendation) ưu tiên đưa các món ăn này vào lộ trình thực đơn tương lai.

Chức năng chính:

- Ghi nhận hành động thêm/xóa khỏi danh sách yêu thích của người dùng.
- Sử dụng Index phức hợp để chặn triệt để lỗi logic một người dùng thích cùng một món ăn hai lần.

2.4.2.7 Session (Phiên hoạt động)**Các trường chính:**

- **_id:** Định danh duy nhất của phiên (ObjectId).
- **userId:** Tham chiếu đến User sở hữu phiên đăng nhập.
- **refreshToken:** Chuỗi mã thông báo dùng để cấp lại Access Token mới, giúp duy trì phiên làm việc.
- **expireAt:** Thời gian hết hạn của phiên đăng nhập (Refresh Token).
- **createdAt, updatedAt:** Thời gian tạo và cập nhật tự động.
- **Liên kết:** Gắn trực tiếp với User.

Chức năng & vai trò:

- Quản lý chặt chẽ trạng thái đăng nhập của người dùng trên hệ thống, đảm bảo tính bảo mật.
- Đóng vai trò kiểm soát việc cấp quyền truy cập liên tục vào hệ thống mà không cần người dùng phải đăng nhập lại nhiều lần.
- Hỗ trợ cơ chế tự động dọn dẹp dữ liệu (TTL - Time-To-Live Index), hệ thống cơ sở dữ liệu sẽ tự động xóa phiên làm việc khi hết hạn để giải phóng bộ nhớ.

Chức năng chính:

- Ghi nhận và lưu trữ Refresh Token khi người dùng đăng nhập thành công.
- Cung cấp cơ sở dữ liệu để xác thực và cấp phát lại Access Token mới khi token cũ hết hạn.
- Tự động xóa Document phiên làm việc trong cơ sở dữ liệu khi thời gian đạt mốc expireAt.

2.4.2.8 MealPlan (Lộ trình thực đơn AI)

Các trường chính:

- **_id:** Định danh lộ trình thực đơn.
- **userId:** Tham chiếu User yêu cầu tạo lộ trình.
- **startDate, endDate:** Chu kỳ thời gian áp dụng của thực đơn (thường là lộ trình 7 ngày).
- **dailyMenus:** Mảng dữ liệu cấu trúc lồng nhau (Nested Object) chứa thực đơn chi tiết theo từng ngày.
- **totalDailyCalories:** Tổng lượng năng lượng trung bình ước tính mỗi ngày của lộ trình.
- **isGeneratedByFallback:** Cờ (Boolean) đánh dấu thực đơn này do LLM (Gemini AI) tạo ra (false) hay do Cơ chế dự phòng lỗi 503 tạo ra (true).
- **createdAt, updatedAt:** Thời gian tạo/cập nhật.
- **Liên kết:** Gắn kết chặt chẽ với User và Meal.

Chức năng & vai trò:

- Lưu trữ toàn bộ kết quả phân tích, xử lý và kiến tạo từ Trợ lý ảo AI.
- Đóng vai trò là bản kế hoạch dinh dưỡng chi tiết để người dùng tuân thủ và theo dõi mỗi ngày.
- Lưu lại cờ trạng thái để quản trị viên đo lường được hiệu suất phản hồi của API AI.

Chức năng chính:

- Tạo mới lộ trình khi có yêu cầu phân tích từ người dùng.
- Hiển thị lịch trình ăn uống trực quan trên giao diện.
- Hỗ trợ cung cấp dữ liệu cho tính năng "Ghi nhận 1 chạm", cho phép đẩy nhanh các món từ MealPlan thẳng vào MealLog.

2.4.2.9 ChatHistory (Lịch sử Chatbot AI)**Các trường chính:**

- **_id:** Định danh phiên trò chuyện.
- **userId:** Tham chiếu User đang tương tác.
- **title:** Tiêu đề đoạn hội thoại (tự động tạo dựa trên nội dung câu hỏi đầu tiên của người dùng).
- **messages:** Mảng chứa chi tiết các tin nhắn (gồm người gửi, nội dung tin nhắn, thời gian gửi).

- **isActive:** Trạng thái hiển thị đoạn chat (true/false).
- **createdAt, updatedAt:** Thời gian tạo/cập nhật.

Liên kết: Gắn với User.

Chức năng & vai trò:

- Lưu trữ ngữ cảnh (Context) của các cuộc trò chuyện giữa người dùng và chuyên gia dinh dưỡng ảo.
- Cung cấp bộ nhớ ngắn hạn cho mô hình LLM, giúp AI hiểu được luồng câu chuyện và đưa ra các lời khuyên, gợi ý thay thế nguyên liệu nhất quán trong cùng một phiên làm việc.

Chức năng chính:

- Ghi nhận, lưu trữ và hiển thị lịch sử tin nhắn thời gian thực.
- Tạo phiên chat mới hoặc tải lại lịch sử tin nhắn của phiên chat cũ.
- Ẩn/Xóa đoạn hội thoại khi người dùng không còn nhu cầu.

2.4.2.10 Notification (Thông báo)

Các trường chính:

- **_id:** Định danh thông báo.
- **userId:** Tham chiếu User nhận thông báo.
- **type:** Phân loại thông báo (nhắc nhở ghi nhật ký, thông báo hệ thống, thông báo AI đã tạo xong thực đơn).
- **title, content:** Tiêu đề và nội dung chi tiết.
- **isRead:** Đánh dấu trạng thái đã đọc (true/false).
- **createdAt:** Thời gian gửi.

Liên kết: Gắn với User.

Chức năng & vai trò:

- Gửi các thông báo cá nhân hóa nhằm thúc đẩy, động viên người dùng duy trì kỷ luật ăn uống theo lộ trình.
- Giải quyết vấn đề Trải nghiệm người dùng (UX) khi chờ đợi: Do API AI cần thời gian xử lý, hệ thống sẽ chạy ngầm và gửi thông báo khi thực đơn đã sẵn sàng.

Chức năng chính:

- Tự động tạo và gửi thông báo theo các kịch bản/sự kiện hệ thống.

- Hiển thị danh sách thông báo trên giao diện người dùng.
- Đánh dấu đã đọc hoặc xóa thông báo.

2.4.2.11 Bảng cơ sở dữ liệu chính

Bảng 1. User

STT	Tên thuộc tính	Mô tả	Kiểu dữ liệu
1	_id	ID người dùng	ObjectId
2	username	Tên đăng nhập	string
3	email	Địa chỉ email	string
4	displayName	Họ tên hiển thị	string
5	hashedPassword	Mật khẩu đã mã hóa	string
6	role	Phân quyền (user/admin)	'user' 'admin'
7	height	Chiều cao (cm)	number
8	weight	Cân nặng (kg)	number
9	bmi	Chỉ số khối cơ thể	number
10	healthGoal	Mục tiêu sức khỏe	enum
11	budgetPreference	Ngân sách ăn uống	number
12	interests	Sở thích/Chế độ ăn	[string]
13	isActive	Trạng thái tài khoản	boolean
14	createdAt	Thời gian tạo	Date

Bảng 2. Meal

STT	Tên thuộc tính	Mô tả	Kiểu dữ liệu
1	_id	ID món ăn	ObjectId
2	name	Tên món ăn	string
3	slug	Đường dẫn tĩnh	string
4	description	Mô tả món ăn	string
5	categoryIds	Danh sách danh mục	[ObjectId]
6	ingredients	Danh sách nguyên liệu	[object]
7	instructions	Hướng dẫn chế biến	[string]
8	imageUrl	Ảnh đại diện món ăn	string
9	prepTime	Thời gian chuẩn bị	number
10	cookTime	Thời gian nấu	number

11	servings	Số khẩu phần	number
12	totalNutrition	Tổng giá trị dinh dưỡng	object
13	totalEstimatedCost	Tổng chi phí ước tính	number
14	isActive	Trạng thái hiển thị	boolean

Bảng 3. MealLog

STT	Tên thuộc tính	Mô tả	Kiểu dữ liệu
1	_id	ID nhật ký	ObjectId
2	userId	ID người dùng	ObjectId
3	mealId	ID món ăn (có thể rỗng)	ObjectId
4	foodName	Tên món ăn đã dùng	string
5	mealType	Bữa ăn (sáng/trưa/tối/phụ)	enum
6	consumedAt	Thời gian ăn	Date
7	servingsConsumed	Số phần thực tế đã ăn	number
8	nutritionSnapshot	Dinh dưỡng tại thời điểm ăn	object
9	notes	Ghi chú	string

Bảng 4. Ingredient

STT	Tên thuộc tính	Mô tả	Kiểu dữ liệu
1	_id	ID nguyên liệu	ObjectId
2	name	Tên nguyên liệu	string
3	slug	Đường dẫn tĩnh	string
4	baseUnit	Đơn vị đo lường chuẩn	string
5	referencePrice	Giá tham khảo (trên 1 đơn vị)	number
6	nutritionalValue	Hàm lượng Macro cơ sở	object
7	description	Mô tả nguyên liệu	string
8	isActive	Trạng thái hiển thị	boolean

Bảng 5. Category

STT	Tên thuộc tính	Mô tả	Kiểu dữ liệu
1	_id	ID danh mục	ObjectId
2	name	Tên danh mục	string
3	slug	Đường dẫn tĩnh	string

4	type	Phân loại danh mục	enum
5	description	Mô tả chi tiết	string
6	isActive	Trạng thái hiển thị	boolean

Bảng 6. FavoriteMeal

STT	Tên thuộc tính	Mô tả	Kiểu dữ liệu
1	_id	ID bản ghi yêu thích	ObjectId
2	userId	ID người dùng	ObjectId
3	mealId	ID món ăn	ObjectId
4	createdAt	Thời gian lưu	Date

Bảng 7. Session

STT	Tên thuộc tính	Mô tả	Kiểu dữ liệu
1	_id	ID phiên	ObjectId
2	userId	ID người dùng	ObjectId
3	refreshToken	Chuỗi xác thực gia hạn	string
4	expiresAt	Thời gian hết hạn phiên	Date
5	createdAt	Thời gian tạo	Date

Bảng 8. MealPlan

STT	Tên thuộc tính	Mô tả	Kiểu dữ liệu
1	_id	ID lộ trình thực đơn	ObjectId
2	userId	ID người dùng	ObjectId
3	startDate	Ngày bắt đầu lộ trình	Date
4	endDate	Ngày kết thúc lộ trình	Date
5	dailyMenus	Chi tiết thực đơn các ngày	[object]
6	totalDailyCalories	Trung bình Calo/ngày	number
7	isGeneratedByFallback	Tạo bởi cơ chế dự phòng	boolean

Bảng 9. ChatHistory

STT	Tên thuộc tính	Mô tả	Kiểu dữ liệu
1	_id	ID phiên trò chuyện	ObjectId
2	userId	ID người dùng	ObjectId
3	title	Tiêu đề đoạn hội thoại	string
4	messages	Lịch sử tin nhắn (AI & User)	[object]

5	isActive	Trạng thái hiển thị	boolean
6	createdAt	Thời gian bắt đầu chat	Date

Bảng 10. Notification

STT	Tên thuộc tính	Mô tả	Kiểu dữ liệu
1	_id	ID thông báo	ObjectId
2	userId	ID người nhận	ObjectId
3	type	Loại thông báo	enum
4	title	Tiêu đề	string
5	content	Nội dung thông báo	string
6	isRead	Trạng thái đã đọc	boolean
7	createdAt	Thời gian gửi	Date

2.5 Mô tả kiến trúc hệ thống

Hệ thống Gợi ý Thực đơn Dinh dưỡng Thông minh (NutriFood) được thiết kế và xây dựng theo mô hình Client – Server kết hợp với tư duy kiến trúc hướng dịch vụ (phục vụ cho phân hệ Trí tuệ nhân tạo). Đây là một mô hình kiến trúc phần mềm hiện đại, giúp tách biệt rõ ràng giữa giao diện người dùng (client), máy chủ xử lý logic nghiệp vụ (server) và dịch vụ AI bên thứ ba. Kiến trúc này không chỉ tăng khả năng mở rộng, tái sử dụng mã nguồn mà còn giúp hệ thống vận hành trơn tru, dễ dàng bảo trì nâng cấp trong tương lai.

Hệ thống bao gồm ba thành phần chính: Frontend (Giao diện người dùng), Backend (Máy chủ API & Cầu nối AI) và Cơ sở dữ liệu (Nơi lưu trữ tập trung). Các thành phần này được triển khai bằng bộ công nghệ MERN Stack cải tiến bao gồm ReactJS, NodeJS, ExpressJS, MongoDB kết hợp cùng API của mô hình ngôn ngữ lớn Google Gemini.

2.5.1 Giao diện người dùng (Client – Frontend)

Phần giao diện người dùng được xây dựng bằng ReactJS, một thư viện JavaScript nổi tiếng chuyên dùng để xây dựng các giao diện web một trang (SPA - Single Page Application) giàu tính tương tác. Kết hợp với Vite (công cụ build dự án siêu tốc) và Tailwind CSS (framework thiết kế giao diện), hệ thống đảm bảo trải nghiệm người dùng mượt mà và tương thích tốt trên đa dạng kích thước màn hình (Responsive).

Frontend chịu trách nhiệm hiển thị dữ liệu trực quan và cung cấp các chức năng tương tác cốt lõi:

- Khai báo và hiển thị "hồ sơ sức khỏe số" (chiều cao, cân nặng, BMI, mục tiêu dinh dưỡng).

- Giao diện Dashboard theo dõi nhật ký lượng calo và Macro tiêu thụ hằng ngày.
- Giao diện tương tác trò chuyện trực tiếp với Chatbot AI chuyên gia dinh dưỡng.
- Hiển thị lộ trình thực đơn theo tuần với tính năng "ghi nhận 1 chạm".
- Hiển thị kho dữ liệu món ăn, tính năng tìm kiếm, lọc theo danh mục và đánh dấu yêu thích.

Frontend không lưu trữ dữ liệu vĩnh viễn mà chỉ duy trì trạng thái tạm thời, liên tục tương tác với Backend thông qua các API RESTful, gửi yêu cầu qua giao thức HTTP/HTTPS và nhận dữ liệu ở định dạng JSON để render lên màn hình.

2.5.2 Máy chủ xử lý nghiệp vụ (Server – Backend)

Thành phần Backend đóng vai trò là "bộ não" điều phối toàn bộ luồng dữ liệu của hệ thống NutriFood. Được xây dựng bằng Node.js kết hợp framework Express.js, Backend không chỉ cung cấp API mà còn đóng vai trò là lớp trung gian (Middleware) bảo mật khi giao tiếp với AI.

Các chức năng chính được thực hiện tại Backend bao gồm:

- **Xác thực và phân quyền:** Quản lý phiên làm việc thông qua JWT (JSON Web Token) kết hợp Refresh Token lưu trong Session, đảm bảo an toàn truy cập cho cả Người dùng và Quản trị viên.
- **Xử lý logic nghiệp vụ tính toán:** Thực hiện các phép tính phức tạp (như tính tỷ lệ Snapshot dinh dưỡng khi ghi nhật ký MealLog, nội suy calo từ nguyên liệu Ingredient) trước khi lưu vào cơ sở dữ liệu để giảm tải cho Frontend.
- **Điều phối Trí tuệ nhân tạo (AI Integration & Fallback):** Đóng vai trò là cầu nối giao tiếp với API Google Gemini. Backend sẽ thu thập tham số sức khỏe của User để thiết kế Prompt, đồng thời tích hợp kỹ thuật RAG (truy vấn kho món ăn hiện có) để gửi cho AI. Đặc biệt, Backend đảm nhận việc xử lý cơ chế Dự phòng (Fallback) – tự động nội suy thực đơn thay thế nếu API của Gemini bị lỗi (Timeout/503), đảm bảo trải nghiệm không bị gián đoạn.
- **Quản trị dữ liệu:** Cung cấp các API kiểm duyệt nghiêm ngặt cho Admin để quản lý danh mục, nguyên liệu và thông tin chuẩn y khoa của món ăn.

2.5.3 Cơ sở dữ liệu (Database)

Hệ thống sử dụng MongoDB làm hệ quản trị cơ sở dữ liệu chính. Đây là một CSDL NoSQL lưu trữ dữ liệu dưới dạng document theo cấu trúc BSON (tương tự JSON), cực kỳ phù hợp để lưu trữ các mảng dữ liệu lồng nhau phức tạp như thành phần dinh dưỡng nguyên liệu (Macros) hay cấu trúc lộ trình thực đơn 7 ngày.

Các dữ liệu cốt lõi trong hệ thống được chia thành các Collection liên kết chặt chẽ với nhau:

- **User & Session:** Quản lý thông tin cá nhân, định mức sức khỏe, mục tiêu và trạng thái bảo mật đăng nhập.
- **Meal, Ingredient, Category:** Kho dữ liệu nền tảng về ẩm thực, lưu trữ công thức, định lượng và phân loại.
- **MealPlan & MealLog:** Lưu trữ kết quả lộ trình AI trả về và nhật ký tiêu thụ calo thực tế của hội viên.
- **ChatHistory:** Lưu trữ ngữ cảnh (Context) hội thoại để mô hình LLM ghi nhớ thông tin người dùng.

Kết nối giữa Backend và MongoDB được thực thi qua thư viện Mongoose. Điểm sáng của kiến trúc này là việc tận dụng sức mạnh của các Mongoose Middleware (pre-save) để tự động hóa các phép tính (như tính BMI, tính tổng chi phí món ăn) ngay tại tầng Database.

2.5.4 Luồng xử lý tổng quát

Quy trình hoạt động giữa các thành phần được mô tả chặt chẽ như sau:

- Người dùng thao tác trên giao diện trình duyệt (Client).
- Frontend ReactJS gửi yêu cầu HTTP Request (mang theo Token xác thực) đến máy chủ Backend NodeJS.
- Backend tiếp nhận, giải mã Token để kiểm tra quyền truy cập hợp lệ.

Tùy thuộc vào loại yêu cầu (Request):

- Nghiệp vụ thông thường (Thêm/Sửa/Xóa, tra cứu): Backend trực tiếp truy vấn MongoDB thông qua Mongoose và xử lý logic.
- Nghiệp vụ AI (Xin lộ trình/Chatbot): Backend trích xuất dữ liệu của User từ MongoDB, tổng hợp Prompt và gọi sang API của Google Gemini. Nếu Gemini phản hồi thành công, Backend lưu kết quả vào MongoDB. Nếu

Gemini quá tải/lỗi, Backend tự động kích hoạt thuật toán Fallback để sinh dữ liệu thay thế.

- Backend định dạng lại dữ liệu và trả về kết quả (JSON Response) cho Frontend hiển thị.

2.5.5 Ưu điểm kiến trúc

- Tách biệt logic (Separation of Concerns): Phân chia rõ ràng vai trò của UI (Frontend), logic nghiệp vụ/bảo mật (Backend) và truy xuất dữ liệu (Database), giúp mã nguồn dễ đọc, dễ bảo trì và làm việc nhóm hiệu quả.
- Độ ổn định và Khả năng chịu lỗi (Fault Tolerance): Nhờ cơ chế Fallback được thiết kế tại tầng Backend, hệ thống NutriFood vẫn có thể hoạt động độc lập để cung cấp thực đơn ngay cả khi dịch vụ AI bên thứ ba gặp sự cố.
- Tối ưu hiệu năng: Việc sử dụng Mongoose Middleware để xử lý tính toán ngầm giúp giảm tải dung lượng tính toán cho server Node.js.
- Sẵn sàng mở rộng: Giao tiếp hoàn toàn thông qua chuẩn RESTful API, dễ dàng tích hợp thêm ứng dụng di động (Mobile App) hoặc các mô hình AI khác trong tương lai.

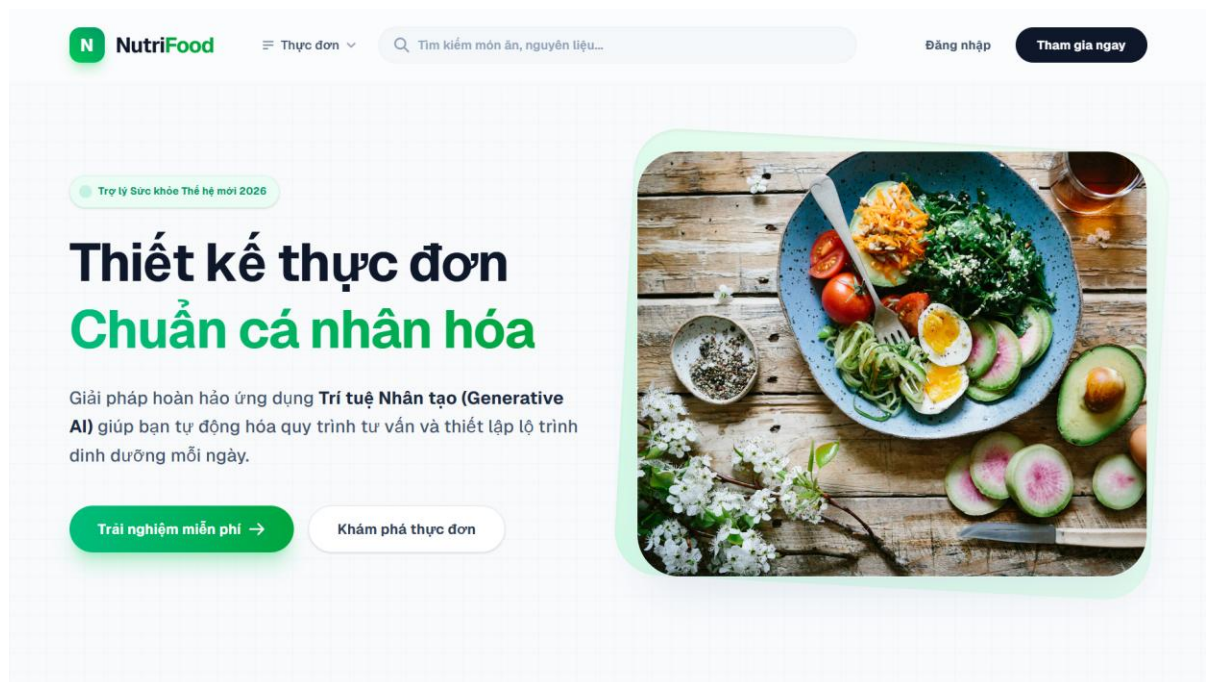
CHƯƠNG 3: KẾT QUẢ NGHIÊN CỨU

3.1 Giao diện người dùng

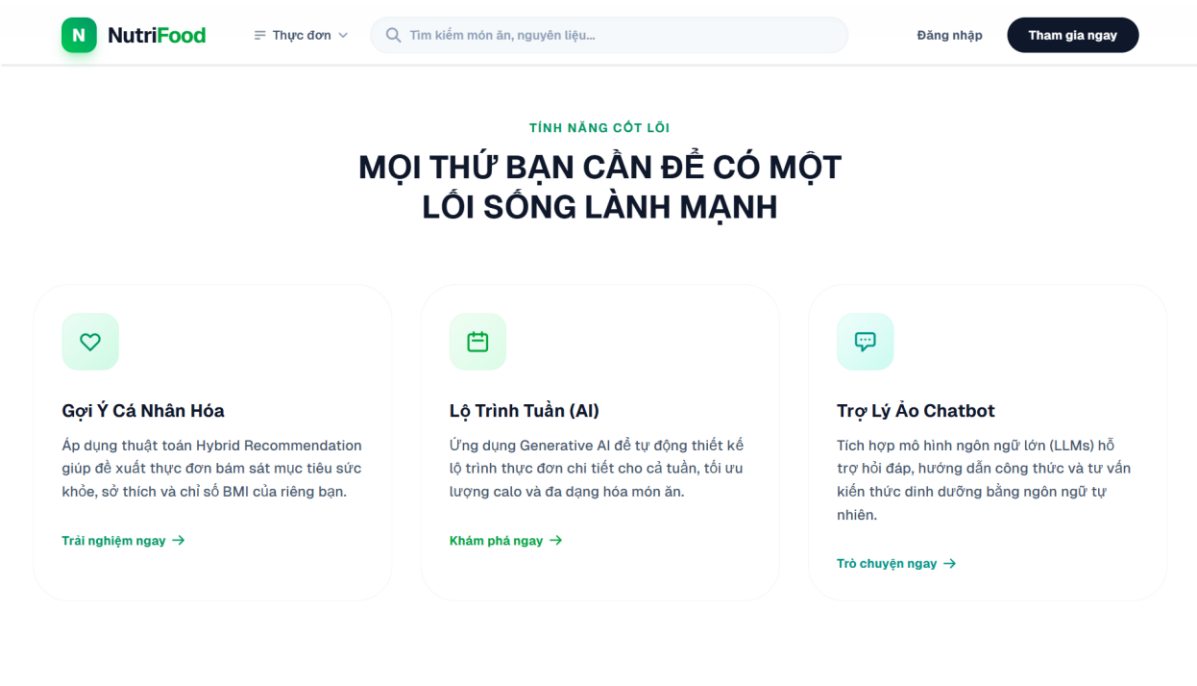
3.1.1 Giao diện trang chủ

Giao diện trang chủ NutriFood được thiết kế với tông màu xanh chủ đạo cùng bố cục hiện đại, tạo cảm giác thân thiện và đáng tin cậy đối với người dùng. Khu vực trung tâm nổi bật với khẩu hiệu “Thiết kế thực đơn chuẩn cá nhân hóa”, các nút hành động như “Trải nghiệm miễn phí” và “Khám phá thực đơn”, kết hợp hình ảnh minh họa bữa ăn lành mạnh giúp người dùng nhanh chóng nhận biết mục tiêu của nền tảng và khuyến khích trải nghiệm dịch vụ.

Bên cạnh đó, trang chủ còn giới thiệu các tính năng cốt lõi của hệ thống như gợi ý thực đơn cá nhân hóa, lộ trình dinh dưỡng bằng AI và trợ lý AI Chatbot, qua đó làm nổi bật khả năng ứng dụng trí tuệ nhân tạo trong việc hỗ trợ xây dựng chế độ ăn uống khoa học và phù hợp với từng cá nhân.



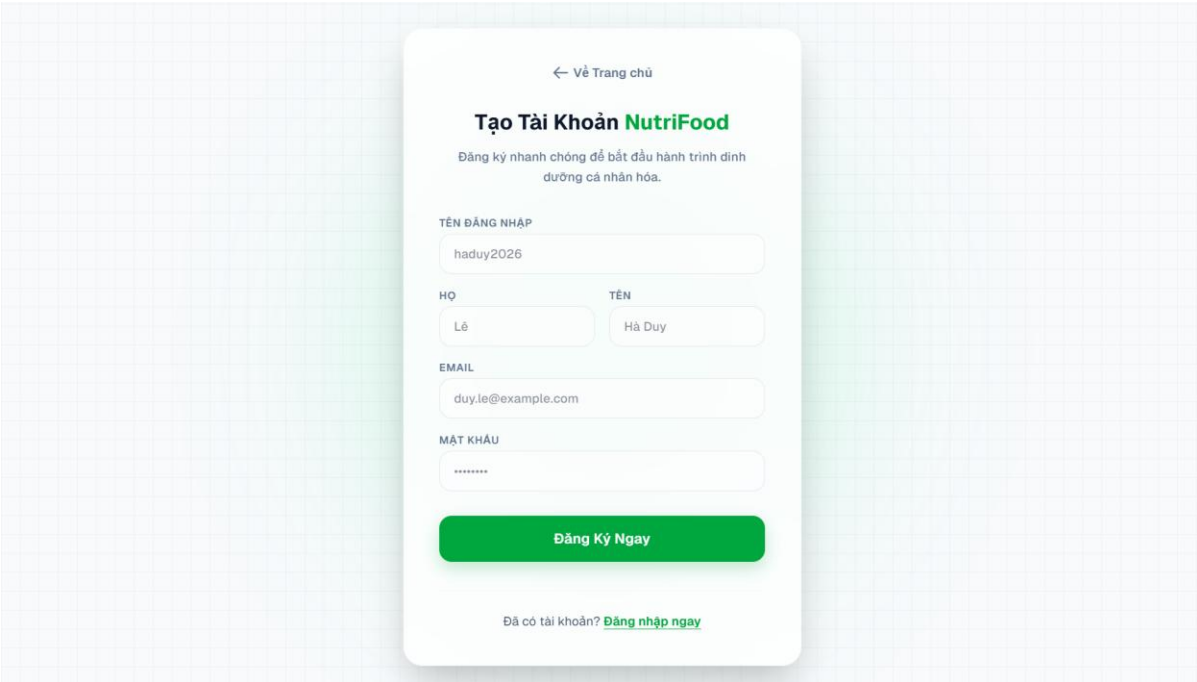
Hình 3.1. Giao diện trang chủ



Hình 3.2. Chức năng cốt lõi ở trang chủ

3.1.2 Giao diện đăng ký

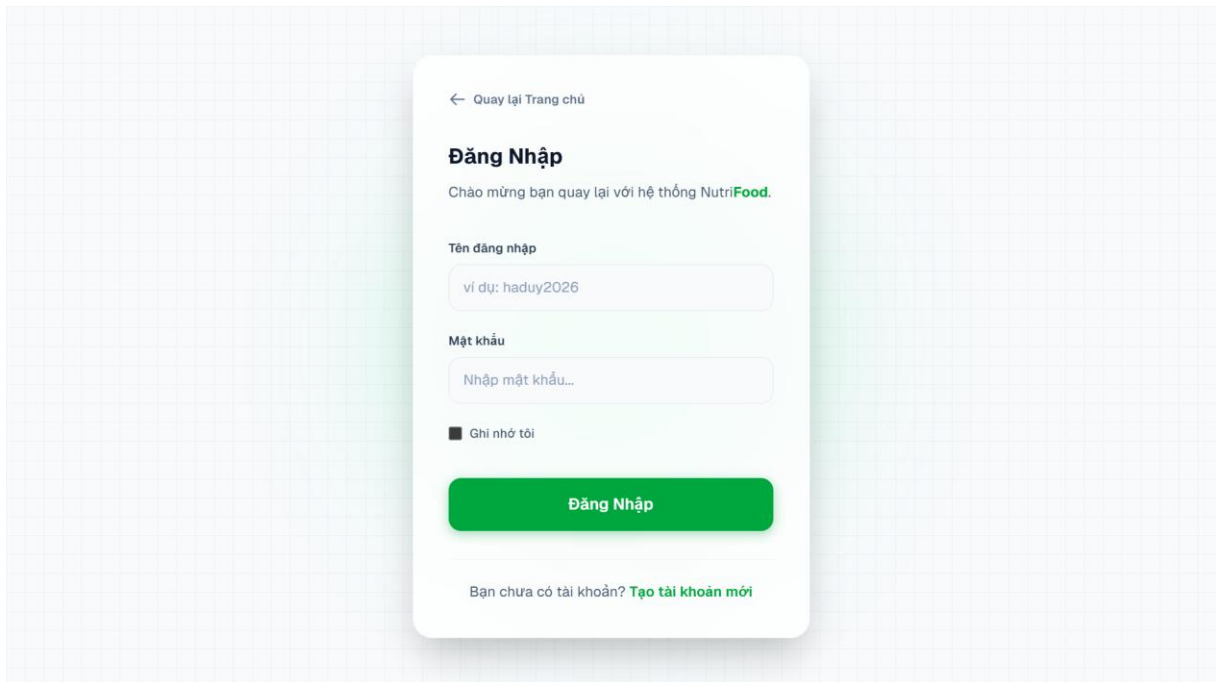
Giao diện cho phép người dùng tạo tài khoản mới để sử dụng các chức năng của hệ thống NutriFood. Người dùng cần cung cấp các thông tin cơ bản như tên đăng nhập, họ tên, email và mật khẩu để hoàn tất quá trình đăng ký. Chức năng này giúp hệ thống quản lý thông tin cá nhân và tạo nền tảng cho việc xây dựng các lộ trình dinh dưỡng và gợi ý thực đơn phù hợp với từng người dùng.



Hình 3.3. Giao diện đăng ký

3.1.3 Giao diện đăng nhập

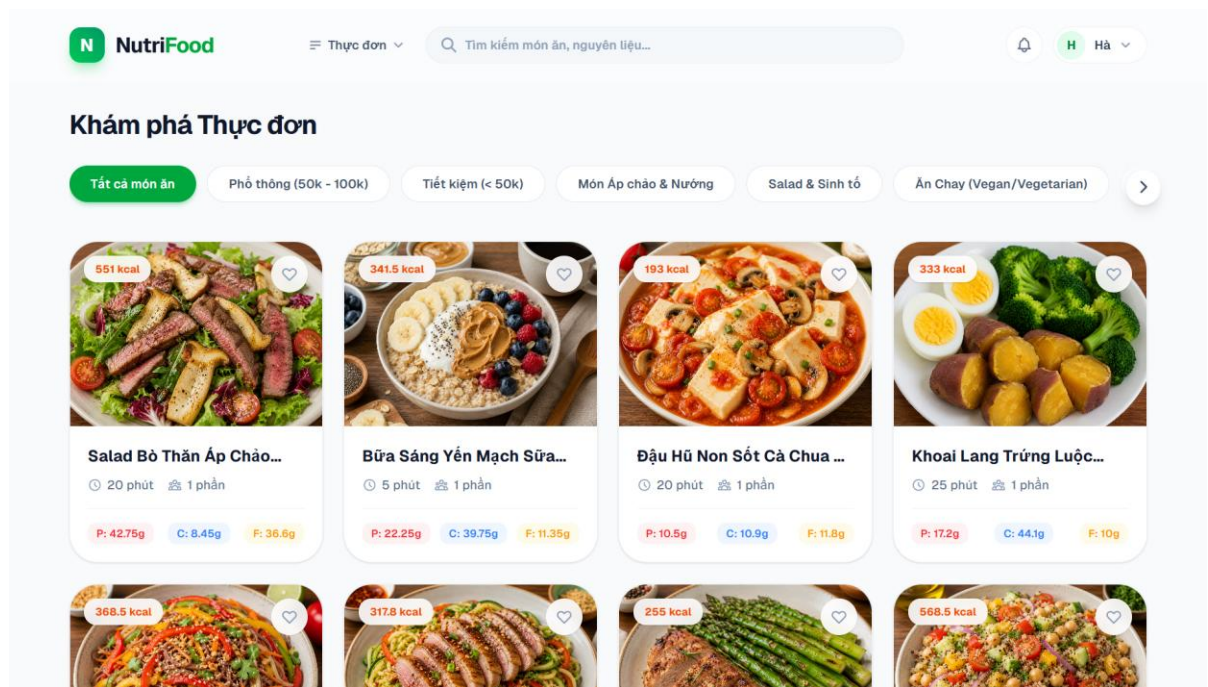
Giao diện cho phép người dùng truy cập vào tài khoản cá nhân thông qua việc xác thực bằng tên đăng nhập và mật khẩu. Sau khi đăng nhập thành công, người dùng có thể sử dụng các chức năng của hệ thống NutriFood như quản lý hồ sơ cá nhân, theo dõi nhật ký dinh dưỡng, nhận gợi ý thực đơn và tương tác với trợ lý AI, đồng thời đảm bảo tính bảo mật và an toàn trong quá trình sử dụng hệ thống.



Hình 3.4. Giao diện đăng nhập

3.1.4 Giao diện khám phá thực đơn

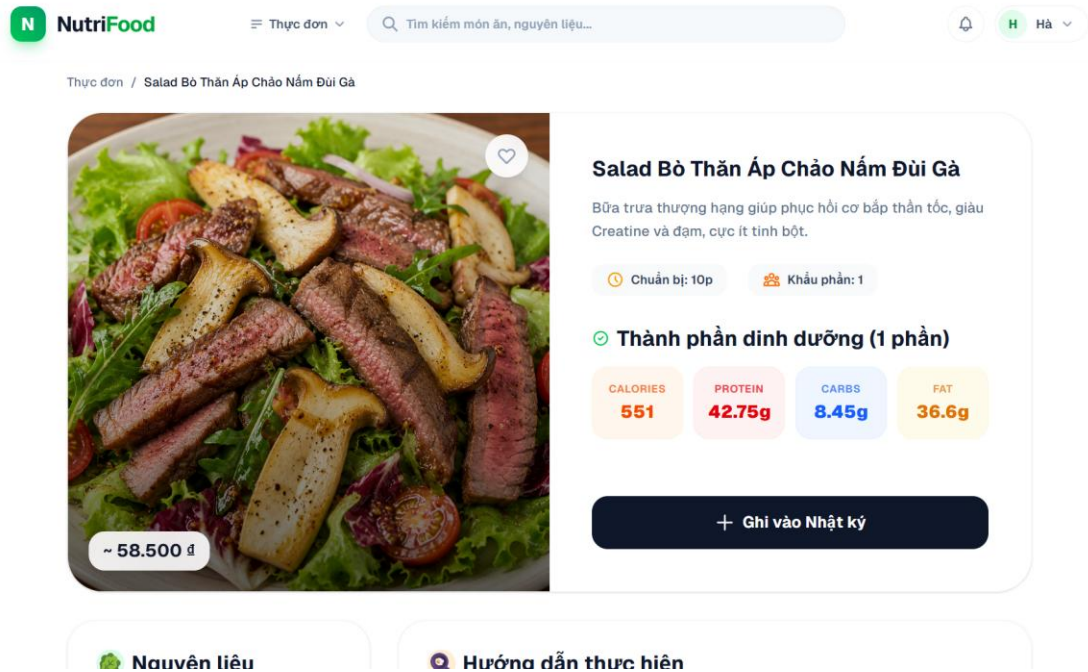
Giao diện cho phép người dùng tra cứu và lựa chọn các món ăn dinh dưỡng phù hợp với nhu cầu cá nhân. Hệ thống hỗ trợ phân loại món ăn theo nhiều danh mục như món phổ thông, tiết kiệm, món áp chảo, salad và món chay nhằm giúp quá trình tìm kiếm trở nên thuận tiện hơn. Mỗi món ăn đều hiển thị đầy đủ thông tin về lượng calo, thời gian chế biến, số khẩu phần và các chỉ số dinh dưỡng như Protein, Carbohydrate và Chất béo, hỗ trợ người dùng dễ dàng xây dựng chế độ ăn uống khoa học và cân bằng.



Hình 3.5. Giao diện khám phá thực đơn

3.1.5 Giao diện chi tiết món ăn

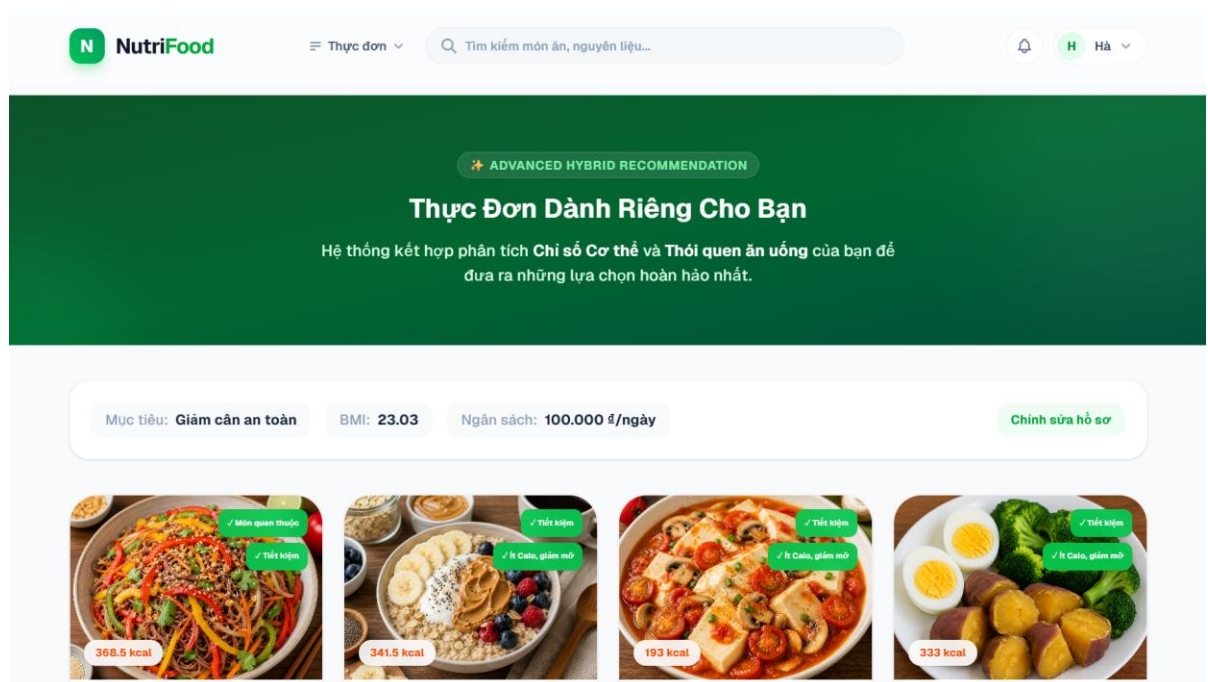
Giao diện cung cấp đầy đủ thông tin về món ăn được lựa chọn, bao gồm hình ảnh minh họa, mô tả, thời gian chuẩn bị, số khẩu phần và các chỉ số dinh dưỡng như lượng calo, protein, carbohydrate và chất béo. Ngoài ra, người dùng có thể xem nguyên liệu, hướng dẫn chế biến và thêm món ăn vào nhật ký dinh dưỡng để hỗ trợ theo dõi chế độ ăn uống hàng ngày.



Hình 3.6. Giao diện chi tiết món ăn

3.1.6 Giao diện gợi ý cá nhân hóa

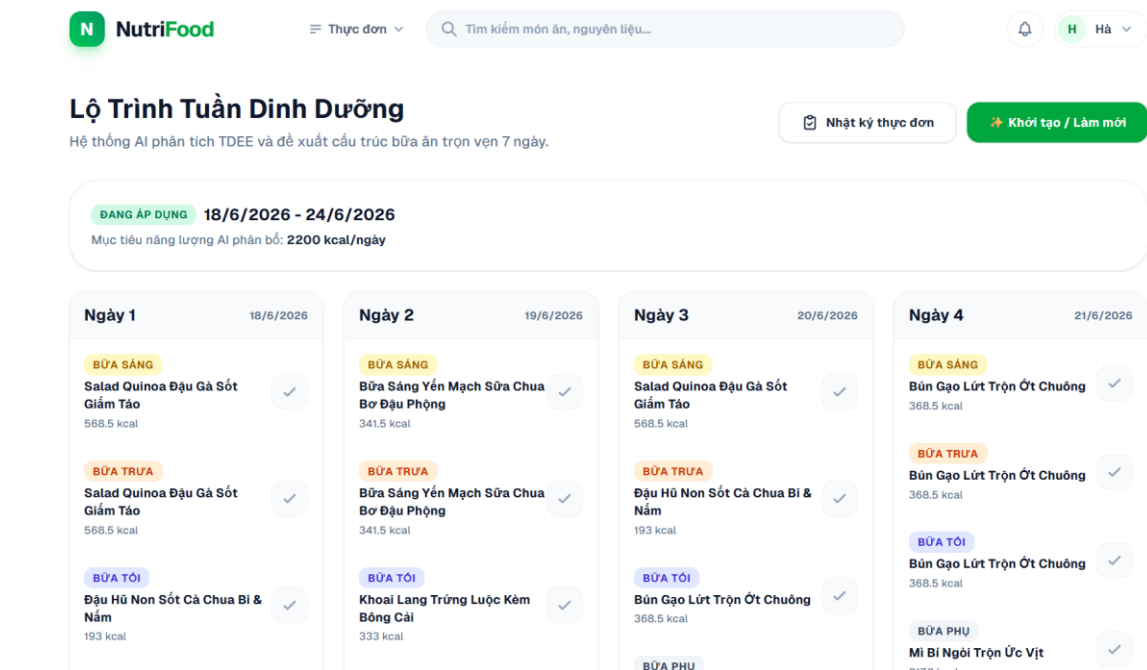
Giao diện cung cấp danh sách thực đơn tối ưu riêng cho người dùng, bao gồm hình ảnh minh họa, lượng calo ước tính, cùng các chỉ số cá nhân như mục tiêu sức khỏe, BMI và ngân sách hàng ngày. Tại đây, người dùng có thể xem nhanh lý do đề xuất của thuật toán AI (như món yêu thích, tiết kiệm, quen thuộc), tùy chỉnh hồ sơ cá nhân hoặc nhấn chọn từng thẻ món ăn để xem chi tiết nguyên liệu và hướng dẫn chế biến.



Hình 3.7. Giao diện gợi ý cá nhân hóa

3.1.7 Giao diện lộ trình tuần bằng AI

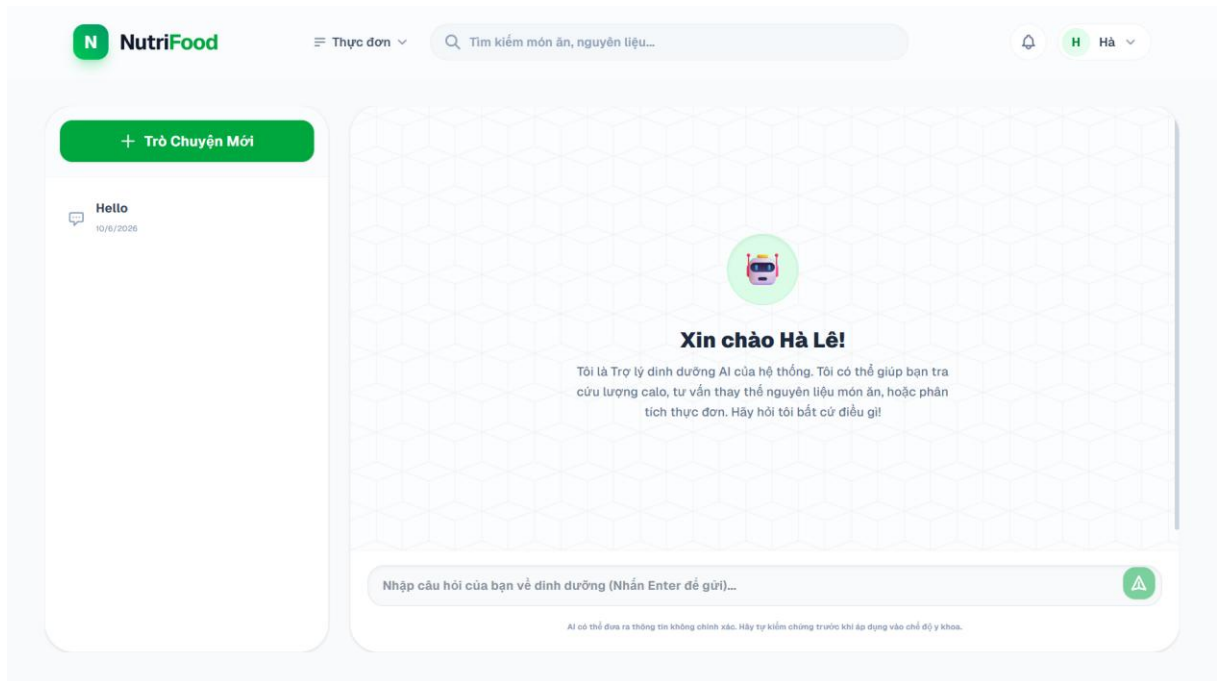
Giao diện cho phép người dùng tạo danh sách thực đơn được tối ưu trọn vẹn trong 7 ngày, bao gồm thông tin về khoảng thời gian áp dụng, mục tiêu năng lượng AI phân bổ (kcal/ngày) và cấu trúc chi tiết các bữa ăn (sáng, trưa, tối, phụ) theo từng ngày. Tại đây, người dùng có thể xem nhanh trạng thái hoàn thành bữa ăn, thực hiện ghi nhận nhanh thực đơn vào nhật ký bằng tính năng một chạm, hoặc nhấn nút khởi tạo để làm mới toàn bộ lộ trình nhằm hỗ trợ tối ưu việc theo dõi chế độ dinh dưỡng hằng tuần.



Hình 3.8. Giao diện lộ trình tuần bằng AI

3.1.8 Giao diện trợ lý tư vấn dinh dưỡng bằng AI

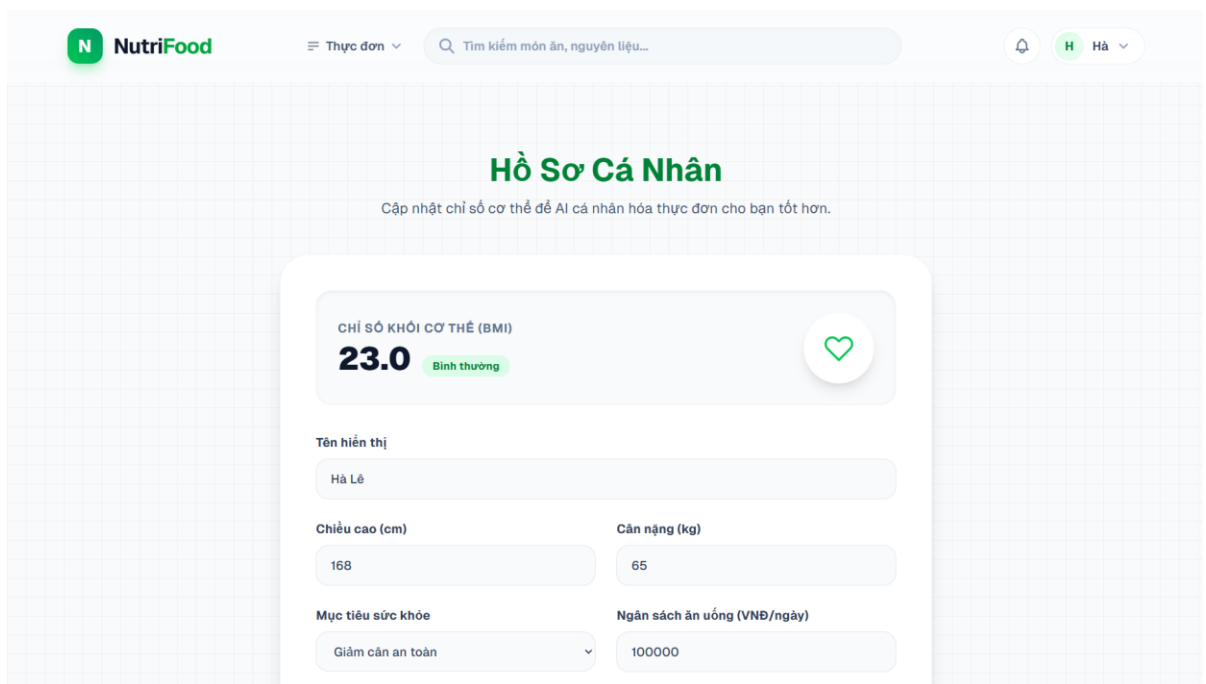
Giao diện cung cấp không gian tương tác trực tuyến giữa người dùng và Trợ lý dinh dưỡng thông minh, bao gồm thanh nhập câu hỏi, khung hiển thị nội dung hội thoại và danh sách các phiên trò chuyện cũ ở thanh bên (sidebar). Tại đây, người dùng có thể xem nhanh các chủ đề đã trao đổi, khởi tạo cuộc hội thoại mới, hoặc gửi các câu hỏi nhờ AI tra cứu lượng calo, tư vấn thay thế nguyên liệu và phân tích thực đơn nhằm hỗ trợ giải đáp nhanh chóng các thắc mắc về chế độ ăn uống hằng ngày.



Hình 3.9. Giao diện trợ lý tư vấn dinh dưỡng bằng AI

3.1.9 Giao diện thông tin cá nhân

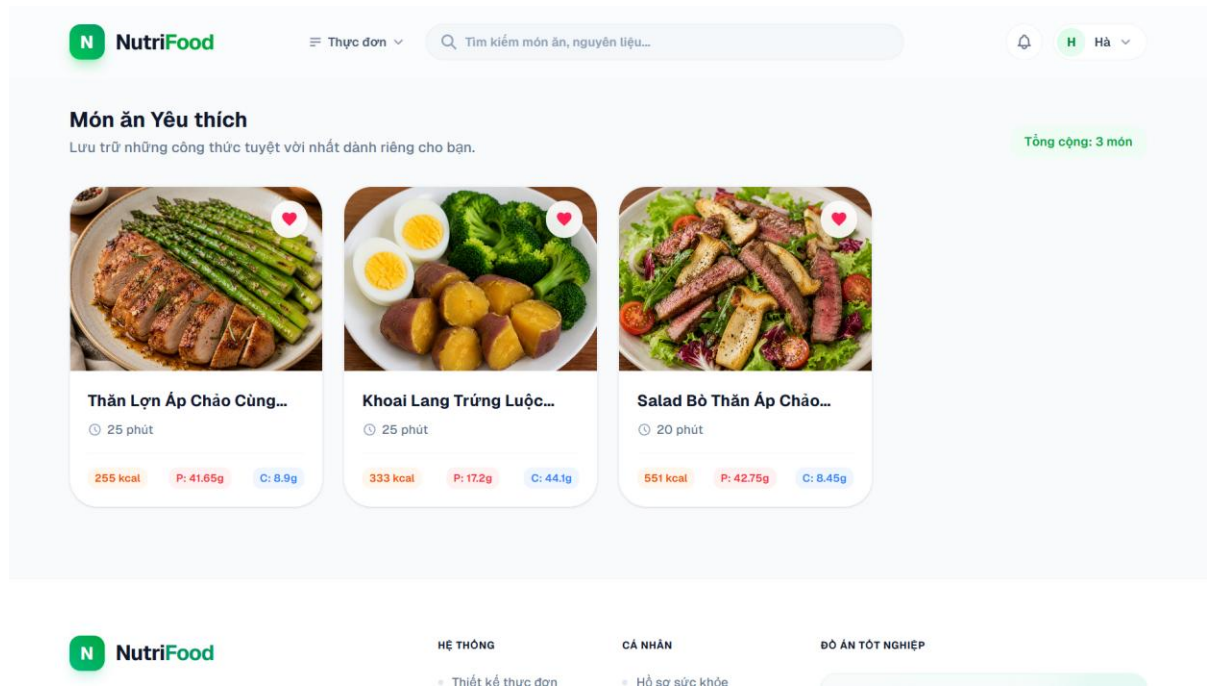
Giao diện đảm nhiệm việc tiếp nhận và quản lý các thông số sinh học cùng mục tiêu cá nhân của người dùng, bao gồm chỉ số BMI, chiều cao, cân nặng, mục tiêu thể trạng và ngân sách ăn uống. Thông qua đó, người dùng có thể theo dõi tình trạng sức khỏe và cung cấp dữ liệu đầu vào cho hệ thống phân tích, gợi ý thực đơn phù hợp.



Hình 3.10. Giao diện thông tin cá nhân

3.1.10 Giao diện món ăn yêu thích

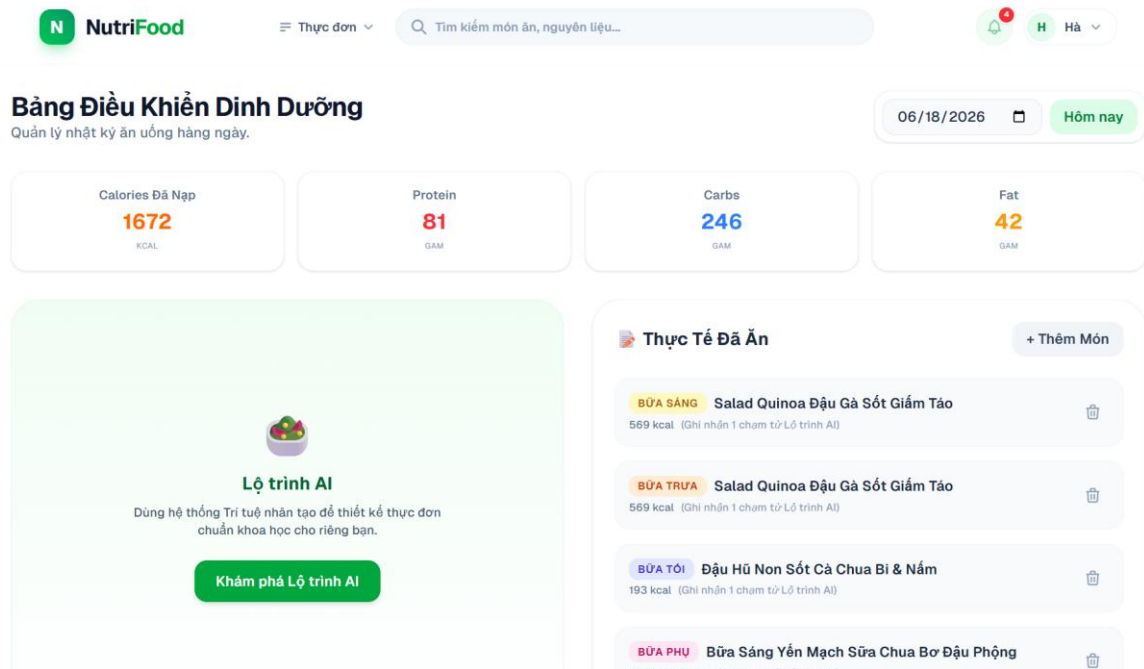
Giao diện này đóng vai trò như một kho lưu trữ cá nhân, tập hợp các món ăn mà người dùng đã đánh dấu yêu thích. Hệ thống hiển thị danh sách món ăn kèm hình ảnh, thời gian chế biến và các thông số dinh dưỡng cơ bản. Người dùng có thể dễ dàng quản lý danh sách yêu thích, xóa món ăn đã lưu hoặc truy cập nhanh vào trang chi tiết để xem công thức chế biến.



Hình 3.11. Giao diện món ăn yêu thích

3.1.11 Giao diện nhật ký thực đơn

Giao diện là nơi theo dõi và trực quan hóa lượng Calories cùng các chỉ số dinh dưỡng chính (Protein, Carbs, Fat) đã tiêu thụ trong ngày. Hệ thống phân chia lịch sử ăn uống theo từng bữa và hỗ trợ chuyển đổi ngày theo dõi. Người dùng có thể thêm hoặc xóa món ăn để cập nhật dữ liệu chính xác, đồng thời kết nối nhanh với các chức năng AI hỗ trợ xây dựng chế độ dinh dưỡng phù hợp.

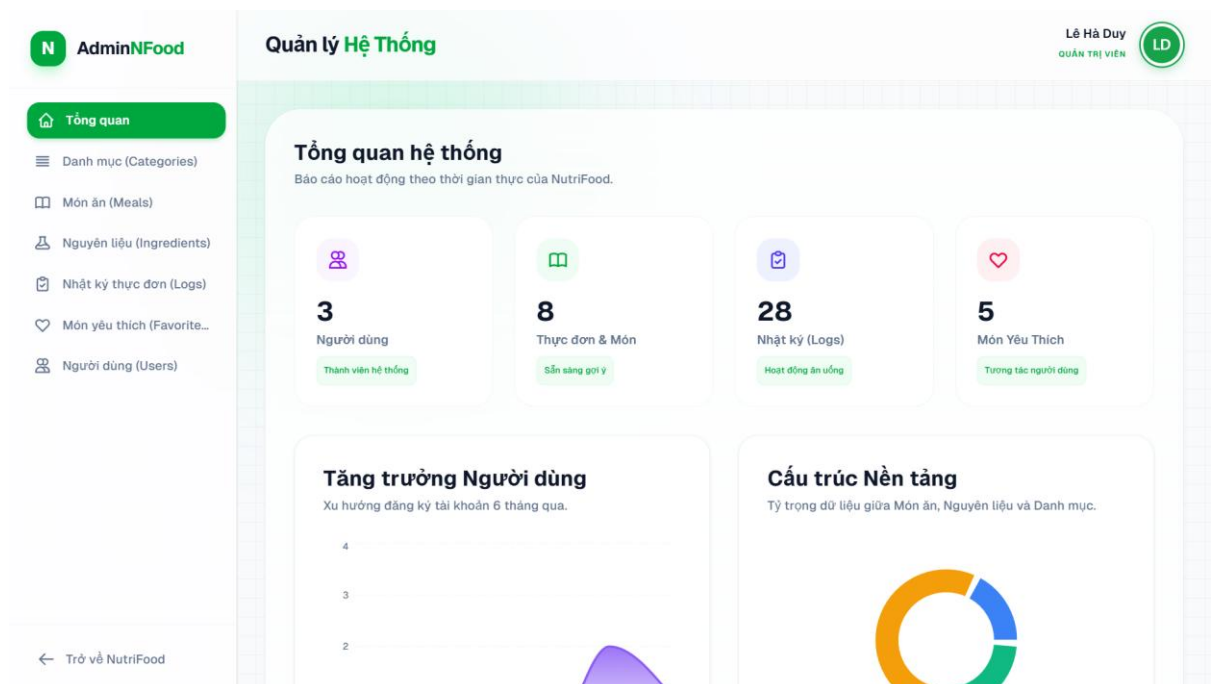


Hình 3.12. Giao diện nhật ký thực đơn

3.2 Giao diện quản lý của Quản trị viên

3.2.1 Giao diện tổng quan hệ thống

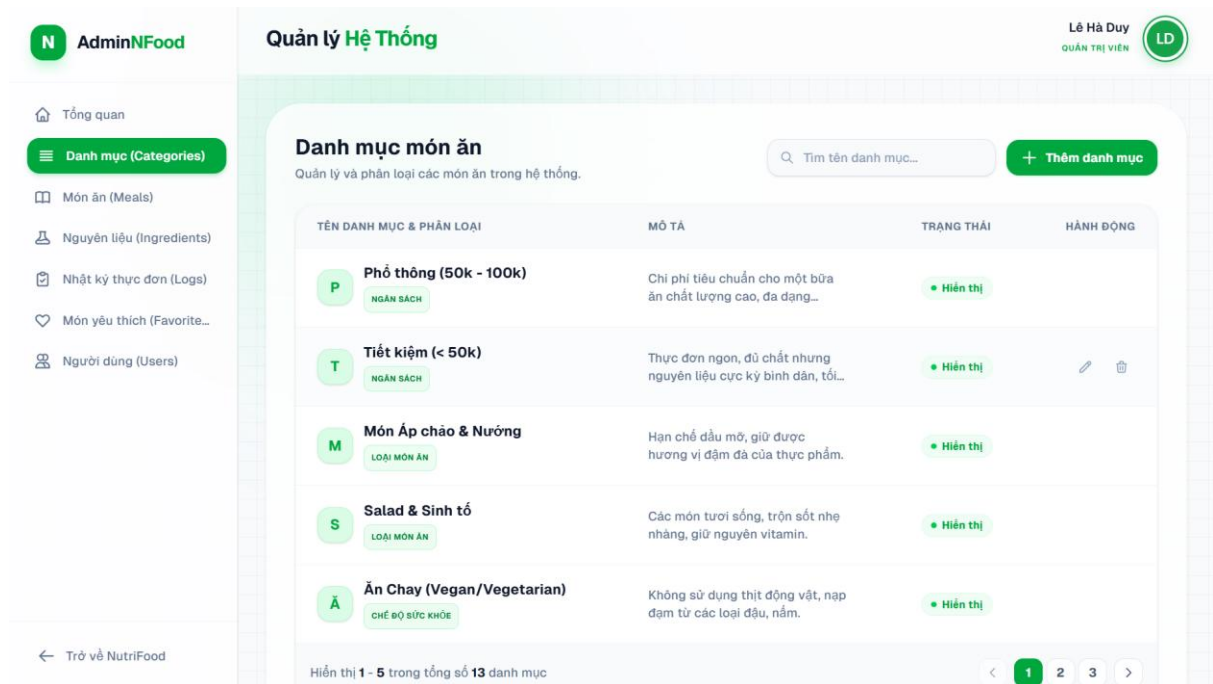
Giao diện đóng vai trò là trung tâm theo dõi các chỉ số hoạt động của hệ thống NutriFood. Phân hệ hiển thị số liệu về người dùng, thực đơn, nhật ký ăn uống và món ăn yêu thích thông qua các thẻ thống kê và biểu đồ trực quan. Nhờ đó, quản trị viên có thể nhanh chóng nắm bắt tình trạng vận hành và dễ dàng truy cập các chức năng quản lý chuyên sâu của hệ thống.



Hình 3.13. Giao diện tổng quan hệ thống

3.2.2 Giao diện quản lý danh mục

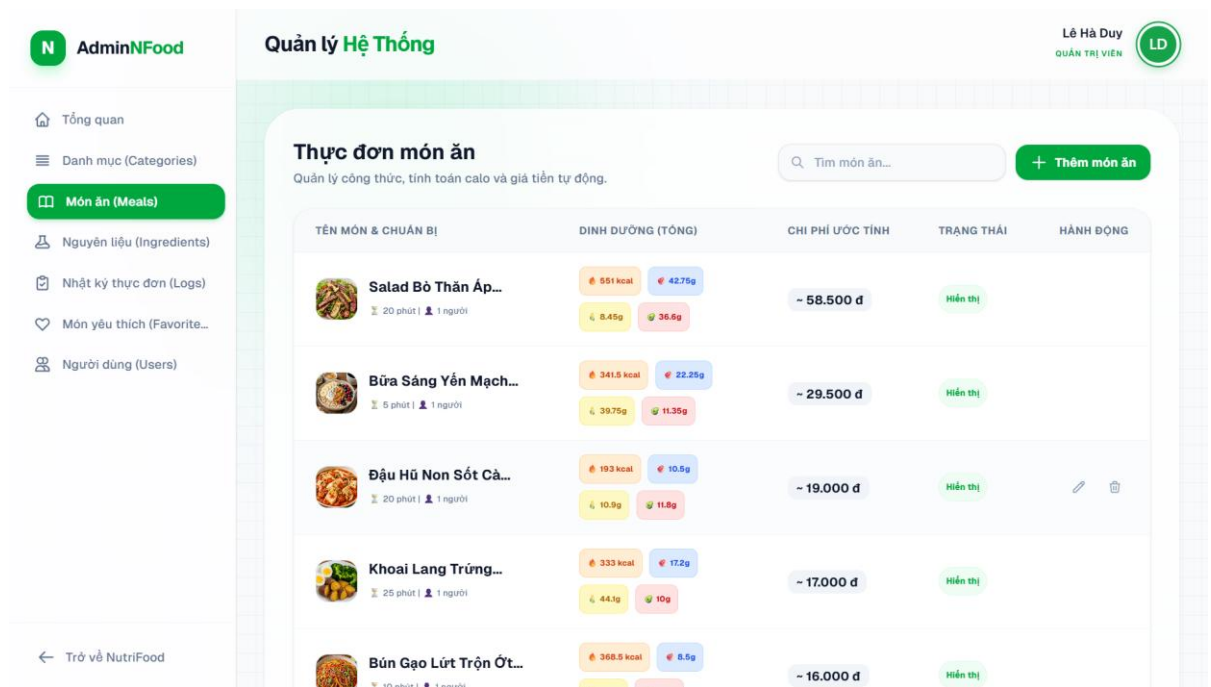
Giao diện hỗ trợ quản trị viên quản lý tập trung các danh mục món ăn trên hệ thống. Thông tin được hiển thị dưới dạng bảng dữ liệu trực quan với tên danh mục, mô tả và trạng thái hoạt động. Quản trị viên có thể tìm kiếm, phân trang, thêm mới và chỉnh sửa danh mục dễ dàng. Đồng thời, trạng thái hiển thị của từng nhóm thực đơn cũng có thể được cập nhật nhanh chóng. Điều này giúp quá trình phân loại và quản lý dữ liệu trở nên hiệu quả hơn.



Hình 3.14. Giao diện quản lý danh mục

3.2.3 Giao diện quản lý món ăn

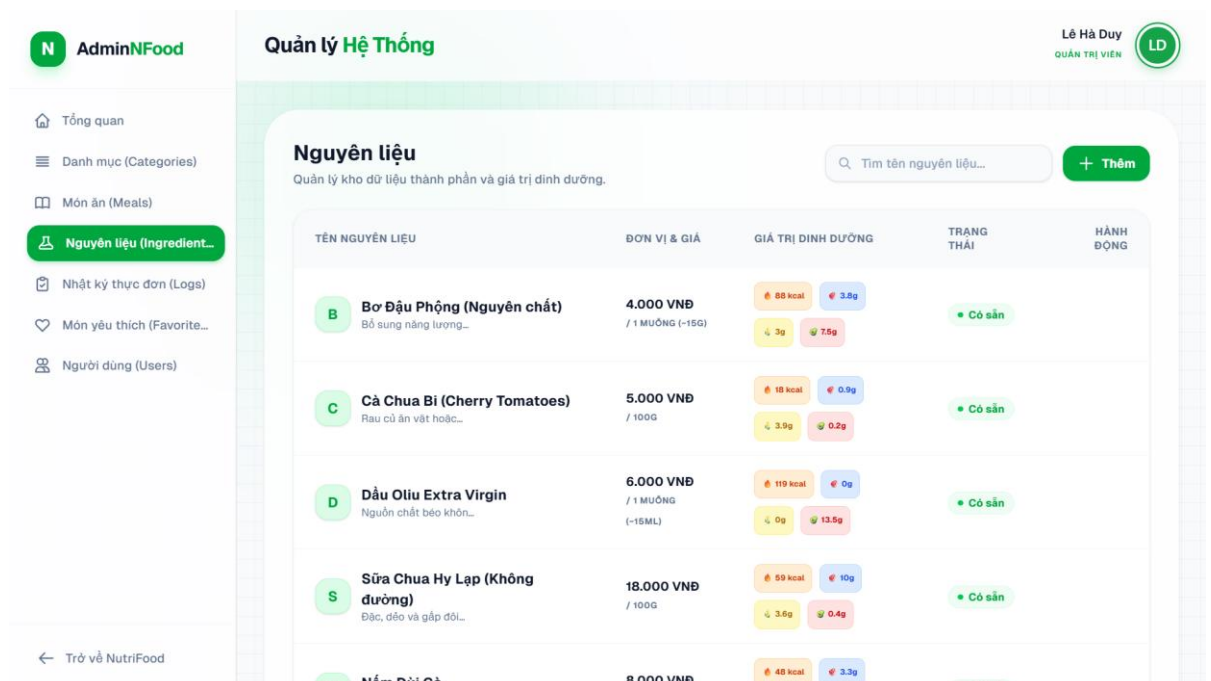
Giao diện hỗ trợ quản trị viên quản lý các món ăn và công thức chế biến trong hệ thống. Bảng dữ liệu hiển thị thông tin về giá trị dinh dưỡng, chi phí, thời gian chế biến, số khẩu phần và trạng thái món ăn. Quản trị viên có thể tìm kiếm, thêm mới, chỉnh sửa hoặc xóa dữ liệu thực đơn. Các thao tác được thực hiện trực tiếp thông qua các nút chức năng trên giao diện. Điều này giúp việc quản lý dữ liệu món ăn trở nên thuận tiện và hiệu quả hơn.



Hình 3.15. Giao diện quản lý món ăn

3.2.4 Giao diện quản lý nguyên liệu

Giao diện hỗ trợ quản trị viên quản lý tập trung dữ liệu nguyên liệu của hệ thống. Bảng thông tin hiển thị tên nguyên liệu, giá trị dinh dưỡng, giá tham khảo và trạng thái sử dụng. Quản trị viên có thể dễ dàng tìm kiếm, thêm mới và cập nhật các chỉ số liên quan. Các dữ liệu này được sử dụng làm cơ sở cho việc tính toán giá trị dinh dưỡng của món ăn. Nhờ đó, hệ thống đảm bảo tính chính xác và đồng nhất của dữ liệu.



Hình 3.16. Giao diện quản lý nguyên liệu

3.2.5 Giao diện quản lý nhật ký thực đơn

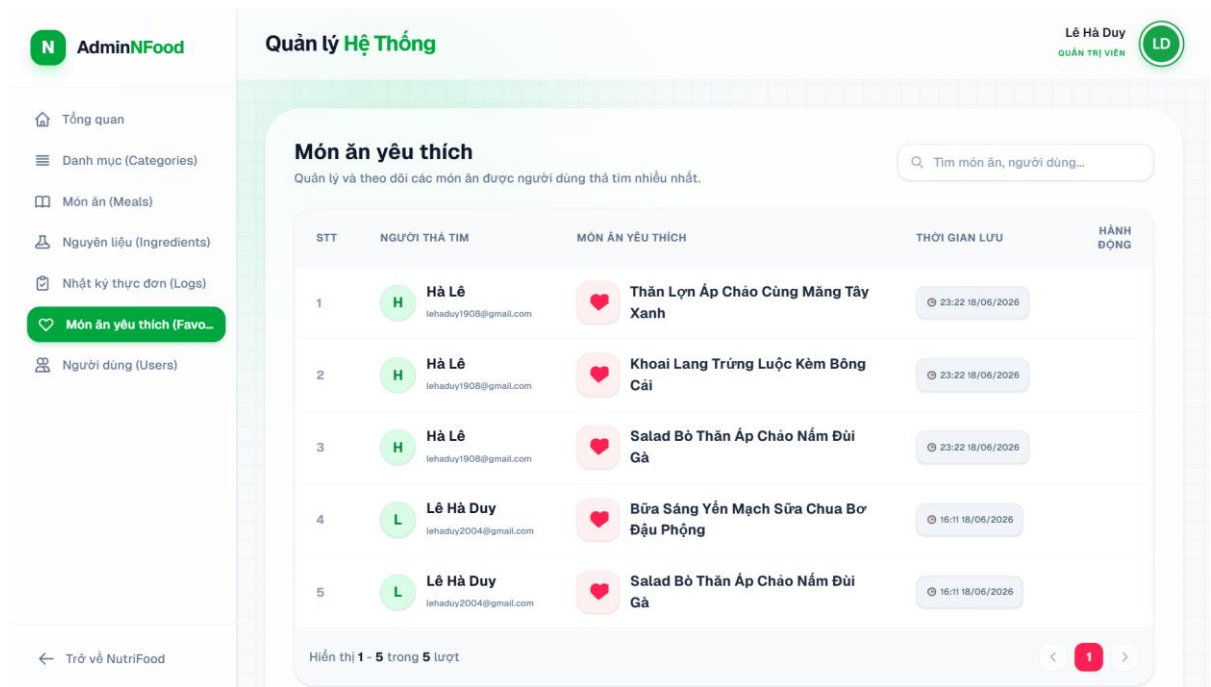
Giao diện hỗ trợ quản trị viên theo dõi và quản lý lịch sử ăn uống của người dùng trên hệ thống. Bảng dữ liệu hiển thị thông tin về món ăn, người dùng, loại bữa ăn, giá trị dinh dưỡng và thời gian ghi nhận. Quản trị viên có thể tìm kiếm dữ liệu theo món ăn hoặc tài khoản người dùng. Đồng thời, hệ thống cho phép xóa các bản ghi không hợp lệ nhằm đảm bảo tính chính xác của cơ sở dữ liệu. Nhờ đó, việc kiểm tra và quản lý nhật ký thực đơn trở nên thuận tiện hơn.

STT	THÔNG TIN NHẬT KÝ	DINH DƯỠNG (ĐÃ TIÊU THỤ)	KHẨU PHẦN & THỜI GIAN	HÀNH ĐỘNG
1	B Bữa Sáng Yến Mạch... BỮA PHỤ bởi Hà Lê	341.5 kcal, 22.25g, 39.75g, 11.35g	1 phần 23:27:31 18/06/2026	
2	Đ Đậu Hũ Non Sốt Cà... BỮA TỐI bởi Hà Lê	193 kcal, 10.5g, 10.9g, 11.8g	1 phần 23:27:30 18/06/2026	
3	S Salad Quinoa Đậu Gà... BỮA TRƯA bởi Hà Lê	568.5 kcal, 24g, 97.6g, 9.15g	1 phần 23:27:29 18/06/2026	
4	S Salad Quinoa Đậu Gà... BỮA SÁNG bởi Hà Lê	568.5 kcal, 24g, 97.6g, 9.15g	1 phần 23:27:29 18/06/2026	
5	B Bữa Sáng Yến Mạch... BỮA PHỤ bởi Hà Lê	341.5 kcal, 22.25g, 39.75g, 11.35g	1 phần 16:22:02 18/06/2026	

Hình 3.17. Giao diện quản lý nhật ký thực đơn

3.2.6 Giao diện quản lý món ăn yêu thích

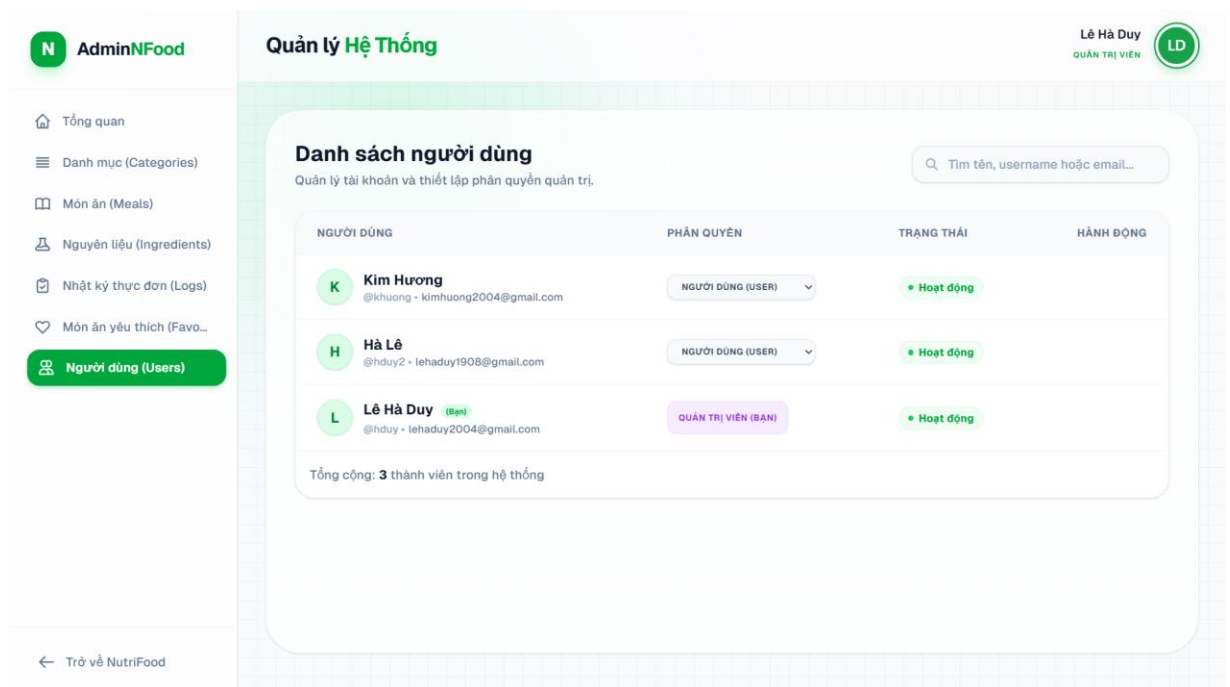
Giao diện hỗ trợ quản trị viên theo dõi và quản lý danh sách món ăn yêu thích của người dùng trên hệ thống. Bảng dữ liệu hiển thị thông tin về người dùng, món ăn được lưu và thời gian tương tác. Quản trị viên có thể tìm kiếm và phân trang dữ liệu một cách thuận tiện. Phần hệ này giúp đánh giá mức độ quan tâm của người dùng đối với từng món ăn. Qua đó, hỗ trợ tối ưu hóa nội dung và nâng cao hiệu quả gợi ý thực đơn.



Hình 3.18. Giao diện quản lý món ăn yêu thích

3.2.7 Giao diện quản lý người dùng

Giao diện hỗ trợ quản trị viên quản lý tập trung toàn bộ tài khoản người dùng trên hệ thống thông qua việc hiển thị các thông tin như họ tên, tên đăng nhập, email, vai trò và trạng thái tài khoản. Tại đây, quản trị viên có thể tìm kiếm, theo dõi số lượng thành viên cũng như thực hiện thay đổi quyền hạn giữa User và Admin nhằm đảm bảo tính bảo mật và phân quyền hiệu quả trong quá trình vận hành hệ thống.



Hình 3.19. Giao diện quản lý người dùng

CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Kết quả đạt được

Nghiên cứu và làm chủ công nghệ: Tìm hiểu, làm chủ và ứng dụng thành thạo mô hình MERN Stack bao gồm: ReactJS (kết hợp Vite và Tailwind CSS) để xây dựng giao diện người dùng; Node.js và ExpressJS để xử lý toàn bộ logic nghiệp vụ, xây dựng hệ thống RESTful API phía máy chủ; kết nối, tổ chức và quản lý dữ liệu linh hoạt, chặt chẽ với hệ quản trị cơ sở dữ liệu phi cấu trúc MongoDB (thông qua Mongoose ODM).

Ứng dụng Trí tuệ nhân tạo (AI): Nghiên cứu và tích hợp thành công các Mô hình ngôn ngữ lớn (LLMs) thông qua API của Google Gemini để hiện thực hóa phân hệ trí tuệ nhân tạo, xây dựng thành công Trợ lý ảo tư vấn trực tuyến và module tự động hóa thiết kế thực đơn khoa học.

Quy trình phát triển khoa học: Thực hiện nghiêm túc và đúng chuẩn quy trình kỹ nghệ phần mềm từ khâu khảo sát, phân tích yêu cầu, thiết kế hệ thống (xây dựng các sơ đồ khối Flowchart, sơ đồ Use Case, thiết kế đặc tả chi tiết 11 cơ sở dữ liệu bảng/collections) cho đến giai đoạn lập trình và kiểm thử.

Tối ưu trải nghiệm người dùng (UI/UX): Giao diện hệ thống được thiết kế hiện đại theo xu hướng tối giản, cấu trúc Typography tinh tế mang lại độ phản hồi trực quan cao. Đảm bảo tính tương thích hiển thị chuẩn xác (Responsive Web Design) trên đa dạng thiết bị như máy tính (PC), máy tính bảng và điện thoại di động.

Triển khai toàn diện nghiệp vụ cốt lõi: Xây dựng cấu trúc phân quyền rõ ràng (User/Admin) và vận hành ổn định các phân hệ chức năng:

- **Phía người dùng (Client):** Đăng ký/Đăng nhập (bảo mật qua cơ chế băm mật khẩu và mã hóa phiên bằng JWT/Refresh Token); Thiết lập "Hồ sơ sức khỏe số" (tự động tính toán các chỉ số cơ thể BMI, BMR, TDEE và mục tiêu thể trạng); Tra cứu và quản lý danh mục món ăn yêu thích; Quản lý Nhật ký ăn uống hằng ngày; Khám phá Lộ trình dinh dưỡng tuần cá nhân hóa bằng AI (tích hợp tính năng ghi nhận nhanh 1-chạm); Tương tác trực tuyến bằng ngôn ngữ tự nhiên với Trợ lý dinh dưỡng AI Chatbot.
- **Phía quản trị viên (Admin):** Hệ thống Bảng điều khiển (Dashboard) thống kê tổng quan hoạt động theo thời gian thực; Thực hiện trọn vẹn các nghiệp vụ quản lý chuyên sâu thông qua các chức năng CRUD đối với Tài khoản

người dùng, Danh mục món ăn, Công thức thực đơn thực phẩm, Kho dữ liệu thành phần nguyên liệu và hệ thống kiểm toán Nhật ký ăn uống toàn nền tảng.

4.2 Ưu nhược điểm

4.2.1 Ưu điểm

Tính cá nhân hóa cao: Hệ thống không chỉ cung cấp thông tin chung chung mà có khả năng phân tích sâu sắc thể trạng, mục tiêu sức khỏe và mức ngân sách tài chính của từng cá nhân để đưa ra các đề xuất thực đơn tối ưu nhất.

Độ tin cậy và minh bạch: Cơ sở dữ liệu về món ăn và nguyên liệu được chuẩn hóa rõ ràng về hàm lượng vi chất dinh dưỡng (Calories, Protein, Carbs, Fat) cùng giá cả, giúp người dùng dễ dàng kiểm soát năng lượng nạp vào cơ thể một cách khoa học.

Kiến trúc hệ thống mạnh mẽ và chịu lỗi tốt: Áp dụng kiến trúc Client-Server hiện đại giúp hệ thống có hiệu năng cao, tốc độ phản hồi API cực nhanh. Đặc biệt, hệ thống đã trang bị cơ chế dự phòng chủ động (Fallback Mechanism) – tự động nỪi suy và trích xuất thực đơn thay thế từ cơ sở dữ liệu nội bộ trong trường hợp API AI quốc tế gặp sự cố, đảm bảo trải nghiệm người dùng luôn liền mạch.

4.2.2 Nhược điểm

Môi trường triển khai hạn chế: Hệ thống hiện tại mới chỉ vận hành và kiểm thử trong môi trường phát triển cục bộ (Local Environment), chưa được triển khai thực tế trên các nền tảng đám mây công cộng (Cloud Hosting).

Dữ liệu ẩm thực chưa bao phủ sâu: Kho cơ sở dữ liệu thành phần nguyên liệu và món ăn tuy đã được thiết lập chuẩn hóa nhưng quy mô còn ở mức độ phục vụ nghiên cứu và học tập, cần được tiếp tục mở rộng để đáp ứng sự đa dạng vùng miền.

Thiếu các liên kết tự động nâng cao: Hệ thống chưa tích hợp tự động các cổng thanh toán điện tử (như VNPay, MoMo) cho các gói dịch vụ cao cấp, chưa có hệ thống tự động gửi thông báo nhắc nhở tiến độ dinh dưỡng qua SMS/Email mà hiện tại mới chỉ giới hạn ở thông báo nội bộ (In-app notification) trên giao diện web.

4.3 Hướng phát triển

Mở rộng và tối ưu cơ sở dữ liệu: Tiếp tục bổ sung và làm phong phú kho dữ liệu thô ẩm thực, tích hợp công nghệ Computer Vision (Thị giác máy tính) vào camera của thiết bị để tự động nhận diện thành phần nguyên liệu qua hình ảnh, nâng cao chất lượng đầu vào cho hệ thống.

Triển khai thực tế và nâng cấp kiến trúc: Đưa hệ thống vận hành chính thức lên các dịch vụ đám mây (như AWS, Google Cloud hoặc Vercel/Render); nghiên cứu tích hợp kỹ thuật RAG (Retrieval-Augmented Generation) để Trợ lý AI có thể truy xuất chính xác và nhanh chóng hơn nữa kho dữ liệu món ăn nội bộ.

Tích hợp các phân hệ dịch vụ bổ trợ: Mở rộng các tính năng nâng cao như liên kết công thanh toán trực tuyến tự động; thiết lập công gửi tin nhắn SMS/Email tự động để nhắc nhở người dùng uống nước, ghi nhật ký ăn uống đúng giờ hoặc thông báo khi AI đã khởi tạo xong lộ trình tuần mới.

Phát triển đa nền tảng: Tận dụng tối đa cấu trúc RESTful API hiện tại để xây dựng và phát triển thêm phiên bản ứng dụng di động (Mobile Application) sử dụng nền tảng React Native, mang lại sự tiện lợi vượt trội cho người dùng trong việc theo dõi sức khỏe và dinh dưỡng mọi lúc mọi nơi.

DANH MỤC TÀI LIỆU THAM KHẢO

Sách/ Tài liệu

- [1] Đoàn Phước Miên, Phạm Thị Trúc Mai (2014). *Thiết kế và lập trình Web*. Trường Đại học Trà Vinh.
- [2] Hà Thị Thúy Vi (2013). *Cơ sở dữ liệu*. Trường Đại học Trà Vinh.
- [3] Phạm Minh Dương (2014). *Phân tích và thiết kế hệ thống thông tin*. Trường Đại học Trà Vinh.

Website

- [4] Tài liệu kỹ thuật nền tảng ReactJS: <https://react.dev/learn> [ngày truy cập: 03/05/2026].
- [5] Tài liệu kỹ thuật Backend Node.js và Express Framework: <https://nodejs.org/en/docs/>, <https://expressjs.com/> [ngày truy cập: 03/05/2026].
- [6] Tài liệu hệ quản trị cơ sở dữ liệu phi cấu trúc MongoDB: <https://www.mongodb.com/> [ngày truy cập: 04/05/2026].
- [7] Tài liệu hướng dẫn sử dụng HTTP RESTful API kết nối Mô hình ngôn ngữ lớn (LLMs): <https://ai.google.dev/docs>, <https://platform.openai.com/docs/api-reference> [ngày truy cập: 05/05/2026].
- [8] Các tài liệu học thuật về thuật toán Lọc cộng tác và Lọc theo nội dung: <https://developers.google.com/machine-learning/recommendation> [ngày truy cập: 06/05/2026].
- [9] “ReactJS - Ưu điểm và nhược điểm”: <https://viblo.asia/p/reactjs-uu-diem-va-nhuoc-diem-V3m5WzexlO7> [ngày truy cập: 16/05/2026].
- [10] “ExpressJS là gì? Hướng dẫn học ExpressJS chi tiết từ A-Z”: <https://codegym.vn/blog/expressjs-la-gi-huongdan-cach-hoc-expressjs-chi-tiet/> [ngày truy cập: 15/05/2026]
- [11] “NodeJS là gì? Tổng quan kiến thức về Node.JS”: <https://vietnix.vn/nodejs-la-gi/> [ngày truy cập: 15/05/2026].
- [12] “RESTful API là gì? Cách thiết kế RESTful API”: <https://topdev.vn/blog/restful-api-la-gi/> [ngày truy cập: 17/05/2026].

- [13] “JavaScript là gì? Những điều cần biết về JavaScript”:
<https://viblo.asia/p/javascript-la-ginhung-dieu-can-biet-ve-javascript-eW65G4RRKDO>
[ngày truy cập: 06/05/2026].
- [14] “MongoDB là gì? Các tính năng nổi bật của MongoDB”:
<https://viblo.asia/p/mongodb-la-gicac-tinh-nang-noi-bat-cua-mongodb-PAoJex2N41j>
[ngày truy cập: 16/05/2026].